

zkRAG: Efficiently Proving RAG Retrieval in Zero Knowledge

Yanze Jiang*
yanzejiang@u.nus.edu
National University of Singapore
Singapore

Xinyang Yang*
e1374509@u.nus.edu
National University of Singapore
Singapore

Xuanming Liu*
hinsliu@zju.edu.cn
Zhejiang University
China

Yanpei Guo
guo.yanpei@u.nus.edu
National University of Singapore
Singapore

Jiaheng Zhang†
jhzhzhang@nus.edu.sg
National University of Singapore
Singapore

Abstract

Retrieval-augmented generation (RAG) systems critically depend on a vector-retrieval stage that selects relevant documents from a large embedding database. In a RAG-as-a-Service setting, however, clients have no visibility into the server’s proprietary embeddings and index structures, and thus cannot distinguish faithful execution from arbitrary deviations. This motivates *service consistency* for retrieval: the returned results must be consistent with executing an agreed approximate nearest neighbor search (ANNS) algorithm on a fixed, committed database together with a fixed, committed ANNS index, while revealing nothing beyond the outputs themselves.

We present the first polynomial interactive oracle proof (PIOP) tailored to the widely deployed HNSW ANNS algorithm. Building on this PIOP, we introduce zkRAG, a zero-knowledge, succinct, non-interactive argument for RAG retrieval that enables practical verification. Our design achieves asymptotically optimal *online* prover efficiency, with prover time linear in the HNSW search trace length, while keeping verification succinct. We introduce several new techniques that may be of independent interest, including a hybrid lookup argument, a highly efficient checker-based PIOP for checking priority-queue updates, and an efficient checker for membership selector vectors. For a benchmark with 10^6 vectors of dimension 128, single-thread zkRAG proves a typical HNSW query in 50 seconds—over 1000× faster than existing baselines—while keeping verification lightweight, demonstrating the feasibility of efficient zero-knowledge service-consistent RAG retrieval.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

Zero-knowledge proofs, RAG, verifiable vector retrieval, approximate nearest neighbors

1 Introduction

Retrieval-augmented Generation (RAG) has emerged as a dominant paradigm for deploying Large Language Models (LLMs) in real-world applications [41]. By augmenting model inference with documents retrieved from an external knowledge database, RAG enables LLMs to generate grounded, up-to-date, and domain-specific responses without requiring frequent retraining. This architecture

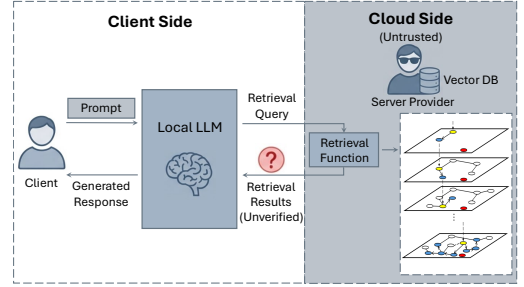


Figure 1: The illustration of cloud-hosted RAG systems. The retrieval function is typically an ANNS, whose results are untrusted, creating a trust gap that this work aims to address.

has been widely adopted, including enterprise knowledge assistants [61], legal and financial analysis tools [54, 58], customer support automation [60], and scientific data exploration [39].

RAG *retrieval* is a critical component that largely determines the effectiveness of such a system: with an *embedding vector* at hand, it queries an external database and fetches a set of *highly relevant* documents (also represented as embedding vectors), which the LLM uses as context to generate high-quality responses. In practice, RAG retrieval typically follows a procedure known as Approximate Nearest Neighbor Search (ANNS) [41], which aims to return the most relevant documents from the database based on the *distance* between their embedding vectors and the query vector. Among various ANNS algorithms, a graph-based method called HNSW [46] is the most widely used due to its combination of exceptional search speed and high retrieval accuracy [1, 46]. Since the retrieved documents serve as the factual basis for LLM generation, any incorrect, incomplete, or adversarially manipulated retrieval results can fundamentally compromise or mislead the output, even if the downstream LLM is correct [18, 36].

In practice, RAG systems are commonly deployed as *cloud-hosted services* [4, 31, 42, 48]. In this paradigm, clients—often individual users or small teams who host or access a standard LLM (e.g., by downloading from platforms such as HuggingFace [35])—rely on a service provider that maintains a domain-specific vector database and handles RAG retrievals on demand. The returned documents (or their embeddings) are then used as context for the client’s own LLM generation. This *RAG-as-a-Service* paradigm [49] significantly lowers the barrier to retraining or deploying domain-grounded

*These authors contributed equally to this work.

†Corresponding author.

LLM assistants, especially for users who lack the resources or infrastructure to host and manage proprietary knowledge bases. See Fig. 1 for an illustration of this paradigm.

However, there is a fundamental trust gap: clients have no direct visibility into the service provider’s database, index structures, or execution process, and thus cannot distinguish faithful retrieval from arbitrary deviations. As a result, an untrusted provider may deviate from the agreed retrieval algorithm for reasons such as better throughput, cost reduction, or business incentives—for example, by returning cached, truncated, biased, or even arbitrary results instead of faithfully executing the prescribed ANNS procedure. For instance, Microsoft reported a case in Copilot Studio where two users issued identical queries over the same internal document yet received divergent RAG answers, because the RAG retrieval process returned different document chunks, causing the LLM to hallucinate when the relevant section was not retrieved [47]. Such deviations are difficult for clients to detect, yet they may directly undermine the integrity and reliability of the client’s LLM outputs, highlighting the trust gap and the need for verifiable and reliable RAG retrieval from the perspective of end clients.

Research gap. A seemingly straightforward approach is to employ general-purpose zkSNARKs [15, 28, 32, 53, 59, 65] to prove the correct execution of RAG retrieval. With a *general* zkSNARK, the service provider can generate a proof that allows the client to *succinctly* verify the correctness of RAG retrieval, *without revealing any information* about its database—except for the retrieval results themselves. However, despite recent advances in zkSNARKs, directly encoding modern ANNS algorithms such as HNSW into general-purpose zkSNARK circuits still incurs prohibitive prover time and memory overhead, rendering existing approaches impractical for real-world services [29]. This is primarily because such algorithms often rely on complex data structures like graphs and heaps, irregular control flow, and adaptive search heuristics, which are features that are either too complex to compile efficiently or remain underexplored in existing zkSNARK research. Besides, recent work [51, 56] has made significant progress in designing *tailored* zkSNARKs for LLM inference (zkLLM). However, these approaches focus exclusively on standard LLMs such as GPT-2, and do not address the verifiability of the RAG retrieval process.

Therefore, in this work, we take a first step toward designing a *tailored zkSNARK for ANNS (HNSW)*, aiming to fill this research gap and efficiently generate proofs for the RAG retrieval process.

1.1 Our contributions

The contributions are summarized as follows:

zkRAG: tailored proofs for RAG retrieval via HNSW. To the best of our knowledge, we are the first to investigate *zero-knowledge verifiable RAG retrieval*. To realize zkRAG, we focus on the dominant practical instantiation of RAG retrieval: ANNS over a vector database, for which HNSW [46] is the most widely used [1, 46]. Our primary technical contribution is a tailored (*zero-knowledge*) *polynomial interactive oracle proof* for HNSW query execution, enabling efficient proof generation for RAG retrieval without leaking additional information. In the online phase, zkRAG achieves optimal prover time $O(\ell_q)$, where ℓ_q is the number of database entries actually visited during the search (typically growing logarithmically

with the database size N). For the client, zkRAG provides succinct verification time and proof size, making it practical for deployment.

Techniques. Along the way to constructing zkRAG, we combine new theoretical ideas with low-level techniques to address the challenges encountered, which may be of independent interest: (i) The first challenge is how to commit to the complex HNSW graph and check its consistency during the online phase without introducing significant overhead (e.g., $O(N)$). To address this, we build on ideas from previous work [23] and construct a hybrid lookup argument to achieve optimal online complexity for the prover. (ii) The second challenge lies in dealing with dynamic priority queue updates. The HNSW algorithm needs to maintain complex data structures such as heaps, which undergo frequent insertions and deletions. Simulating these operations in circuits incurs high costs due to the complexity of heap manipulation. In contrast, we propose a super-efficient *checker* for verifying priority queue updates and construct a PIOP for this checker, thereby achieving optimal prover efficiency. (iii) A third challenge is proving the selector vector representing “the membership in a set V ”. Naively maintaining the set V leads to $O(N)$ prover cost. We instead design a checker for this selector vector directly, eliminating costly set operations and reducing the prover complexity to $O(\ell_q)$ while preserving correctness. Finally, we also incorporate additional useful ideas, such as handling control flow (e.g., loops), and combine the above techniques to obtain zkRAG. See our technical overview (§3) for further details.

Implementation and evaluation. We evaluate zkRAG on the widely used ANNS benchmark SIFT1M [44]. In a database with $N = 10^6$ entries and data dimension $d = 128$, for an HNSW query with a search trace length $\ell_q \in (2^{11}, 2^{12}]$ (a range that covers the majority of practical query instances in this benchmark under the HNSW configuration $\text{efS} = 64$), the prover time is about 50s, which is about $1000\times$ faster than baselines implemented with HyperPlonk [24] and SP1 [2]. The verifier time is 1s, and the proof size is 0.5MB, which are also acceptable. We further evaluate the efficiency of individual components, such as the tailored PIOP for priority queue updates based on our super-efficient checker. In a benchmark consisting of 2^{10} insertions and deletions applied randomly to a heap, our protocol requires only about 4s of prover time. In contrast, proving the same sequence of operations using baseline approaches that simulate the heap directly in circuits incurs more than three orders of magnitude higher cost. Overall, these results demonstrate that zkRAG achieves very low overhead for both the prover and the verifier, making it practical for real-world systems.

Applications. The direct application of zkRAG is to provide an efficient solution that addresses the trust gap in cloud-hosted RAG systems (Fig. 1). Beyond this, zkRAG also applies to other settings where ANNS is a critical system component. For instance, modern recommendation systems and search engines also widely deploy HNSW as a core retrieval backend [20, 37, 40]; zkRAG can be applied to enable efficient and privacy-preserving verification of such systems’ execution, making it possible to detect any attempts to cut corners or deviate from prescribed behavior. Finally, zkRAG has the potential to be linked and integrated with zkLLMs [51, 56], enabling a fully trustless pipeline for RAG-enhanced LLMs.

2 Preliminary

Notation. We use λ to denote the security parameter, and $\text{negl}(\lambda)$ to denote a negligible function in λ . For an integer n , we denote the set $\{0, \dots, n-1\}$ by $[n]$. Bold letters (e.g., $\mathbf{a}$) represent bit-strings, vectors, or matrices. We denote the natural injection that maps an integer to its ℓ -bit binary representation by $\text{bin}_\ell(\cdot)$. For a vector $\mathbf{a}$, we write a_i to denote its i -th element. For a matrix $\mathbf{A}$, we write A_i to denote its i -th row, $A[i]$ to denote its i -th column. We write $A_{[i:j]}$ to denote the submatrix formed by rows i to j (inclusive) and $A[i : j]$ to denote the submatrix formed by columns i to j (inclusive). We define element-wise comparison for matrices and vectors as follows. For matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{R}^{a \times b}$, we write $\mathbf{A} \leq \mathbf{B}$ if $A_i[j] \leq B_i[j]$ for all $i \in [a], j \in [b]$. $\mathbf{A} \geq \mathbf{B}$ is defined analogously. We define the Hadamard product $\mathbf{C} = \mathbf{A} \circ \mathbf{B}$ as $C_i[j] = A_i[j] \cdot B_i[j]$ for all $i \in [a], j \in [b]$. The same convention applies to vectors. For matrices $\mathbf{A} \in \mathbb{R}^{a \times b}$, $\mathbf{B} \in \mathbb{R}^{a \times c}$, and $\mathbf{C} \in \mathbb{R}^{a \times (b+c)}$, we write $\mathbf{C} = (\mathbf{A}, \mathbf{B})$ to denote column concatenation. The same applies to matrix-vector and vector-vector concatenation. Let $\mathbb{F}$ be a finite field of prime order such that $|\mathbb{F}|^{-1} = \text{negl}(\lambda)$. Let $\mathbb{G}$ and $\mathbb{G}_t$ be groups with generator g , and let $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$ be an efficiently computable non-degenerate pairing. We write group operations additively and denote a group element as $[x]_g := x \cdot g$.

Encoding. We usually encode a vector $\mathbf{v}$ as a polynomial such that its evaluation over a specified domain equals $\mathbf{v}$. More specifically, in this work, we consider two encoding methods.

Univariate (Lagrange) extension. A vector $\mathbf{v}$ of size N can be encoded by its Lagrange extension, i.e., a univariate polynomial $v(X) = \sum_{i=0}^{N-1} v_i L_i(X)$, where $\{L_i(X)\}$ are the Lagrange polynomials defined over the N -th roots of unity Ω_N . Let $Z(X) = X^N - 1$ denote the vanishing polynomial over Ω_N .

Multilinear extension (MLE). A polynomial is called *multilinear* if it is multivariate, in which the degree of each variable is at most one. A vector $\mathbf{v}$ of size 2^ℓ can be encoded as its MLE, i.e., a multilinear polynomial $v : \mathbb{F}^\ell \rightarrow \mathbb{F}$ such that $v(\mathbf{b}) = v_{\text{bin}_\ell^{-1}(\mathbf{b})}$ for all $\mathbf{b} \in \{0, 1\}^\ell$. Specifically, v is expressed as $v(\mathbf{x}) = \sum_{\mathbf{b} \in \{0, 1\}^\ell} \tilde{e}q(\mathbf{x}, \mathbf{b}) \cdot v_{\text{bin}_\ell^{-1}(\mathbf{b})}$, where $\tilde{e}q(\mathbf{x}, \mathbf{b}) = \prod_{i=1}^\ell ((1 - x_i)(1 - b_i) + x_i b_i)$, and b_i denotes the i -th bit of $\mathbf{b}$. For any $\mathbf{r} \in \mathbb{F}^\ell$, $v(\mathbf{r})$ can be computed in $O(2^\ell)$ time.

2.1 Backgrounds of zkSNARK

Succinct Non-interactive ARgument of Knowledge (SNARK) for the satisfiability of an NP relation $\mathcal{R}(x; w) = 1$ is a tuple of algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$. The prover, on the public input x and the witness w , runs $\mathcal{P}$ to generate a concise proof π for the relation, which can later be checked by $\mathcal{V}$ with only the public input. A SNARK satisfies completeness, knowledge soundness, and succinctness; A zkSNARK additionally satisfies a property called zero knowledge. We provide formal definitions for these properties in §A. Modern zkSNARK [15, 19, 28, 53] constructions often follow a recipe that compiles a Polynomial Interactive Oracle Proof (PIOP) using a Polynomial Commitment Scheme (PCS), and finally applies the Fiat-Shamir transformation [25] to render it non-interactive.

Polynomial Interactive Oracle Proof. A PIOP [15, 19, 28, 53] is a public-coin interactive protocol that proceeds in multiple rounds, where in each round $\mathcal{P}$ sends several polynomial oracles to $\mathcal{V}$, who checks the relation $\mathcal{R}$ by querying the oracles and issuing back

some random challenges to proceed. In practice, the polynomial oracles are instantiated with PCS.

Polynomial Commitment Scheme. A PCS [38, 50] allows $\mathcal{P}$ to commit to a polynomial f as com_f and later prove the correctness of an evaluation $v := f(u)$ at a $\mathcal{V}$ -chosen point u . The formal definitions are provided in §A. A PCS satisfies completeness, binding, and knowledge soundness.

Useful gadgets. In this work, we primarily encode the computation trace of the HNSW query as multilinear polynomials. Seminal prior work [15] developed efficient sub-PIOPs and their zero-knowledge version for a wide range of relations over multivariate polynomials. These gadgets are particularly helpful for constructing zkRAG:

- **Sumcheck:** Given an ℓ -variate polynomial f and a claim H , $\mathcal{R}_{\text{sum}}(\ell, H; f) = 1$ iff $H = \sum_{\mathbf{b} \in \{0, 1\}^\ell} f(\mathbf{b})$.
- **Zerocheck:** Given f , $\mathcal{R}_{\text{zero}}(\ell; f) = 1$ iff $f(\mathbf{b}) = 0$ for all $\mathbf{b} \in \{0, 1\}^\ell$.
- **Prodcheck:** Given f and a claim H , $\mathcal{R}_{\text{prod}}(\ell, H; f) = 1$ iff $H = \prod_{\mathbf{b} \in \{0, 1\}^\ell} f(\mathbf{b})$.
- **Permcheck:** Given f, g , and a public permutation mapping $\sigma : \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$, $\mathcal{R}_{\text{perm}}(\ell, \sigma; f, g) = 1$ iff for all $\mathbf{b} \in \{0, 1\}^\ell$, $f(\mathbf{b}) = g(\sigma(\mathbf{b}))$.

The constructions of the sub-PIOP gadgets are included in §B. For all the gadgets above, [15] shows that the complexity of $\mathcal{P}$ and $\mathcal{V}$ is $O(2^\ell)$ and $O(\ell)$, respectively, and the proof size is $O(\ell)$, excluding the cost of invoking the PCS.

Lookup argument. Given a public table $\mathbf{t} \in \mathbb{F}^N$ and a query vector $\mathbf{q} \in \mathbb{F}^L$, a *lookup argument* [62] is a specialized PIOP for the relation $\mathcal{R}_{\text{lookup}}(N, L; \mathbf{t}, \mathbf{q}) = 1$ iff $\mathbf{q} \subset \mathbf{t}$, where $\subset$ denotes that each element in $\mathbf{q}$ appears in $\mathbf{t}$. There are various implementations of the lookup argument [23, 27, 34, 55]. A useful scheme is LogUp [34], which we adopt in this work when the table is temporary and of small size. The complexity of $\mathcal{P}$ and $\mathcal{V}$ in this PIOP is $O(N + L)$ and $O(\log(NL))$, respectively, and the proof size is $O(\log(NL))$. We provide the construction of LogUp in §B. Prior work [34] shows how to extend a lookup argument to a *batched* lookup argument: Given a public matrix table $\mathbf{t} \in \mathbb{F}^{N \times d}$ and a query $\mathbf{q} \in \mathbb{F}^{L \times d}$, $\mathcal{R}_{\text{batchlookup}}(N, L, d; \mathbf{t}, \mathbf{q}) = 1$ iff $\mathbf{q} \subset \mathbf{t}$, where $\subset$ means that each row of $\mathbf{q}$ appears in the table $\mathbf{t}$. The lookup argument is useful for checking complex algebraic relations such as *range checks* [12]: $\mathcal{R}_{\text{range}}(L, a, b; \mathbf{q}) = 1$ iff all q_i lie in $[a, b]$. This can be done by building a table $\mathbf{t} = \{a, \dots, b\}$ and proving $\mathbf{q} \subset \mathbf{t}$ through a lookup argument. Batched lookup argument enables *indexed lookup argument* [55]: Given $\mathbf{id} \in \mathbb{F}^L$, $\mathcal{R}_{\text{idlookup}}(N, L; \mathbf{t}, \mathbf{q}, \mathbf{id}) = 1$ iff $q_i = t_{\mathbf{id}[i]}$ for all $i \in [L]$. This is reduced to a batched lookup argument for $(\mathbf{q}, \mathbf{id}) \subset (\mathbf{t}, \mathbf{t}_{\mathbf{id}})$, where $\mathbf{t}_{\mathbf{id}} = \{0, \dots, N-1\}$.

3 Technical Overview

Given a database $\mathcal{D}$ with N vectors, each of which is referred to as an *embedding*, and an input query vector $\mathbf{q}$, the ANNS algorithm aims to find several closest nodes to $\mathbf{q}$, where the distance is measured using the *inner-product* distance. We denote the embedding dimension by d , and throughout the paper all embeddings are from the vector space $\mathbb{F}^d$. In practice, if real numbers are involved, we can always map them to $\mathbb{F}$ with standard quantization [43].

Unlike naive brute-force search, which requires an impractical $O(N)$ time (especially when N is large), most ANNS algorithms,

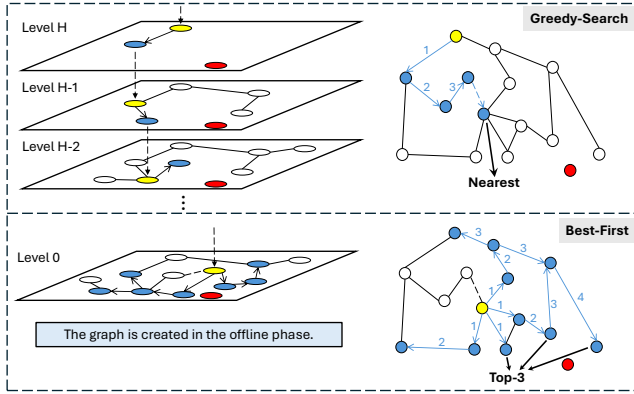


Figure 2: Illustration of the HNSW query process. The left panel depicts the hierarchical graph structure: the query starts from the top level, performs greedy search down to Level 1, and then proceeds with a best-first search at Level 0. Yellow nodes denote entry points, the red node is the query, and blue nodes are the visited results. Arrows indicate the search path. The right panel highlights the greedy search at Level 1 and the subsequent transition into the Level 0 best-first search. Edge labels indicate visiting order, while dashed lines signify that many nodes and edges are omitted for clarity.

such as HNSW, do *not* visit all N entries to find the result. Instead, they preconstruct useful data structures (such as graphs), and only a small subset of ℓ_q entries ($\ell_q \ll N$) is actually accessed for each query q . In this work, we design a **succinct** (zero-knowledge) PIOP for HNSW query with **optimal prover time** $O(\ell_q)$.

Threat model. We consider a setting where the service provider maintains a database $\mathcal{D}$ along with a pre-constructed HNSW graph for retrieval, and promises to offer RAG retrieval service to client(s) using the HNSW algorithm. The service provider is considered potentially *malicious* and may arbitrarily deviate from the agreed algorithm when responding to client queries. The security goal is to ensure (i) *service consistency* for clients—namely, the results returned by the service provider faithfully reflect the prescribed algorithm executed on the claimed database and HNSW graph; and (ii) *privacy* for the service provider—the proof should not reveal any additional information about the database beyond the retrieval results. We formally define the properties of zkRAG in §C.

3.1 Hierarchical Navigable Small World

Revisit HNSW. In the offline phase, an HNSW graph is constructed upon the database $\mathcal{D}$, organized into $H + 1$ levels, where each embedding corresponds to a node (Fig. 2). All nodes appear in level 0, and higher levels are formed by sampling nodes from lower levels. We write $N^{(h)}$ to denote the number of nodes at level h . Within each level $h \in \{1, 2, \dots, H\}$, each node is connected to K neighbors, while at level 0, each node is connected to $2K$ neighbors, where K is a fixed hyperparameter. See §D for further construction details.

In the online phase, the overall query strategy (Fig. 2) can be divided into two procedures: (i) *greedy search* in the upper levels ($H \rightarrow 1$) and (ii) *best-first search* in level 0. The complete algorithm is in Alg. 1. In the upper levels, greedy search (Alg. 2) is performed hierarchically: starting from a designated entry point at the top level, the algorithm searches greedily within each level h to find

Algorithm 1 HNSW_QUERY($G, H, start, q, k, efS$)

- 1: **Input:** Graph G , graph max level H , entry node $start$, query q , result size k , queue size efS ($k \leq efS$)
 - 2: **Output:** Top- k approximate nearest neighbors
 - 3: $ep \leftarrow start$
 - 4: **for** $h = H$ **down to** 1 **do**
 - 5: $ep \leftarrow \text{GREEDYSEARCH}(G, h, ep, q)$
 - 6: $topk \leftarrow \text{BESTFIRSTSEARCH}(G, ep, q, k, efS)$
 - 7: **return** $topk$
-

the node closest to the query at that level. This node then serves as the entry point for level $h - 1$. Finally, upon reaching level 0, a best-first search (Alg. 3) is executed to finalize the query. This step is more complex and aims to identify a high-quality set of approximate nearest neighbors in the base level. Let q denote the query and $\ell_q^{(h)}$ denote the number of node visits (with repetition) at level h . The total node visits number is $\ell_q = \sum_{h=0}^H \ell_q^{(h)}$, which is empirically logarithmic in N [46].

Overall proof strategy. From $\mathcal{P}$'s perspective, we find the algorithm suggests a modular approach to constructing a proof.

Offline: commit to the HNSW graph. First of all, in the offline phase, $\mathcal{P}$ must commit to the structure of the HNSW graph. Recall that the graph is constructed from the database by adding a hierarchical structure and level-wise neighborhood relationships. We show how to commit to this as follows: $\mathcal{P}$ commits to the graph using two tables that record these relationships during the offline phase:

- **Embedding table $t_{\mathcal{D}}$:** This table represents the original embedding database $\mathcal{D}$. Each row i consists of an identifier $id_i \in \mathbb{F}$ and its corresponding embedding vector $emb_i \in \mathbb{F}^d$.
- **Hierarchy table $t_{\mathcal{H}}$:** This table encodes the hierarchical structure of the HNSW graph. Each row i consists of a level identifier $lid_i \in [H + 1]$, a node identifier $id_i \in \mathbb{F}$, and a vector nei_i , where nei_i denotes the list of neighbors connected to node id_i at level lid_i . For $lid_i > 0$, $nei_i \in \mathbb{F}^K$, while for $lid_i = 0$, $nei_i \in \mathbb{F}^{2K}$. We note that $t_{\mathcal{H}}$ contains $O(N)$ rows in total.

Online: lookup the committed graph. In the online execution of Alg. 1, a subset of ℓ_q nodes in $t_{\mathcal{D}}$ and their neighborhood relationships in $t_{\mathcal{H}}$ are used, which is called the *witness*. To prove the correctness of the query algorithm, $\mathcal{P}$ needs to show the consistency between the witness and the committed tables. An important observation is that this can be achieved through a *lookup argument*. Specifically, $\mathcal{P}$ generates a proof to show: (i) all nodes visited come from $t_{\mathcal{D}}$, and (ii) all neighborhood relationships used at each level match $t_{\mathcal{H}}$, which is sufficient to ensure that $\mathcal{P}$ cannot alter the HNSW graph or its topology. Here, a challenge is that generating such a proof typically incurs $O(N)$ complexity, which is undesirable. In §3.2, we show how to construct a lookup argument with $O(\ell_q)$ online prover time through *preprocessing* the tables.

Online: proving two kinds of search logic. Now, the remaining task is to check the well-formedness of the relation among the witnesses, which involves proving the correctness of the greedy search and the best-first search individually. The main challenge here is that, although only $O(\ell_q)$ nodes are involved, many operations (e.g., heaps) in these algorithms are not linear in complexity, making a direct “translation” of their correct execution inefficient. We address these

challenges separately and leverage techniques such as *checkers* to ensure an efficient prover. We present the corresponding solutions in § 3.3 and 3.4, and then combine them into a complete PIOP for the HNSW query, which can be instantiated with PCS to yield zkRAG.

3.2 Offline Commitment and Online Lookup

In the online phase, $\mathcal{P}$ first selects the $O(\ell_q)$ nodes and neighbor relations that are actually used during the query, and uses a lookup argument (§2.1) to prove that they indeed exist in the tables $\mathbf{t}_{\mathcal{D}}$ and $\mathbf{t}_{\mathcal{H}}$, which contain $O(N)$ rows. However, existing schemes fail to meet our efficiency goal: (i) Leading schemes like Logup [34] and Lasso [55] incur a prover complexity of $O(N + \ell_q)$, which is unacceptable when the table is extremely large, while (ii) Other schemes, such as CQ [14, 23, 66], employ an offline preprocessing phase to reduce the online prover complexity to $O(\ell_q \log \ell_q)$, independent of N , but this still exceeds our efficiency target.

Rethinking CQ and LogUp. Recall that both [23, 34] construct lookup arguments by checking the following equation:

$$\sum_{i=0}^{N-1} \frac{\mathbf{m}_i}{r + \mathbf{t}_i} = \sum_{i=0}^{\ell_q-1} \frac{1}{r + \mathbf{q}_i} \quad (1)$$

where $\mathbf{m}_i$ represents the number of times each table element $\mathbf{t}_i$ appears in the query $\mathbf{q} \in \mathbb{F}^{\ell_q}$, and r is a random challenge issued by $\mathcal{V}$. At a high level, checking Eq. (1) can be divided into three subtasks: (i) encoding the entries $\{\frac{\mathbf{m}_i}{r + \mathbf{t}_i}\}$ and $\{\frac{1}{r + \mathbf{q}_i}\}$ into two polynomials A and B , respectively; (ii) checking the well-formedness of A and B separately; and (iii) checking that the summations on A and B are equal. CQ removes the dependency on N in the online prover complexity by observing that in the LHS summation, at most ℓ_q terms are nonzero. A key insight is that this ℓ_q -sparsity allows the online prover cost for the LHS to be reduced to $O(\ell_q)$ when using *univariate* polynomials to encode the entries, at the expense of an $O(N \log N)$ offline preprocessing cost. However, CQ also encodes the RHS of Eq. (1) using univariate polynomials, which brings the prover cost of the RHS (and thus the total cost) to $O(\ell_q \log \ell_q)$. Instead, LogUp encodes both sides using *multilinear* polynomials, which makes it infeasible to utilize preprocessing to eliminate the dependency on N , and thus the prover cost remains $O(N + \ell_q)$.

Hybrid lookup argument. We combine the ideas of [23, 34] and propose a *hybrid* lookup argument that achieves $O(\ell_q)$ online prover complexity. The idea is conceptually simple yet essential for $O(\ell_q)$ online $\mathcal{P}$ complexity: Specifically, we use univariate polynomials to handle the LHS of Eq. (1), and multilinear polynomials to handle the RHS. In the offline phase, we apply the *cached quotients* technique from [23] to commit to and preprocess the table $\mathbf{t}$ (in practice, $\mathbf{t}_{\mathcal{D}}$ and $\mathbf{t}_{\mathcal{H}}$ separately), obtaining quotients $\{[Q_i]_g\}_{i \in [N]}$. This preprocessing requires $O(N \log N)$ time but only needs to be performed once. In the online phase: For the LHS, we encode the entries as a univariate polynomial $A(X)$. With the help of $\{[Q_i]_g\}$ from the offline phase, tasks (ii) and (iii) can be completed in $O(\ell_q)$ prover time. For the RHS, we instead extend the entries into a multilinear polynomial $B(x)$, which allows tasks (ii) and (iii) to also be completed in $O(\ell_q)$ time using PIOPs from §2.1. Hence, the total online prover time remains $O(\ell_q)$. The details are provided in §4.1.

Algorithm 2 GREEDYSEARCH(G, h, ep, q)

```

1: Input: Graph  $G$ , current level  $h$ , entry  $eq$ , query  $q$ 
2: Output: Local minimum node  $best$  at level  $h$ 
   //  $\text{DIST}(q, x.\text{emb})$  is the (inner-product) distance between  $q$  and node  $x$ 's
   // embedding vector  $x.\text{emb}$ 
   //  $\text{NEIGHBORS}(x, \text{level} = h)$  returns the neighbor set of  $x$  at level  $h$ 
3:  $best \leftarrow eq$ 
4:  $f \leftarrow \text{True}$ 
5: while  $f$  do
6:    $cur \leftarrow best$ 
7:    $f \leftarrow \text{False}$ 
8:   for  $n \in \text{NEIGHBORS}(cur, \text{level} = h)$  do
9:     if  $\text{DIST}(q, n.\text{emb}) < \text{DIST}(q, best.\text{emb})$  then
10:       $best \leftarrow n$ ;  $f \leftarrow \text{True}$ 
11: return  $best$ 

```

3.3 Proving Greedy Search in Upper Levels

We now discuss how $\mathcal{P}$ proves the correctness of the execution of Alg. 2, which is the greedy search performed at each upper level: Starting from the entry point, it greedily searches for the neighbor closest to the query q , continuously proceeding to that neighbor until a *local optimum* is found.

Lines 5–10: unrolling loops. The core of Alg. 2 lies in the loops: Lines 5–10 form an outer while-loop, and Lines 8–10 form an inner for-loop, where the number of iterations is determined during the execution of the HNSW query. Following a standard approach, the loops are first unrolled: each iteration of the for-loop in Line 8 contributes one row of nodes $(n, cur, best)$ to the witness, and each iteration of the while-loop in Line 5 contributes a sequence of K such rows. Finally, the complete witness trace at each level is organized as a flattened sequence of rows. For each level $h \in \{1, 2, \dots, H\}$, the trace contains exactly $\ell_q^{(h)}$ rows, where $\ell_q^{(h)}$ was defined earlier as the number of (possibly repeated) node visits at level h .

The variable updates in the algorithm (e.g., Lines 6, 7, 10) are straightforward and can be handled with PIOP gadgets in §2.1, whereas the distance computation in Line 9 requires careful design.

Line 9: checking distance computation. Proving a single distance $\text{dist}_n = \text{DIST}(q, n.\text{emb})$ for a temporarily visited node n is straightforward, as this inner-product check can be handled with a Zerocheck sub-PIOP, which incurs $O(d)$ complexity (d is the embedding dimension). However, the real problem is that each node may be visited *multiple* times, and repeatedly performing the check would incur a large overhead.

Challenge: repeated distance check. The problem arises from at least three aspects: First, each iteration of the for-loop in Line 8 requires checking the same distance between q and the current “best” node, making it unnecessary to prove multiple times. Second, in the computation trace of the whole greedy search in the upper levels, every update of the “best” node must come from a previously visited node n , whose distance to q has already been computed once, meaning that the computation is duplicated at least twice. Moreover, in the best-first search (Line 8 in Alg. 3), this issue becomes even more pronounced, as the algorithm repeatedly examines nodes that have already been visited. As a result, in a naive approach, the total number of distance checks to be performed is $m = O(\ell_q)$, and the total prover cost is $O(d \cdot m)$. Roughly estimated, in level $h \in \{1, 2, \dots, H\}$

Algorithm 3 BESTFIRSTSEARCH(G, ep, q, k, efS)

```

1: Input: Graph  $G$ , entry point  $ep$ , query vector  $q$ , top- $k$  size  $k$ , queue size  $efS$ 
2: Output: Top- $k$  approximate nearest neighbors of  $q$ .
   // For a heap  $H$ ,  $H.DIST$  denotes the distance-key field of its elements (used for heap ordering).
3: Initialize priority queues  $C, R \leftarrow \{ep\}$ ; set  $V \leftarrow \{ep\}$ .
4: while  $C \neq \emptyset$  and  $\min(C.DIST) \leq \max(R.DIST)$  do
5:    $c \leftarrow C.DeleteMin$ 
6:   for  $n \in \text{NEIGHBORS}(c, \text{level} = 0)$  do
7:     if  $n \notin V$  then
8:        $\text{dist}_n \leftarrow \text{DIST}(q, n.\text{emb})$ 
9:        $V.Insert(n)$ 
10:       $C.Insert(n, \text{dist}_n)$ ;  $R.Insert(n, \text{dist}_n)$ 
11:      if  $|R| > efS$  then
12:         $R.DeleteMax$ 
13: return top- $k$  closest nodes in  $R$ 

```

the greedy search has $m^{(h)} \approx 2 \cdot \ell_q^{(h)}$; the best-first search has $m^{(0)} \approx 10 \cdot \ell_q^{(0)}$. Considering that the dimension d is often in the hundreds or even thousands, this overhead becomes unacceptable. *Our solution: subtable lookup.* We improve this via a subtable lookup: Instead of repeatedly checking distances, $\mathcal{P}$ first selects m_{unique} nodes that are *uniquely* visited during the query and constructs a distance table $\mathbf{t}_S = \{(id_i, \text{emb}_i, \text{dist}_i)\}$, where $m_{\text{unique}} < \ell_q$ and dist_i is the distance between node i and the query q . Then, $\mathcal{P}$ commits to $\mathbf{t}_S$ and proves that the distances in the table are correctly computed. Thereafter, whenever a distance is required, she simply performs a lookup into $\mathbf{t}_S$ to avoid redundant distance checks, and demonstrates the correctness through a lookup argument, which we instantiate using LogUp [34] (efficient when the table and query sizes are comparable). In this way, constructing the table and proving the distances incurs $O(d \cdot m_{\text{unique}})$ cost for $\mathcal{P}$, and the lookup argument adds an $O(m)$ cost. Therefore, the total prover cost is reduced to $O(m + d \cdot m_{\text{unique}})$, which is highly efficient in practice.

3.4 Proving Best-First Search in Base Level

The best-first search (Alg. 3) aims to return a set of high-quality nearest neighbors in Level 0. Lines 4–12 in Alg. 3 again consist of an outer while-loop and an inner for-loop, which we model—together with recording the relevant variable states—analogously to §3.3. The core logic (and also the main challenge) is to maintain two *priority queues*, C and R , and a *set* V , which we elaborate on below.

Lines 5, 10, 12: maintaining priority queues. These lines involve maintaining two priority queues, C and R , where C has no capacity limit while R has a fixed capacity efS , a predefined parameter that influences search precision. Two operations are involved: (i) *Insert*, which inserts an element into the queue, and (ii) *DeleteMin* (*DeleteMax*), which deletes the minimum (maximum) element from the priority queue.

Challenge: checking priority queue updates. We find maintaining the correctness of queue updates is a great bottleneck, especially for (ii). A straightforward approach to checking correctness would be to validate that each *DeleteMin* (*DeleteMax*) indeed removes the minimum (maximum) element present in the queue at that time. From $\mathcal{P}$'s perspective, however, this naive strategy requires scanning the

entire queue at every deletion, leading to $O((\ell_q^{(0)})^2)$ complexity for a trace of length $O(\ell_q^{(0)})$, which is clearly infeasible. A reader might instead consider leveraging the standard “heap” implementation of a priority queue. However, directly encoding heap operations into arithmetic circuits still results in $O(\ell_q^{(0)} \log \ell_q^{(0)})$ prover complexity, which remains too costly for our purposes.

Our solution: “checker” for priority queues. Our idea comes from the principle that, to prove a program, $\mathcal{P}$ does not actually need to prove the entire program. Instead, $\mathcal{P}$ only needs to prove that the witness satisfies a *checker*, which can be much more efficient than executing the full program from scratch. This “checker-based” SNARK design philosophy is also leveraged by previous work [5, 6].

Checker with linear complexity. A SNARK-friendly checker is designed for priority queue updates, enforcing two properties: the *multiset property* and the *priority-queue property*. First, the multiset property is straightforward: with a permutation check, we certify that the final state equals the initial state plus all undeleted insertions, and with *indexed lookup* we ensure that each deletion corresponds to a unique earlier insertion. The priority-queue property is more subtle. Inspired by the lower bound systems of prior work [21, 26], for every insertion, we define its *maximum lower-bound* (maxlb) as the maximum among all *DeleteMin* elements during its lifetime. The vector maxlb has size $O(\ell_q^{(0)})$. We show that the property that every *DeleteMin* is indeed the current minimum is *equivalent* to requiring each insertion value to be no smaller than its maxlb . Moreover, we present a linear-time method to prove that every maxlb is correctly defined. This yields a linear-time checker, enabling a PIOP with optimal prover cost $O(\ell_q^{(0)})$, instead of the $O(\ell_q^{(0)} \log \ell_q^{(0)})$ cost of naive approaches.

Proving the validity of maxlb. We need to further prove that each maxlb_i is correctly defined. By definition, maxlb_i 's correctness is equivalent to: (1) for an insertion ins_i , maxlb_i corresponds to the key of some valid deletion that occurs during the lifetime of ins_i ; (2) no deletion with a strictly larger key occurs within ins_i 's lifetime. The first condition is straightforward: we provide a pointer, and an *indexed lookup* is used to verify that it matches a valid deletion in the sequence. For the second condition, we further provide two auxiliary pointers, *prev* and *next*, associated with the deletion corresponding to maxlb_i . The *prev* and *next* identify the closest previous and next deletions with larger keys. By proving that both *prev* and *next* fall outside the lifetime of ins_i , we ensure that no larger deletion occurs in the interval, thereby validating maxlb_i . Fig. 3 is an illustration of this proof strategy.

Thus, correctness reduces to the definition of *prev* and *next*. Since these vectors can be computed using a linear-time monotonic-stack algorithm with simple update rules, we can efficiently generate a proof of their correctness. This further yields a linear-time proof of priority queue update correctness. Full details are given in §4.2.

Lines 7, 9: maintaining membership in the set V . These lines perform insertions into, and membership checks against, the set V . Conceptually, V records all nodes that have been visited, and a selector vector $\mathbf{v} \in \{0, 1\}^{\ell_q^{(0)}}$ is defined accordingly: at step $t \in [\ell_q^{(0)}]$, if $\mathbf{n}_t \in V$ then $\mathbf{v}_t = 1$; otherwise, $\mathbf{v}_t = 0$ and $\mathbf{n}_t$ is inserted into V . The resulting membership selector bits $\mathbf{v}$ are then used to control subsequent variable updates.

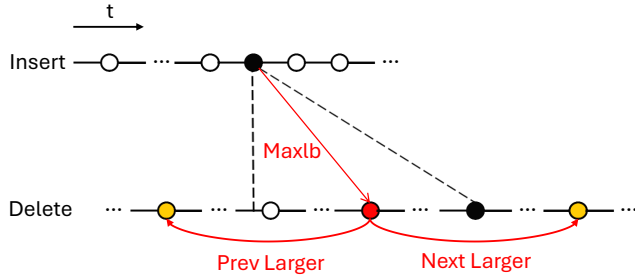


Figure 3: Illustration of proving the definition of maxlb . Insertion and deletion operations occur along a single timeline, but are depicted on separate tracks for clarity. The dashed lines mark an insertion's lifetime. For its maxlb to be valid, the nearest previous and next deletions with larger keys must lie outside this lifetime.

Challenge: Inefficiency from maintaining V . A naive way to check the above logic is to “explicitly” maintain membership in V by assigning *every* node in the database a bit (e.g., via a bitmap or a hash table), and flipping the bit to 1 after the node first appears. However, in a proof system, such a representation introduces a global state of size N , incurring $O(N)$ prover cost.

Our solution: “checker” for membership selector. Our key observation is that the selector vector $\mathbf{v}$ itself can implicitly encode membership, instead of maintaining V explicitly. We design a *checker* for the validity of the *membership selector* $\mathbf{v} \in \{0, 1\}^{\ell_q^{(0)}}$. The intuition is that repeated visits to the same node yield a bit pattern in $\mathbf{v}$ of the form $0, 1, 1, \dots, 1$: the first occurrence is marked with 0, and all subsequent visits with 1. For example, consider nodes $\{3, 4, 5\}$ with visiting pattern $\mathbf{n} = \{5, 4, 5, 3, 4, 5, 4, 5\}$. The corresponding selector is $\mathbf{v} = \{0, 0, 1, 0, 1, 1, 1, 1\}$. If we reorder the selector $\mathbf{v}$ by node $\mathbf{n}$ in ascending order, and for ties (i.e., the same node) by the original trace step $\mathbf{t}$ in ascending order, we obtain $\mathbf{n}^{\text{seq}} = \{3, 4, 4, 4, 5, 5, 5, 5\}$, $\mathbf{t}^{\text{seq}} = \{3, 1, 4, 6, 0, 2, 5, 7\}$, and $\mathbf{v}^{\text{seq}} = \{0, 0, 1, 1, 0, 1, 1, 1\}$. To validate $\mathbf{v}$, we check that: (i) the reordering is correct; (ii) within each group of identical $\mathbf{n}^{\text{seq}}$, the corresponding $\mathbf{v}^{\text{seq}}$ follows the required pattern (first 0, then 1’s). Since all vectors involved are of length $\ell_q^{(0)}$, this checker reduces the prover’s cost from $O(N)$ to $O(\ell_q^{(0)})$.

4 Gadgets for Proving HNSW Query

This section provides details for key gadgets in the PIOP for HNSW query. To compile them, we instantiate the PCS with two schemes: depending on the context, we use KZG [38] for the univariate PCS uPC and PST [50] for the multilinear PCS mPC. The detailed PCS constructions are provided in §B.

4.1 Hybrid Lookup Argument

Consider the lookup relation $\mathcal{R}_{lookup}(N, L; \mathbf{t}, \mathbf{q}) = 1$ iff $\mathbf{q} \subset \mathbf{t}$, where $\mathbf{q} \in \mathbb{F}^L$, $\mathbf{t} \in \mathbb{F}^N$ ($L \ll N$), and the polynomials q and t are the MLE and the Lagrange extension of $\mathbf{q}$ and $\mathbf{t}$, respectively. Below we present a PIOP for $\mathcal{R}_{lookup}$, where the online complexity of $\mathcal{P}$ is only $O(L)$, independent of N .

Similar to CQ [23], in the offline phase, the table t is encoded as a univariate polynomial $t(X)$. $\mathcal{P}$ commits to $t(X)$ as com_t and computes com_Z (recall that $Z(X) = X^N - 1$), and preprocesses it to

obtain parameters $\{\{Q_i\}_g\}_{i \in [N]}$. These parameters can be computed in $O(N \log N)$ time. In the online phase, given the query q , $\mathcal{P}$, $\mathcal{V}$ execute the following steps:

- 1: Let $\mathbf{m}_i$ denote the number of times the element t_i appears in $\mathbf{q}$ for $i \in [N]$, and $m(X)$ be the univariate extension of $\mathbf{m}$. $\mathcal{P}$ computes $\text{com}_m, \text{com}_q$ and sends to $\mathcal{V}$. $\mathcal{V}$ samples a random challenge $r \xleftarrow{\$} \mathbb{F}$ and sends back to $\mathcal{P}$.
- 2: Let $\mathbf{A}_i = \frac{m_i}{t_i + r}$ for $i \in [N]$, and $A(X)$ be the univariate extension of $\mathbf{A}$. $\mathcal{P}$ and $\mathcal{V}$ run the following to check the well-formedness of $A(X)$:
 - a. $\mathcal{P}$ computes $\text{com}_A, \text{com}_Q$ such that $A(X)(t(X) + r) - m(X) = Q(X)Z(X)$ and sends to $\mathcal{V}$. Note that com_Q can be computed from $\text{com}_Q = \sum_{i \in [N], A_i \neq 0} \mathbf{A}_i \cdot [Q_i]_g$.
 - b. $\mathcal{V}$ checks $e(\text{com}_A, \text{com}_t) + e(r \cdot \text{com}_A - \text{com}_m, [1]_g) \stackrel{?}{=} e(\text{com}_Q, \text{com}_Z)$ to check about $A(X)$.
- 3: Let $\mathbf{B}_i = \frac{1}{q_i + r}$ for $i \in [L]$, and $B(\mathbf{x})$ be the MLE of $\mathbf{B}$. $\mathcal{P}$ computes com_B and sends to $\mathcal{V}$. They run a Zerocheck on $B(\mathbf{x}) \cdot (q(\mathbf{x}) + r) - 1$ to check about $B(\mathbf{x})$.
- 4: Let $H := \sum_{X \in \Omega_N} A(X)$. $\mathcal{P}$ computes H and sends to $\mathcal{V}$. They run uPC.Open on $A(0)$, and $\mathcal{V}$ checks $N \cdot A(0) \stackrel{?}{=} H$, thereby checking the summation on the LHS of Eq. (1).
- 5: Finally, $\mathcal{P}, \mathcal{V}$ run a Sumcheck on $H = \sum_{\mathbf{x} \in \{0,1\}^{\log L}} B(\mathbf{x})$ to check the summation on the RHS of Eq. (1).

We refer to the above scheme as HybridLookup. The detailed description of $\{\{Q_i\}_g\}_{i \in [N]}$ and the offline preprocessing, along with the security analysis, are deferred to §F.1.

Complexity. Due to the sparsity of $\mathbf{m}$ and $\mathbf{A}$, steps 1, 2, and 4 incur $O(L)$ complexity, while steps 3 and 5 also incur $O(L)$ complexity. Therefore, the total online prover time is $O(L)$. The verifier time and proof size are both $O(\log L)$.

Extending to batched lookup. Our scheme extends to support batched lookup argument using standard approaches: given $\mathbf{t} \in \mathbb{F}^{N \times d}$, $\mathbf{q} \in \mathbb{F}^{L \times d}$, $\mathcal{V}$ samples $r \xleftarrow{\$} \mathbb{F}$, and it suffices to check that $\sum_{i \in [d]} r^i \cdot \mathbf{q}[i] \subset \sum_{i \in [d]} r^i \cdot \mathbf{t}[i]$. Therefore, in the offline phase, $\mathcal{P}$ preprocesses each column of $\mathbf{t}$ individually, and in the online phase applies HybridLookup to the random linear combination of the table and query columns.

4.2 PIOP for Priority Queue Updating

Notation. Each *element* of the multiset is the ordered pair (id, v) where $id \in \mathbb{Z}_{\geq 0}$ is an identifier and $v \in \mathbb{R}$ is the value. Let $\sigma = (op_1, \dots, op_n)$ be a finite sequence of multiset operations, where each op_i is either an Insert or a Delete of the form (t_i, id_i, v_i) , with timestamp $t_i \in \mathbb{Z}$, the element $(id_i, v_i) \in \mathbb{Z}_{\geq 0} \times \mathbb{R}$. At most one Insert and one Delete occur at any given time. σ can be equivalently denoted as two strictly time-ordered subsequences: insertions $\sigma_{\text{ins}} = (\text{ins}_1, \dots, \text{ins}_m)$ and deletions $\sigma_{\text{del}} = (\text{del}_1, \dots, \text{del}_k)$, where each insertion $\text{ins}_i = (t_i, id_i, v_i)$ and each deletion $\text{del}_j = (t'_j, id'_j, v'_j)$, with $t_1 < t_2 < \dots$ and $t'_1 < t'_2 < \dots$ respectively.

Definition 1 (Valid Multiset Operation Sequence). Assume the multiset is $\emptyset$ in the beginning, and MS_{final} is a given final multiset state containing only elements. We use $\uplus$ to denote multiset sum.

We say that $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ is a *valid multiset operation sequence* for MS_{final} if there exists a matching $\pi : [k] \rightarrow [m]$ such that:

- (1) $\forall j \in [k], t_{\pi(j)} < t'_j, \text{id}_{\pi(j)} = \text{id}'_j$,
- (2) π is an injective matching,
- (3) $\text{MS}_{\text{final}} \uplus (\sigma_{\text{del}})[1 : 2] = (\sigma_{\text{ins}})[1 : 2]$.

Definition 2 (Valid Priority Queue Operation Trace). Assume the multiset is $\emptyset$ in the beginning, and PQ_{final} is a multiset containing only elements. A operation sequence $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ is a *valid priority queue operation trace* for PQ_{final} if:

- (1) **Multiset Operation Validity:** Given PQ_{final} , σ is a valid multiset operation sequence satisfying Definition 1;
- (2) **Delete-Min Correctness:** For every deletion $\text{del}_j = (t'_j, \text{id}'_j, v'_j) \in \sigma_{\text{del}}$, the key v'_j is the minimum among keys of all elements in the queue at time t'_j .

Theorem 1 (A Checker for Valid Multiset Operation Sequence). Let MS_{final} be a multiset of elements. Then $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ is a valid multiset operation sequence if and only if there exists a pointer vector $\mathbf{p} \in \mathbb{Z}_{\geq 0}^k$ such that:

$$\forall j \in [k], t_{\mathbf{p}_j} \leq t'_j \quad (2)$$

$$\forall j \in [k], (\text{id}_{\mathbf{p}_j}, v_{\mathbf{p}_j}) = (\text{id}'_j, v'_j) \quad (3)$$

$$\forall j_1, j_2 \in [k], j_1 \neq j_2, \mathbf{p}_{j_1} \neq \mathbf{p}_{j_2} \quad (4)$$

$$\text{MS}_{\text{final}} \uplus (\sigma_{\text{del}})[1 : 2] = (\sigma_{\text{ins}})[1 : 2]. \quad (5)$$

Theorem 2 (A Linear-Complexity Checker for Minimum Priority Queue Update Correctness). Let PQ_{final} be a multiset of elements and $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ be an operation sequence. Define the *lifetime* of insertion ins_i as the interval (t_i, t_{d_i}) , where t_{d_i} is the time of deletion of id_i if it is ever deleted, or ∞ otherwise. Define the *maximum lower bound* for ins_i as:

$$\text{maxlb}_i = \max \{ \{v'_j \mid t'_j \in [t_i, t_{d_i})\} \cup \{-\infty\} \}$$

The sequence $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ is a valid execution trace of a minimum priority queue iff: (i) There exists a vector $\mathbf{p}$ satisfying the conditions of Theorem 1; (ii)

$$\forall i \in [m] : v_i \geq \text{maxlb}_i \quad (6)$$

Detailed proofs of Theorem 1 and Theorem 2 are in §E.

Proving the correctness of maxlb_i . We focus on the nontrivial case where an insertion's maxlb is not $-\infty$, i.e., when the insertion has a corresponding maximum deletion during its lifetime. $\mathcal{P}$ commits to the following witness:

- For each insertion ins_i with $\text{maxlb}_i \neq -\infty$, a pointer L_i such that $\text{maxlb}_i = v'_{L_i}$ (v' is the third column of del).
- For each deletion del_j , two auxiliary pointers: (1) prev_j , the nearest previous deletion with a larger key; (2) next_j , the nearest subsequent deletion with a larger key.

For each insertion ins_i with $\text{maxlb}_i \neq -\infty$, we check:

$$\text{maxlb}_i = v'_{L_i} \quad (7)$$

$$t_i \leq t'_{L_i} < t_{d_i} \quad (8)$$

$$t'_{\text{prev}_{L_i}} < t_i \quad (9)$$

$$t'_{\text{next}_{L_i}} \geq t_{d_i} \quad (10)$$

These constraints jointly guarantee that: (1) maxlb_i is the key v'_{L_i} of a valid deletion del_{L_i} occurring during the lifetime of ins_i

(by (7)–(8)), and (2) no larger deletion appears strictly within that interval (by (9)–(10)). Figure 3 illustrates why these conditions are both complete and sound for insertions with $\text{maxlb}_i \neq -\infty$. Note that the index L_i only needs to exist, whereas the witness pointers prev_j and next_j —which identify the nearest larger deletions before and after each del_j —must themselves be proven correct. To enforce these constraints, we introduce a bit vector over insertions marking those with $\text{maxlb}_i \neq -\infty$, and use it as a selector to apply the full set of checks above. For insertions with $\text{maxlb}_i = -\infty$, we instead employ standard techniques to prove that no DeleteMin occurs during their lifetime. Further details are provided in §F.3.

Proving the nearest greater neighbors of all elements. The objective is to prove that the pointers prev and next associated with each del are correctly defined. Specifically, for every del , these pointers must designate the nearest greater element to its left and right, respectively. To prove this, we show that they coincide with the output of the *monotone stack algorithm* (This detailed algorithm Alg. 6 is in §F.2) when executed on del as input. Since the update rule of the monotone stack is simple and explicit, its correctness can be proved directly by proving the execution of the algorithm.

Specifically, the monotone stack processes the sequence from left to right while maintaining a stack of candidate indices in decreasing order of values. When a new element arrives, it pops any smaller elements from the stack; for each popped element, the new element is recorded as its “next-greater” pointer. At the same time, the surviving top element of the stack (if any) becomes the “previous-greater” pointer for the new element. Thus the algorithm constructs prev and next arrays that characterize the nearest-greater-neighbor structure in $O(n)$ time, with exactly n pushes and n pops. The details, including the formal relation definition, the PIOP construction, complexity, and security analysis, are deferred to §F.2.

PIOP for priority queue updates. The constraints (2)–(10), along with those checking prev , next , and insertions with $\text{maxlb} = -\infty$, can all be proved using the gadgets in §2.1. By Theorem 2, this enables $\mathcal{P}$ to generate a complete and sound proof, yielding an efficient PIOP for min-priority queue updates (and analogously for max-priority queues). Full details and analysis are deferred to §F.3.

Complexity. Let m and k denote the numbers of Insert and DeleteMin. Constraints (2)–(10) have size $O(m + k)$, while those for proving prev and next have size $O(k)$. Using the gadgets in §2.1, the prover time for checking minimum priority queue is $O(n)$ ($n = m + k$ is the total number of Insert and DeleteMin operations), rather than $O(n \log n)$. The verifier time and proof size are succinct in n .

4.3 PIOP for Checking Membership Selector

We consider maintaining the membership in a set, which is formed by a sequence $\mathbf{n} = (\mathbf{n}_0, \mathbf{n}_1, \dots, \mathbf{n}_{\ell-1}) \in \mathbb{Z}^\ell$. At each time step t , the element $\mathbf{n}_t$ attempts to enter a dynamic set V that records all distinct elements observed so far. If $\mathbf{n}_t$ has not appeared previously in the stream, we set the membership flag as a bit $v_t = 0$ and then insert it into V ; otherwise, we set $v_t = 1$. That said, for each time step $t \in [\ell]$,

$$v_t = \begin{cases} 0 & \text{if } \mathbf{n}_t \neq \mathbf{n}_s \text{ for all } s < t, \\ 1 & \text{otherwise.} \end{cases} \quad (11)$$

We call the resulting vector $\mathbf{v}$ as the membership selector.

Checker for the membership selector. The definition of $\mathbf{v}_t$ can be verified directly from the pairs $(\mathbf{n}_t, \mathbf{v}_t)$ without explicitly reconstructing the evolving set V . To this end, let $(\mathbf{t}^{\text{seq}}, \mathbf{n}^{\text{seq}}, \mathbf{v}^{\text{seq}})$ denote the sequence obtained by sorting all triples $(t, \mathbf{n}, \mathbf{v})$ lexicographically: first by ascending $\mathbf{n}$, and then (for ties) by ascending t . The membership selector $\mathbf{v}$ satisfies Eq. (11) if and only if the following constraints hold:

$$\forall i \in [\ell], \mathbf{v}_i \cdot (1 - \mathbf{v}_i) = 0 \quad (12)$$

$$\mathbf{v}^{\text{seq}}_0 = 0, \quad (13)$$

$$\forall i \in [\ell] \setminus \{0\}, \mathbf{v}^{\text{seq}}_i \cdot (\mathbf{n}^{\text{seq}}_i - \mathbf{n}^{\text{seq}}_{i-1}) = 0. \quad (14)$$

$$\forall i \in [\ell] \setminus \{0\}, \mathbf{v}^{\text{seq}}_i + (1 - \mathbf{v}^{\text{seq}}_i) \cdot (\mathbf{n}^{\text{seq}}_i - \mathbf{n}^{\text{seq}}_{i-1}) \neq 0. \quad (15)$$

These conditions are jointly complete and sound for checking the membership selector $\mathbf{v}$, as formalized in Eq. (11); the detailed theorem and proof are deferred to §F.4. In addition to the above conditions, we must also use standard ways to check the sorting correctness of the witness $(\mathbf{t}^{\text{seq}}, \mathbf{n}^{\text{seq}}, \mathbf{v}^{\text{seq}})$.

PIOP for checking membership selector. Constraints (12)–(15) can be proved using PIOPs from §2.1. We apply a Permcheck gadget to prove that $(\mathbf{t}^{\text{seq}}, \mathbf{n}^{\text{seq}}, \mathbf{v}^{\text{seq}})$ is a permutation of $(t, \mathbf{n}, \mathbf{v})$; then, we use Zerocheck and Rangecheck gadgets to verify the correctness of the two-level sorting procedure. Due to the space limit, we put the formal relation definition, the full PIOP, the detailed complexity analysis, and the security analysis in §F.4.

Complexity. Let ℓ denote the sequence length. Since all constraints are $O(\ell)$ and the PIOPs in §2.1 have linear prover time with succinct verification, the overall construction achieves $O(\ell)$ prover time and succinct verifier time and proof size.

5 zkRAG: Verifiable RAG retrieval via HNSW

Offline phase. In the offline phase, $\mathcal{P}$ commits to the embedding table $\mathbf{t}_{\mathcal{D}}$ and the HNSW hierarchy table $\mathbf{t}_{\mathcal{H}}$. All valid nodes are indexed from 1, with identifier 0 reserved for a special *dummy node*. This dummy node $\perp \in \mathbb{F}^2$ is defined to be at the maximum possible distance from every valid embedding.

5.1 Proving Greedy Search

Notation. We define a *time step* as a single execution of the for-loop, i.e., examining one neighbor of the current node. Each for-loop thus consists of exactly K time steps. Let ℓ_q denote the total number of time steps, with $\mathbf{c}, \mathbf{f} \in \mathbb{F}^{\ell_q}$ and $\mathbf{n}, \mathbf{b} \in \mathbb{F}^{\ell_q \times 2}$. Here, $\mathbf{c}_t$ denotes the current source node at step t , $\mathbf{n}_t[0]$ is the identifier of the neighbor candidate examined at step t , with distance $\mathbf{n}_t[1]$ to the query vector $\mathbf{q}$. After the comparison, $\mathbf{b}_t[0]$ denotes the identifier of the best node so far, and $\mathbf{b}_t[1]$ its distance to $\mathbf{q}$. $\mathbf{f}_t$ is a flag bit indicating whether the best node is updated at step t .

Proving while-loop termination. We use a selector vector $\mathbf{z}$ to mark the termination step: the termination condition is proved to fail when $\mathbf{z}_t = 0$ and to hold when $\mathbf{z}_t = 1$. This technique is standard in proofs of iterative computation [9, 63].

Proving the distances are correct. $\mathcal{P}$ organizes a subtable $\mathbf{t}_{\mathcal{S}}$ containing all the unique node id and their embeddings and distances. $\mathbf{t}_{\mathcal{S}}[0]$ is id and $\mathbf{t}_{\mathcal{S}}[1:d]$ is the embedding and $\mathbf{t}_{\mathcal{S}}[d+1]$ is the distance. $\mathcal{P}$ runs a Zerocheck with $\mathcal{V}$ to check the correctness of the distances inside $\mathbf{t}_{\mathcal{S}}$. Then we check that $\mathbf{t}_{\mathcal{S}}$ is in $\mathbf{t}_{\mathcal{D}}$ and that

$\mathbf{n}$ and $\mathbf{b}$ are in $\mathbf{t}_{\mathcal{S}}$. That is:

$$\mathbf{t}_{\mathcal{S}}[0:d] \subset \mathbf{t}_{\mathcal{D}} \quad \mathbf{n}, \mathbf{b} \subset (\mathbf{t}_{\mathcal{S}}[0], \mathbf{t}_{\mathcal{S}}[d+1]) \cup \{\perp\} \quad (16)$$

Proving variables update in the for-loop. In Alg. 2, each while-loop triggers one $\mathbf{c}$ update (Line 6) and one full for-loop (Line 8). Inside each for-loop, the current source node $\mathbf{c}$ remains unchanged. Remember that in the witness trace of greedy search, every fixed K rows correspond to one full for-loop. So the update rule of $\mathbf{c}$ is: for time steps $t = k \cdot K, k = 0, 1, \dots, \mathbf{c}_t = \mathbf{b}_t[0]$ and $\mathbf{c}_t = \mathbf{c}_{t+1} = \dots = \mathbf{c}_{t+K-1}$. We introduce a special mask vector mask_{for} , where $\text{mask}_{\text{for},i} = 1$ for $i = k \cdot K, k = 0, 1, \dots, \frac{\ell_q}{K}$ and 0 otherwise. Thus we rewrite the update rule of $\mathbf{c}$ as: $\forall i \in [\ell_q]$,

$$\text{mask}_{\text{for},i} \cdot (\mathbf{c}_i - \mathbf{b}_i[0]) + (1 - \text{mask}_{\text{for},i}) \cdot (\mathbf{c}_i - \mathbf{c}_{i-1}) = 0. \quad (17)$$

The update rule of the current examined node $\mathbf{n}$ (Line 8) is:

$$(h, \mathbf{c}_t, \mathbf{n}_t, \dots, \mathbf{n}_{t+K-1}) \subset \mathbf{t}_{\mathcal{H}} \quad (18)$$

The update rule of the current best node $\mathbf{b}$ and bit flag $\mathbf{f}$ (Line 9–10) is: $\mathbf{b}_t[1] = \min\{\mathbf{b}_{t-1}[1], \mathbf{n}_t[1]\}$, i.e., $\forall i \in [\ell_q] \setminus \{0\}$,

$$\mathbf{f}_i \cdot (1 - \mathbf{f}_i) = 0 \quad \mathbf{b}_0[0] = \mathbf{n}_0[0] \quad (19)$$

$$\mathbf{f}_i \cdot (\mathbf{b}_i[1] - \mathbf{n}_i[1]) + (1 - \mathbf{f}_i) \cdot (\mathbf{b}_i[1] - \mathbf{b}_{i-1}[1]) = 0, \quad (20)$$

$$(2 \cdot \mathbf{f}_i - 1) \cdot (\mathbf{b}_{i-1}[1] - \mathbf{n}_i[1]) \geq 0 \quad (21)$$

PIOP for greedy search. We use gadgets in §2.1 to check constraints (17), (19)–(21). In order to check (16), We use a HybridLookup to check that $\mathbf{t}_{\mathcal{S}}$ is in $\mathbf{t}_{\mathcal{D}}$ and run a LogUp to check that $\mathbf{n}$ and $\mathbf{b}$ are in $\mathbf{t}_{\mathcal{S}}$. We use a HybridLookup to check (18) and a Zerocheck to check the distances in $\mathbf{t}_{\mathcal{S}}$. Due to the space limit, we present the detailed PIOP and its complexity and security analysis in §G.1.

Complexity. All above constraints have size at most $O(d \cdot \ell_q)$, except for HybridLookup. The HybridLookup introduced in §4.1 has $\mathcal{P}$ complexity linear in the query vector's size, namely $\mathbf{t}_{\mathcal{S}}$, where $|\mathbf{t}_{\mathcal{S}}| < O(\ell_q)$ in our construction. Since all gadgets in §2.1 have a linear-time prover, succinct verification, the overall PIOP runs in $O(d \cdot \ell_q)$ prover time with succinct verifier time and proof size.

5.2 Proving Best-first Search

Notation. Let ℓ_q denote the total number of time steps, and $\mathbf{n}, \mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}} \in \mathbb{F}^{\ell_q \times 2}$, with the two columns representing pairs of the form (id, distance). $\mathbf{c}, \mathbf{v} \in \mathbb{F}^{\ell_q}$. $\mathbf{c}$ and $\mathbf{n}$ are defined the same as in greedy search. $\mathbf{v}_t$ is the first-occurrence bit indicating whether $\mathbf{n}_t[0]$ is visited for the first time. At time t , $\mathbf{C}_{\text{ins},t}[0]$ and $\mathbf{R}_{\text{ins},t}[0]$ each denote a single node inserted into C and R , respectively. Similarly, $\mathbf{C}_{\text{del},t}[0]$ and $\mathbf{R}_{\text{del},t}[0]$ denote the single node deleted from C and R , respectively. We adopt the convention that, when insertions and deletions occur at the same time step t , $\mathbf{C}_{\text{del},t}$ is applied *before* $\mathbf{C}_{\text{ins},t}$, whereas $\mathbf{R}_{\text{del},t}$ is applied *after* $\mathbf{R}_{\text{ins},t}$. Finally, $\mathbf{C}_{\text{final}}$ and $\mathbf{R}_{\text{final}}$ denote the final states of C and R , where $\mathbf{C}_{\text{final}} \in \mathbb{F}^{(\ell_q - \ell_q/K + 1) \times 2}$, $\mathbf{R}_{\text{final}} \in \mathbb{F}^{\text{efS} \times 2}$. $\text{ep} \in \mathbb{F}^2$ is the entry point of the best-first search.

Proving while-loop termination. The outer while-loop termination logic can be proved similarly to that in §5.1.

Proving the distances are correct. $\mathcal{P}$ organizes $\mathbf{t}_{\mathcal{S}}$ containing all the unique node id and their embeddings and distances. $\mathbf{t}_{\mathcal{S}}[0]$ is id and $\mathbf{t}_{\mathcal{S}}[1:d]$ is the embedding and $\mathbf{t}_{\mathcal{S}}[d+1]$ is the distance. We check the distances inside $\mathbf{t}_{\mathcal{S}}$. Then, we check that $\mathbf{t}_{\mathcal{S}}$ is in $\mathbf{t}_{\mathcal{D}}$ and that

$\mathbf{n}$, C_{ins} , C_{del} , R_{ins} , R_{del} are in $\mathbf{t}_S$. That is:

$$\mathbf{t}_S[0 : d] \subset \mathbf{t}_D \quad (22)$$

$$\mathbf{n}, C_{\text{ins}}, C_{\text{del}}, R_{\text{ins}}, R_{\text{del}} \subset (\mathbf{t}_S[0], \mathbf{t}_S[d+1]) \cup \{\perp\} \quad (23)$$

Proving the correctness of $\mathbf{v}$. The vector $\mathbf{v}$ serves as the membership selector for the sequence $\mathbf{n}[0]$, but with the set initialized as $\{\text{ep}\}$. To check $\mathbf{n}, \mathbf{v}$ satisfy (11), we run a slightly modified PIOP for checking membership selector from §4.3. The only change is in condition (14), which becomes $\forall i \in [\ell_q] \setminus \{0\}, (1 - \mathbf{v}^{\text{seq}_i}) \cdot (\mathbf{n}^{\text{seq}_i}[0] - \mathbf{n}^{\text{seq}_{i-1}}[0]) \cdot (\mathbf{n}^{\text{seq}_i}[0] - \text{ep}[0]) = 0$ using the notation of this subsection. Note that this change is because in best-first search, V begins with $\{\text{ep}\}$, so whenever $\mathbf{n}[0] = \text{ep}[0]$, the original condition (14) would fail.

Proving variables update in the for-loop. The update rules of $\mathbf{c}$ and $\mathbf{n}$ (Line 6 in Alg. 3) are: For $t = k \cdot K, k = 0, 1, \dots$:

$$\text{mask}_{\text{for } t} \cdot (c_t - C_{\text{del}}[0]) + (1 - \text{mask}_{\text{for } t}) \cdot (c_t - c_{t-1}) = 0, \quad (24)$$

$$(0, c_t[0], \mathbf{n}_t[0], \dots, \mathbf{n}_{t+K-1}[0]) \subset \mathbf{t}_H \quad (25)$$

The main challenge lies in justifying the priority-queue logic. The correctness of C_{ins} , C_{del} , R_{ins} , R_{del} , C_{final} , and R_{final} requires two ingredients: firstly, consistency of these sequences with the control flow of Alg. 3; secondly, two separate PIOPs for priority queue update from §4.2 showing that C and R indeed satisfy their respective priority-queue semantics. From the control flow we know: (i) Lines 7 and 10 insert $\mathbf{n}_t$ into C and R only if it has not appeared before (i.e., $\mathbf{v}_t = 1$). (ii) Line 5 performs C_{del} only at time steps that are multiples of K . (iii) Lines 11–12 enforce R as a max-priority queue of fixed capacity efS , so when $\mathbf{v}_t = 1$, a deletion R_{del} from R occurs only if the queue is already full.

To construct the insertion and deletion sequences for the two PIOPs, naively taking only the subsequence with $\mathbf{v}_t = 1$ is insufficient, since it is difficult to prove that this subsequence originates from C_{ins} and R_{ins} . Instead, we insert $\perp$ into C and R whenever $\mathbf{v}_t = 0$. For C , we require valid DeleteMin only at time steps that are multiples of K , while for R we stipulate that R_{del} occurs whenever $\mathbf{v}_t = 0$, and when $\mathbf{v}_t = 1$ only if the queue is full. To decouple R 's deletions from $\mathbf{v}$, we initialize it with efS dummy $\perp$ insertions at $-\text{efS}, \dots, -2$ and one ep at -1 (recall both C and R start with $\{\text{ep}\}$), and then take the entire R_{del} sequence as deletions. Formally, we set $\text{ins}_R \in \mathbb{F}^{(\ell_q + \text{efS}) \times 3}$ with the first column as time and the next two as R_{ins} , and $\text{del}_R \in \mathbb{F}^{\ell_q \times 3}$ with the first column as time and the rest as R_{del} . For C , we also initialize with ep at -1 and define C_{del}^* as those rows of C_{del} whose indices are multiples of K . We then set $\text{ins}_C \in \mathbb{F}^{(\ell_q + 1) \times 3}$, where the first column is the time step incremented by $+1$ and the next two columns are given by C_{ins} . This offset guarantees that a DeleteMin operation on C is executed strictly before any insertion at the same step. We set $\text{del}_C \in \mathbb{F}^{(\ell_q/K) \times 3}$ with the first column as time and the last two columns as C_{del} . In summary, we first check that:

$$(1 - \mathbf{v}_t) \cdot (C_{\text{ins}_t}[0] - \perp[0]) + \mathbf{v}_t \cdot (C_{\text{ins}_t}[0] - \mathbf{n}_t[0]) = 0 \quad (26)$$

$$(1 - \mathbf{v}_t) \cdot (R_{\text{ins}_t}[0] - \perp[0]) + \mathbf{v}_t \cdot (R_{\text{ins}_t}[0] - \mathbf{n}_t[0]) = 0 \quad (27)$$

Next, we run two PIOPs for priority queue updates. Specifically, the tuple $(\text{ins}_C, \text{del}_C, C_{\text{final}})$ is provided as input to check the correctness of C , while the tuple $(\text{ins}_R, \text{del}_R, R_{\text{final}})$ is provided as input to check the correctness of R .

Soundness. Recall that $\perp$ is the special dummy node with a maximum distance value. We now argue the soundness of the above checks. The candidate queue C serves only to provide its DeleteMin

output for updating c , while the result queue R is maintained as a fixed-size max-priority queue retaining the closest nodes. Thus, although $\perp$ may be temporarily inserted into both queues, it will *never* be returned as the DeleteMin of C , and it will *always* be removed as the DeleteMax of R . Moreover, since R is pre-filled with efS copies of $\perp$, each valid insertion into R is paired with a DeleteMax that discards one $\perp$, until R eventually contains only valid nodes. Hence the correctness of C_{del} and R_{del} follows directly from these rules, without requiring any additional constraints.

PIOP for Best-First Search. HybridLookup is used to check (22) and (25); the PIOP for membership selector from §4.3 proves $\mathbf{v}$; two PIOPs for priority queue updating from §4.2 validate the heap operations, together with Zerocheck for (24), (26)–(27); and LogUp verifies (23). Due to space limit, the full protocol is presented in §G.2.

Complexity. All gadgets used have size at most $O(d \cdot \ell_q)$, and all invoked PIOPs run in linear prover time with succinct verifier time and proof size. Thus, we achieve $O(d \cdot \ell_q)$ prover time and succinct verifier complexity and proof size.

5.3 Putting Everything Together

PIOP for HNSW query. In the full protocol, $\mathcal{P}$ reveals the commitment of entry point vector com_{ep} . $\mathcal{P}$ and $\mathcal{V}$ open com_{ep} to check that $\text{ep}^{(H)}$ matches the public input $\text{ep}_{\text{default}}$. At the base level ($h = 0$), $\mathcal{P}$ reveals the final top- k results. The proof consists of H instances of GreedySearchCheck (one per upper level) and one instance of BestFirstCheck at the base level. Importantly, a single sub-table $\mathbf{t}_S$ suffices for all H GreedySearchCheck instances and the final BestFirstCheck, thereby reducing the checks of (16) and (22) to a single overall constraint $\mathbf{t}_S[0 : d] \subset \mathbf{t}_D$. Finally, $\mathcal{P}$ and $\mathcal{V}$ also enforce the following cross-level consistency checks: (1) for all $h \in [H]$, the entry points match, i.e., $\text{ep}^{(h-1)} = \text{best}_{\ell_q^{(h)}}$; (2) at the Level 0, the claimed top- k results coincide with the k smallest elements of R_{final} . Our PIOP can be made non-interactive through the standard Fiat-Shamir transformation [25] to obtain zkRAG.

Adding zero-knowledge. Most components in zkRAG, such as Sumcheck, can be made zero-knowledge using standard techniques from [15]; the zero-knowledge property of lookup arguments requires additional consideration: we show how to design zero-knowledge versions of LogUp and HybridLookup in §I.

Due to space constraints, the non-zero-knowledge PIOP construction for zkRAG and its zero-knowledge version, together with their security analyses, are deferred to §H and §I.4, respectively.

Complexity. The offline phase takes $O(dN \cdot \log(dN))$ time complexity to commit and preprocess the tables $\mathbf{t}_D$ and $\mathbf{t}_H$. In the online phase, the main PIOPs used are HybridLookup, GreedySearchCheck, and BestFirstCheck, all of which have linear $\mathcal{P}$ time and succinct $\mathcal{V}$ time and proof size. Thus, the PIOP for HNSW query achieves total $\mathcal{P}$ complexity $O\left(d \cdot \sum_{h=0}^H \ell_q^{(h)}\right)$, which is optimal for layered graph traversal. Since both H and k are small constants, we omit them in the final complexity expression. Adding zero knowledge incurs only a constant-factor overhead, as the zero-knowledge version introduces a constant number of additional low-degree masked claims and auxiliary commitments, without any super-linear computation.

Table 1: Prover time for heap updates under different schemes. The number of heap operations is denoted by n . “-” indicates failure due to out-of-memory.

n	zkRAG	SP1 [2]	HyperPlonk [24]
2^{10}	4.28 s	46 min	394 min
2^{15}	134.8 s	-	-
2^{20}	4312 s	-	-

6 Evaluation

We fully implement and evaluate the protocols: In §6.2, we first microbenchmark PQCheck to validate its efficiency. In §6.3, we then evaluate zkRAG on end-to-end HNSW queries, measuring its scalability across different search trace lengths ℓ_q and embedding dimensions d . We also compare zkRAG with baselines, demonstrating orders-of-magnitude improvements in prover time, and provide a breakdown analysis to identify the main bottlenecks.

6.1 Experimental Setup

Implement, benchmark and baselines. We implement our protocols in approximately 6,000 lines of Rust. We use a widely adopted ANNS library, Faiss [22], to implement the HNSW algorithm, including both offline graph construction and online queries. We set $K = 32$ during offline graph construction. We evaluate on the standard SIFT benchmark [44], which contains 10^6 database vectors and 10^4 query vectors in dimension $d = 128$. We compare against two baselines: (i) a general-purpose zkSNARK, HyperPlonk [24], and (ii) a zkVM, SP1 [2], which translates any program into RISC-V instructions and generates proofs for them. We choose HyperPlonk because it uses MLE Sumcheck and other MLE gadgets. We use it without our retrieval-specific circuit optimizations. All experiments are conducted on a server with an Intel Xeon 6126 16-core CPU and 200GB of RAM, without multi-threaded acceleration.

Offline. In the offline phase, given the HNSW graph, $\mathcal{P}$ executes the preprocessing procedure of HybridLookup (§4.1) to commit to the tables $t_{\mathcal{D}}$ and $t_{\mathcal{H}}$. Although this preprocessing is relatively expensive, it is performed only once. For the **SIFT1M** dataset, the offline phase takes about two days; detailed preprocessing times for other dataset sizes are reported in §J.

6.2 Microbenchmark

We first evaluate the efficiency of PQCheck when handling heap updates. In each experiment, n insertions and n deletions are performed on a heap: after every 2^5 insertions, one deletion is performed; once all n insertions are completed, the remaining elements are deleted until the heap becomes empty. This setup prevents overly simple insertion-deletion patterns from enabling low-level compiler optimizations that could bias performance (e.g., SP1 may benefit from such optimizations). After these operations, zkRAG, SP1, and HyperPlonk each generate a proof: for SP1, the program is proved directly, while for HyperPlonk, the program is compiled into a low-level arithmetic circuit before being proved.

By varying the number of updates n , we compare the prover time of zkRAG, SP1, and HyperPlonk, as shown in Tab. 1. PQCheck outperforms the others by a large margin: it is more than 600×

Table 2: Performance of zkRAG when varying ℓ_q . N is the dataset size. $|\mathcal{P}_1|$, $|\mathcal{P}_2|$, and $|\mathcal{P}|$ represent the prover time for the greedy search, best-first search, and the total, respectively. $|\mathcal{V}|$, $|\pi|$ denote the total verifier time and proof size.

N	$\sum_{h=1}^H \ell_q^{(h)}$	$\ell_q^{(0)}$	$ \mathcal{P}_1 $	$ \mathcal{P}_2 $	$ \mathcal{P} $	$ \mathcal{V} $	$ \pi $
2^{15}	2^9	2^{11}	0.13 s	25.5 s	25.6 s	0.91 s	0.5 MB
2^{18}	2^{10}	2^{11}	0.25 s	25.5 s	25.8 s	0.92 s	0.5 MB
2^{20}	2^{11}	2^{11}	0.50 s	25.5 s	26.0 s	0.93 s	0.5 MB
2^{20}	2^{11}	2^{12}	0.50 s	51.0 s	51.5 s	1.00 s	0.5 MB
2^{20}	2^{11}	2^{13}	0.50 s	101.2 s	101.7 s	1.07 s	0.6 MB
2^{20}	2^{11}	2^{14}	0.50 s	203.6 s	204.1 s	1.13 s	0.6 MB
2^{20}	2^{11}	2^{15}	0.50 s	407.1 s	407.6 s	1.20 s	0.7 MB
2^{20}	2^{11}	2^{16}	0.50 s	814.1 s	814.6 s	1.27 s	0.7 MB
2^{20}	2^{11}	2^{17}	0.50 s	1628.2 s	1628.7 s	1.33 s	0.8 MB

Table 3: Prover time of HNSW query under different schemes. “-” indicates failure due to out-of-memory.

$\sum_{h=1}^H \ell_q^{(h)}$	$\ell_q^{(0)}$	zkRAG	SP1 [2]	HyperPlonk [24]
2^{11}	2^{10}	13.3 s	324 min	493 min
2^{11}	2^{11}	26.0 s	671 min	1776 min
2^{11}	2^{12}	51.5 s	1404 min	-
2^{11}	2^{13}	101.7 s	-	-

Table 4: Performance breakdown of zkRAG for a query instance $\sum_{h=1}^H \ell_q^{(h)} = 2^{11}$, $\ell_q^{(0)} = 2^{12}$.

Components		$ \mathcal{P} $	$ \mathcal{V} $	$ \pi $
HybridLookup into $\mathbf{t}_{\mathcal{D}}$		24.8 s	0.154 s	51.1 KB
HybridLookup into $\mathbf{t}_{\mathcal{H}}$		0.16 s	0.021 s	8.63 KB
Greedy Search		0.52 s	0.06 s	54.0 KB
Best First	Proving Set	3.00 s	0.163 s	99.2 KB
	Proving Priority Queue	21.2 s	0.552 s	272.8 KB
	Other Parts	1.84 s	0.050 s	45.7 KB

faster than SP1 and over 5000× faster than HyperPlonk. This highlights zkRAG’s efficiency: our PIOP is highly specialized for priority queue updates and achieves $O(n)$ complexity. In contrast, SP1 and HyperPlonk must simulate heap operations, incurring $O(n \log n)$ prover cost. Moreover, both SP1 and HyperPlonk suffer from prohibitive memory overhead when n grows large, whereas zkRAG runs smoothly across all cases.

6.3 Macrobenchmark

We evaluate the performance of the entire zkRAG in this subsection.

Influence of ℓ_q . For one fixed query, we vary the total search trace length in the greedy search $\sum_{h=1}^H \ell_q^{(h)}$ and in the best-first search $\ell_q^{(0)}$ separately (noting that $\ell_q = \ell_q^{(0)} + \sum_{h=1}^H \ell_q^{(h)} = O(\log N)$), and measure the prover time, verifier time, and proof size of zkRAG. More specifically, we vary $\sum_{h=1}^H \ell_q^{(h)}$ by adjusting the dataset size N , where larger N are simulated by adding randomly generated vectors; vary $\ell_q^{(0)}$ by adjusting the parameter efS in Alg. 3, which determines the size of the heap R at level 0. As shown in Tab. 2, the

prover time for the greedy search in the upper levels ($\mathcal{P}_1$) scales linearly with $\sum_{h=1}^H \ell_q^{(h)}$, while the prover time for the best-first search in the base level ($\mathcal{P}_2$) scales linearly with $\ell_q^{(0)}$. The verifier time and proof size remain sublinear in both $\sum_{h=1}^H \ell_q^{(h)}$ and $\ell_q^{(0)}$.

Influence of d . We vary the dimension d by appending randomly generated coordinates or by truncating from the benchmark vectors, and evaluate its impact on zkRAG. The experiments show that $\mathcal{P}$ time scales linearly with d , with only a small constant offset. Due to space constraints, the details are deferred to Tab. 6 in §J.

Comparison with the baselines. We compare zkRAG against SP1 and HyperPlonk; Tab. 3 summarizes the results. We evaluate four instances where the HNSW search-trace length ℓ_q ranges from 2^{11} to 2^{13} . Across all instances, zkRAG’s $\mathcal{P}$ time is more than $1000\times$ faster than SP1 and more than $2000\times$ faster than HyperPlonk. These results confirm that a specialized PIOP construction can dramatically reduce proof generation time for complex HNSW queries. Directly encoding the program execution logic using general-purpose frameworks (e.g., HyperPlonk’s MLE gadgets) leads to prohibitive prover time and memory overhead.

Performance breakdown and analysis. Tab. 2 and Tab. 4 show that the greedy search on the upper levels incurs only minimal prover cost compared with the best-first search at level 0. Tab. 4 further shows that within the proof for the best-first search at level 0, HybridLookup into $\mathbf{t}_{\mathcal{D}}$ and PQCheck are the main bottlenecks, accounting for 48% and 41% of the total prover time, respectively. For verifier time and proof size $|\pi|$, PQCheck dominates.

Finally, we report query time for HNSW. With $\text{eS} = 64$, we select 100 queries with $\ell_q^{(0)} = 2^{11}$ (greedy search length varies slightly). On average, query time is $t_q = 13.86$ ms, prover time 26.0 s, verifier time 0.93 s, and proof size 0.5 MB. Thus, it shows that although zkRAG is highly optimized, proof generation remains slower than plain queries—an unavoidable overhead.

7 Related Work

The security and privacy of ANNS. The closest related work relies on *authenticated data structures* (ADS) to achieve verifiability for ANNS retrieval over vector databases, where the service provider returns verification objects (VOs). For example, VIS [57] and AN-Proof [45] adapt ADS to ANN graphs (e.g., HNSW), aiming to reduce VO size and prover cost. However, in these approaches, verifier time and proof size still grow with the search trace length (i.e., the number of visited nodes), preventing succinct verification. Besides, these VO-based approaches are inherently *not zero-knowledge*: the VOs are revealed to the verifier and can leak substantial information about the underlying dataset and index structure beyond the retrieval outputs. Another line of work studies privacy-preserving ANNS, aiming to hide the query and/or the database from the server, which is orthogonal to our setting. Representative approaches rely on heavy cryptographic primitives such as homomorphic encryption, ORAM, or PIR [17, 52, 67], and address a different threat model.

Verifiable machine learning. A related line of work studies verifiable machine learning (ML) in zero knowledge [3, 16, 43, 51, 56, 64], with a particular focus on verifiable inference [51, 56, 64] of ML models, including decision trees, deep neural networks, and large language models. Recent advances in zkLLM demonstrate that LLM

inference can be verified efficiently in zero knowledge. However, these works focus exclusively on the execution of lightweight models and do not address the retrieval component required for RAG, leaving this important component underexplored.

8 Discussion

Batching multiple queries. When generating proofs for multiple queries $\mathbf{q}_1, \dots, \mathbf{q}_K$, it is desirable that $\mathcal{V}$ ’s cost is independent of K . Since zkRAG mainly consists of the sumcheck protocol and the PCS, we can adopt standard approaches, as in prior work [15], to *batch* them together. A more noteworthy point is the batching of the PCS. As observed in [15, 33], generating evaluation proofs for different polynomials (even at different points), can be batched at minimal cost to the prover while ensuring that only a single evaluation proof needs to be provided and verified. This ensures the succinctness.

Other ANNS schemes. We choose HNSW as the subject of our study since it is the most widely used ANNS algorithm. Here, we briefly discuss other common ANNS schemes, such as IVF [7]. It is noted that an IVF query has a complexity of $O(\sqrt{N})$, in contrast to HNSW, where the query cost grows logarithmically with the database size N . Therefore, although it may be straightforward to construct a proof for IVF with $O(\sqrt{N})$ prover complexity, the efficiency will be inferior to that of HNSW in this work.

Proving graph construction and update. The HNSW graph construction (§D) can be viewed as N incremental insertions, where each insertion consists of (i) an HNSW query on the current graph state and (ii) a small update to the graph memory. Therefore, a naive but workable proof of construction can be built by composing our proof of query with an authenticated memory update. At a high level, the prover commits to the dataset $\mathbf{t}_{\mathcal{D}}$ and an initial (empty) hierarchy table $\mathbf{t}_{\mathcal{H}}$, and then, for each inserted point, proves that the prescribed search procedure is executed correctly (using the PIOPs in this work) and that the resulting adjacency updates are applied consistently to $\mathbf{t}_{\mathcal{H}}$ via a verifiable RAM. This can be done *one-time* in the offline phase and yields a succinct proof that the final committed $\mathbf{t}_{\mathcal{H}}$ is exactly the output of running the specified HNSW construction algorithm on the committed dataset. Another interesting direction is proving insertions (updates) into an HNSW graph: One can model HNSW updates as m repeated *query-then-write* steps over an authenticated graph: starting from a commitment to the current hierarchy table $\mathbf{t}_{\mathcal{H}}$, the prover checks each insertion using our PIOP on the current committed state, and certifies the resulting adjacency changes as authenticated RAM writes that update the commitment to the next state. These are all offline proofs and the cost of proof generation can be amortized. We leave further improvements to the proof of graph construction as future work.

9 Conclusion

This paper introduces zkRAG, the first zero-knowledge proof that enforces verifiable vector retrieval for RAG: the server proves that the returned results are exactly those obtained by executing a committed HNSW-based ANNS algorithm on a fixed database and a pre-constructed HNSW graph, while revealing nothing beyond the outputs. By designing a PIOP specialized to HNSW query—featuring

efficient graph access, a linear-time priority-queue checker, and succinct dynamic membership verification—zkRAG achieves asymptotically optimal prover time linear in the search trace. Our evaluation demonstrates that zero-knowledge verification of practical RAG retrieval is feasible, and positions zkRAG as a concrete step toward fully verifiable and privacy-preserving RAG-as-a-Service.

References

- [1] 2023. Approximate Nearest Neighbor (ANN) Search Explained: IVF vs HNSW vs PQ. <https://www.pingcap.com/article/approximate-nearest-neighbor-ann-search-explained-ivf-vs-hnsw-vs-pq/>.
- [2] 2024. SP1 zkVM. <https://github.com/succinctlabs/sp1>.
- [3] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. 2024. Zero-Knowledge Proofs of Training for Deep Neural Networks. *Cryptology ePrint Archive, Paper 2024/162*. <https://eprint.iacr.org/2024/162>
- [4] Amazon Web Services. 2024. Knowledge Bases for Amazon Bedrock. <https://aws.amazon.com/bedrock/knowledge-bases/>. Accessed: 2025-07-11.
- [5] Sebastian Angel, Andrew J Blumberg, Eleftherios Ioannidis, and Jess Woods. 2022. Efficient representation of numerical optimization problems for {SNARKs}. In *31st USENIX Security Symposium (USENIX Security 22)*. 4273–4290.
- [6] Sebastian Angel, Eleftherios Ioannidis, Elizabeth Margolin, Srinath Setty, and Jess Woods. 2024. Reef: Fast succinct {Non-Interactive} {Zero-Knowledge} regex proofs. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3801–3818.
- [7] Artem Babenko and Victor Lempitsky. 2014. The inverted multi-index. *IEEE transactions on pattern analysis and machine intelligence* 37, 6 (2014), 1247–1260.
- [8] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P Ward. 2019. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology—EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part I* 38. Springer, 103–128.
- [9] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2014. Succinct {Non-Interactive} zero knowledge for a von neumann architecture. In *23rd USENIX Security Symposium (USENIX Security 14)*. 781–796.
- [10] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2021. Halo infinite: Proof-carrying data from additive polynomial commitments. In *Annual International Cryptology Conference*. Springer, 649–680.
- [11] Sean Bowe, Ariel Gabizon, and Matthew D. Green. 2017. A multi-party protocol for constructing the public parameters of the Pinocchio zk-SNARK. *Cryptology ePrint Archive, Paper 2017/602*. <https://eprint.iacr.org/2017/602>
- [12] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*. IEEE, 315–334.
- [13] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. 2020. Transparent SNARKs from DARK compilers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 677–706.
- [14] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. 2024. Lookup arguments: improvements, extensions and applications to zero-knowledge decision trees. In *IACR International Conference on Public-Key Cryptography*. Springer, 337–369.
- [15] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 499–530.
- [16] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 560–574.
- [17] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. 2020. {SANNs}: Scaling up secure approximate {k-Nearest} neighbors search. In *29th USENIX Security Symposium (USENIX Security 20)*. 2111–2128.
- [18] Y. et al. Chen. 2024. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. *arXiv preprint arXiv:2402.07867* (2024).
- [19] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 738–768.
- [20] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [21] Hans De Nivelle and Ruzica Piskac. 2005. Verification of an off-line checker for priority queues. In *Third IEEE International Conference on Software Engineering and Formal Methods (SEFM’05)*. IEEE, 210–219.
- [22] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [23] Liam Eagen, Dario Fiore, and Ariel Gabizon. 2022. cq: Cashed quotients for fast lookups. *Cryptology ePrint Archive* (2022).
- [24] Espresso Systems. 2023. HyperPlonk. <https://github.com/EspressoSystems/hyperplonk>. GitHub repository.
- [25] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.
- [26] Ulrich Finkler and Kurt Mehlhorn. 1999. Checking priority queues. In *Proceedings of the tenth annual ACM-SIAM symposium on Discrete algorithms*. 901–902.
- [27] Ariel Gabizon and Zachary J Williamson. 2020. plonk: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive* (2020).
- [28] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for ocumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).
- [29] Glaze. 2024. How Small Teams Can Pick Cloud ZK Proof Infrastructure. <https://hackernoon.com/how-small-teams-can-pick-cloud-zk-proof-infrastructure>
- [30] Alexander Golovnev, Jonathan Lee, Srinath TV Setty, Justin Thaler, and Riad S Wahby. 2021. Brakedown: Linear-time and post-quantum SNARKs for R1CS. *IACR Cryptol. ePrint Arch.* 2021 (2021), 1043.
- [31] Google Cloud. 2024. Vector Search Overview. <https://docs.cloud.google.com/vertex-ai/docs/vector-search/overview>. Accessed: 2025-07-11.
- [32] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. 305–326.
- [33] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. 2024. DeepFold: Efficient Multilinear Polynomial Commitment from Reed-Solomon Code and Its Application to Zero-knowledge Proofs. *Cryptology ePrint Archive, Paper 2024/1595*. <https://eprint.iacr.org/2024/1595>
- [34] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive* (2022).
- [35] Hugging Face. [n. d.]. Hugging Face Platform. <https://huggingface.co/>. Accessed: 2025-07-11.
- [36] Jeongyeon Hwang, Junyoung Park, Hyejin Park, Dongwoo Kim, Sangdon Park, and Jungseul Ok. 2025. Retrieval-augmented generation with estimation of source reliability. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 34267–34291.
- [37] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [38] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology—ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5–9, 2010, Proceedings* 16. Springer, 177–194.
- [39] Varun Kumar, Leonard Gleyzer, Adar Kahana, Khemraj Shukla, and George Em Karniadakis. 2023. Mycrunchgpt: A llm assisted framework for scientific machine learning. *Journal of Machine Learning for Modeling and Computing* 4, 4 (2023).
- [40] Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W Lee, and Tae Jun Ham. 2022. Anna: Specialized architecture for approximate nearest neighbor search. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 169–183.
- [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [42] LexisNexis. 2024. Nexis Data Plus. <https://www.lexisnexis.com/en-us/products/nexis-data-plus.page>. Accessed: 2025-07-11.
- [43] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. ZkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2968–2985.
- [44] David G Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International journal of computer vision* 60, 2 (2004), 91–110.
- [45] Lingling Lu, Zhenyu Wen, Ye Yuan, Qiming He, Jianhai Chen, and Zhengguang Liu. 2024. ANNProof: Building a verifiable and efficient outsourced approximate nearest neighbor search system on blockchain. *Future Generation Computer Systems* 156 (2024), 206–220.
- [46] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [47] Microsoft. 2024. Inconsistent and Hallucinatory Responses from Copilot Studio. <https://learn.microsoft.com/en-us/answers/questions/5654951/inconsistent-and-hallucinatory-responses-from-copi>. Microsoft Q&A forum post, accessed: 2025-01-10.

- [48] Microsoft Azure. 2024. Use Your Data with Azure OpenAI. <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/use-your-data>. Accessed: 2025-07-11.
- [49] MindInventory. 2024. What Is RAG as a Service? <https://www.mindinventory.com/blog/what-is-rag-as-a-service/>. Accessed: 2025-07-11.
- [50] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. 2013. Signatures of correct computation. In *Theory of Cryptography Conference*. Springer, 222–242.
- [51] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium (USENIX Security 25)*.
- [52] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. 2022. Private approximate nearest neighbor search with sublinear communication. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 911–929.
- [53] Srinath Setty. 2020. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Annual International Cryptology Conference*. Springer, 704–737.
- [54] Spurthi Setty, Harsh Thakkar, Alyssa Lee, Eden Chung, and Natan Vidra. 2024. Improving retrieval for rag based question answering models on financial documents. *arXiv preprint arXiv:2404.07221* (2024).
- [55] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the Lookup Singularity with Lasso. In *Advances in Cryptology – EUROCRYPT 2024*, 180–209.
- [56] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. *arXiv preprint arXiv:2404.16109* (2024).
- [57] Chenzhao Wang, Jilian Zhang, Xuyang Liu, Kaimin Wei, and Bingwen Feng. 2024. Verifiable Graph-Based Approximate Nearest Neighbor Search. In *International Conference on Advanced Data Mining and Applications*. Springer, 3–17.
- [58] Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu Nkisi-Orji, Ruvan Weerasinghe, Anne Liret, and Bruno Fleisch. 2024. CBR-RAG: case-based reasoning for retrieval augmented generation in LLMs for legal question answering. In *International Conference on Case-Based Reasoning*. Springer, 445–460.
- [59] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*. Springer, 733–764.
- [60] Zhenhao Xu, Mark Jerome Cruz, Matthew Guevara, Tie Wang, Manasi Deshpande, Xiaofeng Wang, and Zheng Li. 2024. Retrieval-augmented generation with knowledge graphs for customer service question answering. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 2905–2909.
- [61] Tan Yu, Wenfei Zhou, Leiyang Leiyang, Aaditya Shukla, Mmadugula Mmadugula, Pritam Gundecha, Nicholas Burnett, Anbang Xu, Viseth Viseth, Tbar Tbar, et al. 2025. EKRA: Benchmark RAG for Enterprise Knowledge Question Answering. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*. 152–159.
- [62] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. 2022. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3121–3134.
- [63] zcash. 2024. The Halo2 zero-knowledge proving system. <https://github.com/zcash/halo2>
- [64] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. 2020. Zero knowledge proofs for decision tree predictions and accuracy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2039–2053.
- [65] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2020. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 859–876.
- [66] Yuncong Zhang, Shi-Feng Sun, and Dawu Gu. 2024. Efficient KZG-Based Univariate Sum-Check and Lookup Argument. In *IACR International Conference on Public-Key Cryptography*. Springer, 400–425.
- [67] Mingxun Zhou, Elaine Shi, and Giulia Fanti. 2024. Pacmann: Efficient private approximate nearest neighbor search. *Cryptology ePrint Archive* (2024).

A Additional Preliminaries

A.1 zkSNARK

An interactive argument for an NP relation $\mathcal{R}$ allows $\mathcal{P}$ to convince $\mathcal{V}$ that there exists a statement-witness pair $(x, w) \in \mathcal{R}$. To qualify as an interactive Argument of Knowledge (AoK), we require w can be efficiently extracted by an extractor $\mathcal{E}$.

Definition 3. $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ is a zero-knowledge interactive argument of knowledge for $\mathcal{R}$ if it satisfies the following properties:

- **Completeness.** For every pp output by $\mathcal{G}(1^\lambda)$, a statement-witness pair (x, w) such that $\mathcal{R}(x, w) = 1$, we have

$$\Pr [\langle \mathcal{P}(w), \mathcal{V} \rangle(x, \text{pp}) = 1] = 1$$

- **Knowledge soundness.** For any PPT prover $\mathcal{P}^*$, there exists a PPT extractor $\mathcal{E}$ such that for every pp output by $\mathcal{G}(1^\lambda)$, any input x , and the extractor’s output $w^* \leftarrow \mathcal{E}^{\mathcal{P}^*}(\text{pp}, x)$, the following probability is $\text{negl}(\lambda)$:

$$\Pr [\langle \mathcal{P}^*, \mathcal{V} \rangle(x, \text{pp}) = 1 \wedge \mathcal{R}(x, w^*) \neq 1]$$

- **Zero knowledge.** There exists a PPT simulator S such that for any PPT algorithm $\mathcal{V}^*$, $(x; w) \in \mathcal{R}$, and public parameters pp output by $\mathcal{G}(1^\lambda)$, it holds that

$$\text{View}(\mathcal{V}^*(\text{pp}, x)) \approx S^{\mathcal{V}^*}(x).$$

In the definition of zero knowledge, $\text{View}(\mathcal{V}^*(\text{pp}, x))$ denotes the view that the verifier observes during the execution of the interactive protocol with $\mathcal{P}$, while $S^{\mathcal{V}^*}(x)$ denotes the view generated by the simulator S given input x and the transcript of $\mathcal{V}^*$. The notation $\approx$ denotes that the two distributions are perfectly indistinguishable. This definition is commonly used in existing transparent zero-knowledge proof schemes [15, 51, 56, 64].

We say it is *succinct* if the running time of the verifier and the total proof size are both $\text{poly}(\lambda, |x|, \log |w|)$.

zkSNARK. A public-coin succinct zero-knowledge interactive AoK can be made non-interactive by applying the Fiat-Shamir transformation in the random oracle model, thereby obtaining a zkSNARK, which includes the algorithms:

- $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow \text{pp}$: It takes the NP relation $\mathcal{R}$ as inputs, and outputs the public parameter pp.
- $\text{Prove}(\text{pp}, x, w) \rightarrow \pi$: It takes pp and a statement-witness pair x, w as inputs, and outputs a proof π .
- $\text{Verify}(\text{pp}, x, \pi) \rightarrow 0/1$: It takes pp, the statement x , and the proof π as inputs, and outputs a bit b indicating acceptance ($b = 1$) or rejection ($b = 0$).

A.2 Polynomial Commitment Scheme

A PCS allows $\mathcal{P}$ to commit to a polynomial f and later prove its evaluation at a given point. The definition of a (univariate or multivariate) PCS is as follows:

Definition 4. A PCS for ℓ -variate polynomials has a tuple of algorithms $(\text{PC.Setup}, \text{PC.Commit}, \text{PC.Open}, \text{PC.Verify})$:

- $\text{PC.Setup}(1^\lambda) \rightarrow \text{pp}$: It outputs a public parameter pp.
- $\text{PC.Commit}(f, \text{pp}) \rightarrow \text{com}_f$: It takes as input a ℓ -variate polynomial f and outputs a commitment com_f .
- $\text{PC.Open}(f, \mathbf{u}, \text{pp}) \rightarrow (z, \pi)$: It outputs $z := f(\mathbf{u})$ for a given evaluation point $\mathbf{u}$ and a corresponding proof π .
- $\text{PC.Verify}(\text{com}_f, \mathbf{u}, z, \pi, \text{pp}) \rightarrow 0/1$: It verifies the proof π using pp, com_f , z and outputs 1 if the proof is valid.

When $\ell = 1$, it is called a univariate PCS; otherwise, it is called a multivariate PCS. The PCS should satisfy completeness, binding, and knowledge soundness. We refer readers to [30] for the formal definitions of these properties.

B Standard Gadgets

We list the definitions and constructions of the gadgets used, including the KZG univariate PCS, the PST multilinear PCS, multivariate sumcheck, zerocheck, prodcheck, permcheck, and the lookup argument. For detailed security proofs of these constructions, we refer the reader to [15, 34, 38, 50].

B.1 KZG Univariate PCS

We instantiate the univariate PCS (uPC) using the KZG [38]. It requires a trusted setup:

- $\text{uPC.Setup}(D) \rightarrow \text{pp}$: It samples $\alpha \xleftarrow{\$} \mathbb{F}$ as a trapdoor and outputs the public parameters $\text{pp} := \{[\alpha^0]_g, \dots, [\alpha^D]_g\}$.
- $\mathcal{P}$ and $\mathcal{V}$ run the following to commit to and check the evaluation of a degree- D univariate polynomial $f(X)$:
 - $\text{uPC.Commit}(f, \text{pp}) \rightarrow \text{com}_f$: Outputs $\text{com}_f := [f(\alpha)]_g$. Note that the evaluation on the power of g can be computed with the aid of prepared parameters efficiently.
 - $\text{uPC.Open}(f, u, \text{pp}) \rightarrow (z, \pi)$: $\mathcal{P}$ evaluates f at u as $z := f(u)$. $\mathcal{P}$ computes polynomials Q that satisfy $f(X) - z = (X - u) \cdot Q(X)$, and outputs a proof $\pi := [Q(\alpha)]_g$.
 - $\text{uPC.Verify}(\text{com}_f, z, u, \pi) \rightarrow 0/1$: $\mathcal{V}$ parses the proof and checks $e(\text{com}_f - [z]_g, [1]_g) \stackrel{?}{=} e([\alpha - u]_g, \pi)$. If it passes, $\mathcal{V}$ outputs 1; 0 otherwise.

Complexity. The prover time is dominated by $O(D)$ group operations, the verifier time is $O(1)$ bilinear pairings operations, and the proof size is $O(1)$ group elements.

B.2 PST Multilinear PCS

We instantiate the multilinear PCS (mPC) using the PST [50], which is a multivariate extension of the scheme in §B.1. Similarly, it requires a trusted setup:

- $\text{mPC.Setup}(\ell) \rightarrow \text{pp}$: It samples $\alpha \xleftarrow{\$} \mathbb{F}^\ell$ as a trapdoor and outputs the public parameters $\text{pp} := \{[\tilde{e}q(\alpha, b)]_g\}_{b \in \{0,1\}^\ell}$.
- $\mathcal{P}$ and $\mathcal{V}$ run the following to commit to and check the evaluation of an ℓ -variate multilinear polynomial f :
 - $\text{mPC.Commit}(f, \text{pp}) \rightarrow \text{com}_f$: Outputs $\text{com}_f := [f(\alpha)]_g$. Note that the evaluation on the power of g can be computed with the aid of prepared parameters efficiently.
 - $\text{mPC.Open}(f, u, \text{pp}) \rightarrow (z, \pi)$: $\mathcal{P}$ evaluates f at u as $z := f(u)$. $\mathcal{P}$ computes polynomials $\{Q_i\}_{i \in [\ell]}$ that satisfy $f(x) - z = \sum_{i=1}^\ell (x_i - u_i) \cdot Q_i(x_{i+1}, \dots, x_\ell)$, and outputs a proof $\pi := \{\pi_i\}_{i \in [\ell]}$, where $\pi_i := [Q_i(\alpha_{i+1}, \dots, \alpha_\ell)]_g$.
 - $\text{mPC.Verify}(\text{com}_f, z, u, \pi) \rightarrow 0/1$: $\mathcal{V}$ parses the proofs and checks $e(\text{com}_f - [z]_g, [1]_g) \stackrel{?}{=} \sum_{i=1}^\ell e([\alpha_i - u_i]_g, \pi_i)$. If it passes, $\mathcal{V}$ outputs 1; 0 otherwise.

Complexity. The prover time is $O(2^\ell)$ group operations, the verifier time is $O(\ell)$ bilinear pairings operations, and the proof size is $O(\ell)$ group elements.

B.3 Sumcheck

Consider the sumcheck relation $\mathcal{R}_{\text{sum}}(\ell, H; f) = 1$ iff $H = \sum_{x \in \{0,1\}^\ell} f(x)$, where f is a degree- D ℓ -variate polynomial and $H \in \mathbb{F}$ is a sum claim. $\mathcal{P}$ and $\mathcal{V}$ run the following:

- 1: In the i -th round, where $1 \leq i \leq \ell$,
 - (1) $\mathcal{P}$ computes $f_i(x) := \sum_{b \in \{0,1\}^{\ell-i}} f(r_1, \dots, r_{i-1}, x, b)$ and sends it $\mathcal{V}$.
 - (2) $\mathcal{V}$ checks $f_i(0) + f_i(1) \stackrel{?}{=} H$. If passes, $\mathcal{V}$ samples $r_i \xleftarrow{\$} \mathbb{F}$ and updates $H \leftarrow f_i(r_i)$. If $i < \ell$, $\mathcal{V}$ sends back r_i .
- 2: $\mathcal{V}$ checks the correctness of $f(r_1, \dots, r_\ell)$ through the polynomial oracle of f .

In the last step, the oracle of $f(r)$ for a multilinear f is often instantiated with a multilinear PCS. For a high-degree polynomial f , we treat f as the output of an arithmetic circuit of multiple multilinear polynomials evaluations.

Complexity. The prover time is $O(2^\ell)$, the verifier time and proof size are both $O(\ell)$, and the round complexity is $O(\ell)$.

Security. According to the protocol description, it is perfectly complete. By the Schwartz–Zippel lemma, each round incurs a knowledge soundness error of $\frac{1}{|\mathbb{F}|}$. By the union bound, the total knowledge soundness error is at most $\frac{\ell}{|\mathbb{F}|}$.

B.4 Zerocheck

Consider a zerocheck relation $\mathcal{R}_{\text{zero}}(\ell; f) = 1$ iff for any $x \in \{0,1\}^\ell$, $f(x) = 0$, where f is a degree- D ℓ -variate polynomial. $\mathcal{P}$ and $\mathcal{V}$ run the following:

- 1: $\mathcal{V}$ sends $\mathcal{P}$ a random challenge $r_2 \xleftarrow{\$} \mathbb{F}^\ell$.
- 2: $\mathcal{P}$, $\mathcal{V}$ run a sumcheck on $\sum_{x \in \{0,1\}^\ell} f(x) \cdot \tilde{e}q(x, r_2) = 0$.

Complexity. $\mathcal{V}$ can evaluate $\tilde{e}q(r_1, r_2)$ locally in $O(\ell)$ time, where r_1 is the randomness used in the sumcheck protocol. Since zerocheck can be reduced to a degree- $(D+1)$ sumcheck, the efficiency metrics are analogous.

Security. This protocol is perfect complete. The knowledge soundness error is $\frac{O(2^\ell)}{|\mathbb{F}|}$.

B.5 Prodcheck

Consider a prodcheck relation $\mathcal{R}_{\text{prod}}(\ell, H; f) = 1$ iff $H = \prod_{x \in \{0,1\}^\ell} f(x)$, where f is a ℓ -variate polynomial and $H \in \mathbb{F}$ is a product claim. $\mathcal{P}$ and $\mathcal{V}$ run the following:

- 1: Let v denote an $(\ell+1)$ -variate polynomial such that $v(0, x) = f(x)$ and $v(1, x) = v(x, 0) \cdot v(x, 1)$, for any $x \in \{0,1\}^\ell$. $\mathcal{P}$ computes v and sends com_v to $\mathcal{V}$.
- 2: $\mathcal{P}$ and $\mathcal{V}$ run zerocheck on $p(x) := v(0, x) - f(x)$ and $q(x) := v(1, x) - v(x, 0) \cdot v(x, 1)$.
- 3: $\mathcal{P}$ and $\mathcal{V}$ run mPC.Open on $v(1, \dots, 1, 0) = H$.

Complexity. Since it is reduced to zerocheck, and computing v requires $O(2^\ell)$ time, the efficiency metrics are analogous.

Security. The protocol is perfect complete and the knowledge soundness error is $\frac{O(2^\ell)}{|\mathbb{F}|}$.

B.6 Permcheck

Consider a permcheck relation $\mathcal{R}_{\text{perm}}(\ell, \sigma; f, g) = 1$ iff for any $x \in \{0,1\}^\ell$, $f(x) = g(\sigma(x))$, where f, g are two ℓ -variate linear polynomials, and $\sigma : \{0,1\}^\ell \rightarrow \{0,1\}^\ell$ is a public permutation. $\mathcal{P}$ and $\mathcal{V}$ run the following:

- 1: Let s_{id}, s_σ denote ℓ -variate multilinear polynomials such that for any $x \in \{0, 1\}^\ell$, $s_{id}(x) := \text{bin}^{-1}(x)$, $s_\sigma(x) := \text{bin}^{-1}(\sigma(x))$. $\mathcal{P}$ sends $\text{com}_f, \text{com}_g, \text{com}_{s_{id}}, \text{com}_{s_\sigma}$ to $\mathcal{V}$.
- 2: $\mathcal{V}$ samples $\alpha, \beta \xleftarrow{\$} \mathbb{F}$ and sends them to $\mathcal{P}$.
- 3: $\mathcal{P}, \mathcal{V}$ run a prodcheck on $\prod_{x \in \{0, 1\}^\ell} h(x) = 1$, where $h(x) := \frac{f(x) + \alpha s_{id}(x) + \beta}{g(x) + \alpha s_\sigma(x) + \beta}$ and a zerocheck on $q(x) := h(x) \cdot (g(x) + \alpha s_\sigma(x) + \beta) - (f(x) + \alpha s_{id}(x) + \beta)$.

Complexity. Since the permcheck is reduced to the prodcheck and zerocheck, the efficiency are analogous.

Security. The protocol is perfect complete. The knowledge soundness error is $\frac{O(2^\ell)}{|\mathbb{F}|}$.

B.7 LogUp

Consider the lookup relation $\mathcal{R}_{lookup}(N, L; t, q) = 1$ iff $q \subset t$, where $q \in \mathbb{F}^L$, $t \in \mathbb{F}^N$, and the multilinear polynomials q and t are the MLEs of q and t , respectively. The LogUp [34] scheme is based on Eq. (1), where $\mathcal{P}$ and $\mathcal{V}$ run the following:

- 1: Let m_i denote the number of times the element t_i appears in q for $i \in [N]$, and $m(x)$ be the MLE of m . $\mathcal{P}$ computes $\text{com}_t, \text{com}_q, \text{com}_m$ and sends to $\mathcal{V}$. $\mathcal{V}$ samples $r \xleftarrow{\$} \mathbb{F}$ and sends back to $\mathcal{P}$.
- 2: Let $A_i = \frac{m_i}{t_i + r}$ for $i \in [N]$ and $B_i = \frac{1}{q_i + r}$ for $i \in [L]$, and let $A(x), B(x)$ be the MLEs of A, B . $\mathcal{P}$ computes $\text{com}_A, \text{com}_B$ and sends them to $\mathcal{V}$.
- 3: $\mathcal{P}$ and $\mathcal{V}$ run Zerocheck on $A(x) \cdot (t(x) + r) - m(x)$, $B(x) \cdot (q(x) + r) - 1$ to check the well-formedness of A, B .
- 4: Let H be the summation of LHS (RHS) in Eq. (1). $\mathcal{P}, \mathcal{V}$ run Sumcheck on $H = \sum_{x \in \{0, 1\}^{\log N}} A(x)$ and $H = \sum_{x \in \{0, 1\}^{\log L}} B(x)$.

LogUp can be easily extended to support $\mathcal{R}_{idlookup}, \mathcal{R}_{batchlookup}$ using standard techniques.

Complexity. The prover time is $O(N + L)$, the verifier time and proof size are both $O(\log(NL))$.

Security. The completeness error is $\frac{O(N+L)}{|\mathbb{F}|}$, as it requires that the randomness r does not zero out denominators in $\{A_i, B_i\}$. The knowledge soundness error is $\frac{O(N+L)}{|\mathbb{F}|}$.

C Defining Zero-Knowledge RAG Retrieval

Recall that in our cloud-hosted RAG setting, the embedding table t_D (the underlying vector database) and the hierarchy table t_H (encoding the HNSW graph topology) are fixed to provide a consistent retrieval service across clients. Both t_D and t_H are treated as the service provider's intellectual property. Accordingly, the provider preprocesses them, commits to them, and publishes the commitments com_{t_D} and com_{t_H} . In addition, the prescribed HNSW hyperparameters $H, \text{ep}_{\text{default}}, K, \text{efC}, \text{efS}$, and k are fixed and public. We denote the above public hyperparameters by pp_R . We designate a slightly modified HNSW retrieval algorithm (Algorithm Alg. 1) that, besides returning k vector results topk , also outputs auxiliary public metadata consisting of a per-level trace-length vector ℓ_q recording the total number of iterations of the inner for-loop at each level, and the element $\text{minC}_{\text{final}}$ with the minimum value in C_{final} , and the element $\text{maxR}_{\text{final}}$ with the maximum value in R_{final} (where C_{final} and R_{final} are the final best-first-search heap states),

which all leak little information about t_D or t_H and can hardly been utilized. For brevity, we omit $\text{minC}_{\text{final}}$ and $\text{maxR}_{\text{final}}$ from the notation below, as they are handled identically to ℓ_q .

The RAG retrieval service is as follows. Given a query embedding $q \in \mathbb{F}^d$, the designated retrieval algorithm outputs a result set together with the per-level trace-length vector

$$(\text{topk}, \ell_q) \leftarrow \text{Retrieve}(t_D, t_H, pp_R, q),$$

where topk is subsequently consumed by a downstream language model. We now define a zero-knowledge verifiable RAG retrieval scheme zkRAG, consisting of the following algorithms:

- $\text{pp} \leftarrow \text{zkRAG.Setup}(1^\lambda)$: on input the security parameter, generate public parameters pp .
- $(\text{com}_{t_D}, \text{com}_{t_H}) \leftarrow \text{zkRAG.Commit}(t_D, t_H; \text{pp}, r)$: commit to the embedding table and hierarchy table using randomness r .
- $(\text{topk}, \ell_q, \pi) \leftarrow \text{zkRAG.Prove}(t_D, t_H, pp_R, q; \text{pp})$: execute Retrieve on q to obtain (topk, ℓ_q) and generate a proof π .
- $\{0, 1\} \leftarrow \text{zkRAG.Verify}(\text{com}_{t_D}, \text{com}_{t_H}, pp_R, q, \text{topk}, \ell_q, \pi; \text{pp})$: verify the proof and accept iff π is valid.

Here π is a zero-knowledge proof produced by the prover, and the verifier outputs 1 upon acceptance and 0 otherwise.

Security Definition. We say that a scheme zkRAG is a *zero-knowledge verifiable RAG retrieval scheme* if it satisfies the following properties.

Definition 5. A scheme zkRAG satisfies:

- **Completeness.** For any tables t_D, t_H , public retrieval hyperparameters pp_R , and any query q , letting $(\text{topk}, \ell_q, \pi) \leftarrow \text{zkRAG.Prove}(t_D, t_H, pp_R, q; \text{pp})$ and $(\text{com}_{t_D}, \text{com}_{t_H}) \leftarrow \text{zkRAG.Commit}(t_D, t_H; \text{pp}, r)$, it holds that

$$\Pr[\text{zkRAG.Verify}(\text{com}_{t_D}, \text{com}_{t_H}, pp_R, q, \text{topk}, \ell_q, \pi; \text{pp}) = 1] = 1.$$

- **Soundness.** For any PPT adversary $\mathcal{A}$, the following probability is negligible in λ :

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{zkRAG.Setup}(1^\lambda) \\ (t_D^*, t_H^*, pp_R^*, q^*, \text{topk}^*, \ell_q^*, \pi^*, r^*) \leftarrow \mathcal{A}(1^\lambda, \text{pp}) \\ (\text{com}_{t_D}^*, \text{com}_{t_H}^*) \leftarrow \text{zkRAG.Commit}(t_D^*, t_H^*; \text{pp}, r^*) \\ \text{zkRAG.Verify}(\text{com}_{t_D}^*, \text{com}_{t_H}^*, pp_R^*, q^*, \text{topk}^*, \ell_q^*, \pi^*; \text{pp}) = 1 \\ (\text{topk}^*, \ell_q^*) \neq \text{Retrieve}(t_D^*, t_H^*, pp_R^*, q^*) \end{array} \right].$$

- **Zero Knowledge.** For security parameter λ , let $\text{pp} \leftarrow \text{zkRAG.Setup}(1^\lambda)$.

For any PPT distinguisher $\mathcal{A}$, and any fixed tables (t_D, t_H) and public hyperparameters pp_R , there exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that the following two experiments are computationally indistinguishable:

Real $_{\mathcal{A}, (t_D, t_H), pp_R}(\text{pp})$:

- (1) $(\text{com}_{t_D}, \text{com}_{t_H}) \leftarrow \text{zkRAG.Commit}(t_D, t_H; \text{pp}, r)$
- (2) $q \leftarrow \mathcal{A}(\text{pp}, \text{com}_{t_D}, \text{com}_{t_H})$
- (3) $(\text{topk}, \ell_q, \pi) \leftarrow \text{zkRAG.Prove}(t_D, t_H, pp_R, q; \text{pp})$
- (4) $b \leftarrow \mathcal{A}(\text{pp}, \text{com}_{t_D}, \text{com}_{t_H}, pp_R, q, \text{topk}, \ell_q, \pi)$
- (5) Output b

Algorithm 4 HNSW_CONSTRUCT($\mathcal{D}$, K , efC)

```

1: Input: dataset  $\mathcal{D}$ , degree cap  $K$ , construction queue size efC
2: Output: multi-level graph  $G$  with max level  $H$  and entry node  $start$ 
3: Initialize empty  $G$ ;  $H \leftarrow -1$ ;  $start \leftarrow \text{null}$ 
4: for each point  $x \in \mathcal{D}$  do
5:    $L_x \leftarrow \text{SAMPLELEVEL}()$ 
6:   if  $L_x > H$  then  $H \leftarrow L_x$ ; if  $start = \text{null}$  then  $start \leftarrow x$ 
7:    $ep \leftarrow start$ 
8:   for  $\ell = H$  down to  $L_x + 1$  do
9:      $ep \leftarrow \text{GREEDYSEARCH}(G, \ell, ep, x)$ 
10:  for  $\ell = \min(H, L_x)$  down to 0 do
11:     $S_\ell \leftarrow \text{HNSW\_QUERY}(G, \ell, ep, x, \text{efC}, \text{efC})$ 
12:     $M_{\text{cap}} \leftarrow (\ell = 0) ? 2K : K$ 
13:     $N_\ell \leftarrow \text{SELECTNEIGHBORS}(x, S_\ell, M_{\text{cap}})$ 
14:    for each  $y \in N_\ell$  do
15:       $\text{adj}_\ell[x] \leftarrow \text{adj}_\ell[x] \cup \{y\}$ 
16:       $\text{adj}_\ell[y] \leftarrow \text{adj}_\ell[y] \cup \{x\}$ 
17:      if  $|\text{adj}_\ell[x]| > M_{\text{cap}}$  then
18:         $\text{adj}_\ell[x] \leftarrow \text{SELECTNEIGHBORS}(x, \text{adj}_\ell[x], M_{\text{cap}})$ 
19:      if  $|\text{adj}_\ell[y]| > M_{\text{cap}}$  then
20:         $\text{adj}_\ell[y] \leftarrow \text{SELECTNEIGHBORS}(y, \text{adj}_\ell[y], M_{\text{cap}})$ 
21:     $ep \leftarrow \text{ARGMINDIST}(N_\ell \cup \{ep\})$ 
22:    if  $L_x > \text{level}(start)$  then  $start \leftarrow x$ 
23: for  $\ell = H$  down to 0 do
24:    $M_{\text{cap}} \leftarrow (\ell = 0) ? 2K : K$ 
25:   for each  $x$  with  $|\text{adj}_\ell[x]| < M_{\text{cap}}$  do
26:      $C \leftarrow \bigcup_{z \in \text{adj}_\ell[x]} \text{adj}_\ell[z] \setminus (\text{adj}_\ell[x] \cup \{x\})$ 
27:     for  $y \in \text{TOPK\_BY\_DIST}(x, C, M_{\text{cap}} - |\text{adj}_\ell[x]|)$  do
28:        $\text{adj}_\ell[x] \leftarrow \text{SELECTNEIGHBORS}(x, \text{adj}_\ell[x] \cup \{y\}, M_{\text{cap}})$ 
29:        $\text{adj}_\ell[y] \leftarrow \text{SELECTNEIGHBORS}(y, \text{adj}_\ell[y] \cup \{x\}, M_{\text{cap}})$ 
30: return  $(G, H, start)$ 

```

Algorithm 5 SELECTNEIGHBORS(x, S, M)

```

1: Input: node  $x$ , candidate set  $S$  returned by query, target degree  $M$ 
2: Output: pruned neighbor set  $N$  with  $|N| \leq M$ 
3:  $C \leftarrow$  elements of  $S$  sorted by increasing  $\text{dist}(x, \cdot)$ 
4:  $N \leftarrow \emptyset$ 
5: for each  $y \in C$  do
6:   if  $\forall z \in N : \text{dist}(x, y) < \text{dist}(y, z)$  then
7:      $N \leftarrow N \cup \{y\}$ 
8:   if  $|N| = M$  then break
9: return  $N$ 

```

Ideal $_{\mathcal{A}, S, pp_R}(\text{pp})$:

- (1) $(\text{com}_{t_D}, \text{com}_{t_H}) \leftarrow S_1(1^\lambda, \text{pp})$
- (2) $q \leftarrow \mathcal{A}(\text{pp}, \text{com}_{t_D}, \text{com}_{t_H})$
- (3) $(\text{topk}, \ell_q, \pi) \leftarrow S_2(\text{pp}, \text{com}_{t_D}, \text{com}_{t_H}, pp_R, q)$, given oracle access to $(\text{topk}, \ell_q) = \text{Retrieve}(t_D, t_H, pp_R, q)$
- (4) $b \leftarrow \mathcal{A}(\text{pp}, \text{com}_{t_D}, \text{com}_{t_H}, pp_R, q, \text{topk}, \ell_q, \pi)$
- (5) **Output** b

The outputs of the two experiments differ with probability at most $\text{negl}(\lambda)$.

D Detailed HNSW Construction Algorithm

Alg. 4 is our detailed HNSW construction algorithm. In Alg. 4, the procedure $\text{SAMPLELEVEL}(\cdot)$ can in principle be chosen arbitrarily. The most common choice, adopted in the original HNSW paper [46], is an exponentially decaying distribution defined as

$$\Pr[L \geq \ell] = K^{-\ell}, \quad \ell \geq 0,$$

where K denotes the maximum degree parameter. Our experiments rely on the FAISS implementation [22], which also follows this distribution. Consequently, the expected number of nodes present at level ℓ is

$$N_\ell = N \cdot K^{-\ell},$$

where N is the total number of the dataset $\mathcal{D}$.

After executing Alg. 4, we obtain a hierarchical graph structure defined over all points in $\mathcal{D}$, together with neighborhood relations at each level. In particular, level 0 contains exactly all points in $\mathcal{D}$. It is worth noting that in each level $h > 1$, all nodes reach the maximum degree of K and in level 0, all nodes reach the maximum degree of $2K$.

E Proofs of Theorem 1 and Theorem 2

Proof of Theorem 1. The Detailed proof of Theorem 1 is:

PROOF. ($\Rightarrow$) If σ is a legal multiset operation sequence, then by Definition 1, there exists an injective matching π from deletes to inserts satisfying causality and value agreement. Let $p_i = \pi(i)$ for each delete operation del_i , and set p_i arbitrarily otherwise. The injectivity of π ensures that the p_i values are distinct among deletes. The remaining unmatched insertions define PQ_{final} .

($\Leftarrow$) Conversely, given such a pointer list P satisfying the listed conditions, we may define a matching $\pi(i) = p_i$ for all delete operations i . Conditions (1)–(4) ensure that this matching satisfies the requirements of Definition 1, and (5) defines the correct final queue state. Hence σ is a legal multiset operation sequence. $\square$

Revisit to Theorem 2. [Static Certificate for Minimum Priority Queue Update Correctness] Define the *lifetime* of insertion ins_i as the interval (t_i, t_{d_i}) , where t_{d_i} is the time of deletion of id_i if it is ever deleted, or ∞ otherwise. Then define the *maximum lower bound* for ins_i as:

$$\text{maxlb}_i = \max \{v'_j \mid \text{del}_j \text{ occurs in } (t_i, t_{d_i})\} \cup \{-\infty\}$$

The sequence $\sigma = (\sigma_{\text{ins}}, \sigma_{\text{del}})$ is a valid execution trace of a minimum priority queue if and only if the following two conditions hold: (i) There exists a certificate $\mathbf{p}$ satisfying the conditions of Theorem 1; (ii)

$$\forall i \in [m] : v_i \geq \text{maxlb}_i$$

PROOF. ($\Rightarrow$) Suppose σ is a valid execution trace of a priority queue. According to Theorem 1, the certificate P exists. Furthermore, by definition, each deletion removes the current minimum among live keys. It follows that for every insertion $\text{ins}_i = (t_i, \text{id}_i, v_i)$, the key v_i must be no smaller than any key deleted during its lifetime (t_i, t'_i) , where t'_i is the time it is deleted or ∞ if unmatched. Hence, $v_i \geq \text{maxlb}_i$ holds for all i .

($\Leftarrow$) Conversely, suppose there exists a pointer list P certifying multiset operation completeness (i.e., all conditions of Theorem 1

hold), and for each insertion ins_i , maxlb_i satisfies its definition and Equation 6. According to Theorem 1, σ is a legal multiset sequence.

Consider any deletion $\text{del}_k = (t_k, \text{id}_k, v_k)$. At time t_k , let S_k be the set of all elements still alive. For each $\text{ins}_i \in S_k$, its key v_i satisfies $v_i \geq \text{maxlb}_i \geq v_k$, since del_k occurs during its lifetime. Hence v_k is less than or equal to all live elements at time t_k , and thus is a valid minimum. Therefore, σ constitutes a valid priority queue execution. $\square$

F Detailed Construction of Gadgets in zkRAG

F.1 Details and Analysis of HybridLookup

Offline phase. Let $t(X) = \sum_{i \in [N]} t_i L_i(X)$ denote the univariate extension of t , where $\{L_i(X)\}$ are the Lagrange polynomials defined over the N -th roots of unity Ω_N . In the offline phase, $\mathcal{P}$ commits to the table t (i.e., $t(X)$) and runs the following preprocessing procedure, similar to CQ [23]. Given pp from the KZG setup (§B.1), $\mathcal{P}$ computes:

- 1: $\text{com}_t = [t(\alpha)]_g$, where α is the trapdoor.
- 2: $\text{com}_Z = [\alpha^N - 1]_g$, where $Z(X) = X^N - 1$.
- 3: $[L_i(\alpha)]_g$ and $[Q_i]_g := [q_i(\alpha)]_g$ such that $L_i(X) \cdot t(X) = t_i \cdot L_i(X) + Z(X) \cdot q_i(X)$, for $i \in [N]$.

Offline complexity. We claim that the above procedure requires $O(N \log N)$ group and field operations, as analyzed below: Clearly, com_t and com_Z can be computed with $O(N)$ group operations using the KZG parameters. Moreover, $\{L_i(\alpha)\}_g$ can be computed in $O(N \log N)$ group operations via FFT [11, Section 3.3]. Finally, according to [23, Lemma 3.1], $\{[Q_i]_g\}$ can be computed in $O(N \log N)$ group and field operations.

Security of HybridLookup. We first analyze completeness. The most crucial part is the equation $\text{com}_Q = \sum_{i \in [N]} A_i \cdot [Q_i]_g$ in Step 2 and Step 4. The former is proved in [23, Theorem 3.2] using the sparsity of $A(X)$. The latter equation, $\sum_{X \in \Omega_N} A(X) = N \cdot A(0)$, follows from the univariate sumcheck lemma in [8, Section 2.1]. Finally, according to the description of the protocol, it uses the logarithmic derivative lemma [34], ensuring that $\mathcal{V}$ cannot sample a challenge that makes the denominator zero, which would otherwise cause a completeness error of $\frac{O(L+N)}{|\mathbb{F}|}$.

Considering soundness, by the Schwartz–Zippel lemma, Step 2 incurs a knowledge soundness error of $\frac{O(N)}{|\mathbb{F}|}$; Step 3 incurs an error of $\frac{O(L)}{|\mathbb{F}|}$; and Step 5 incurs an error of $\frac{O(\log L)}{|\mathbb{F}|}$. By the union bound, the total knowledge soundness error is at most $\frac{2N+2L+\log L}{|\mathbb{F}|}$.

F.2 PIOP for the Nearest Greater Neighbors

The detailed algorithm of computing the nearest greater neighbors in a sequence is in Alg. 6.

Formally, we encode each intermediate state of the stack and each processed element into arrays $c, \text{top}, \text{top}', \text{top}_{\text{last}} \in \mathbb{N}^{2k \times 4}$, $f \in \mathbb{N}^{2k}$, where each entry $x \in \mathbb{N}^4$ is of the form $x = (i, t, \text{aux}, v)$, representing the index, tag, auxiliary data, and comparison value. A distinguished null element is written as $\perp_{\text{start}} = (-1, -1, 0, M)$, where M is a value larger than any possible distances. For example, the embedding dataset we use has the embedding value all being integers in $[0, 2^8]$, then the valid distance range is $[0, 2^{16} \cdot d]$. We can set a $M \geq 2^{16} \cdot d$. The following theorem states that if these arrays

Algorithm 6 Nearest Greater Neighbors in a Sequence

```

1: Input: A sequence of elements  $\{(t_i, \text{aux}_i, v_i)\}_{i=0}^{k-1}$  sorted by  $t_i$ 
2: Output: Arrays  $\text{prev}[0..k-1], \text{next}[0..k-1]$ 
3: Initialize  $\text{prev}[0..k-1] \leftarrow -1, \text{next}[0..k-1] \leftarrow k$ 
4: Initialize empty stack  $S$ 
5: for  $j = 0$  to  $k - 1$  do
6:   while  $S$  not empty and  $v[\text{top}(S)] \leq v[j]$  do
7:      $u \leftarrow \text{pop}(S)$ 
8:      $\text{next}[u] \leftarrow j$ 
9:   if  $S$  not empty then
10:     $\text{prev}[j] \leftarrow \text{top}(S)$ 
11:     $\text{push}(S, j)$ 
12: for each  $u$  in  $S$  (in order from top to bottom) do
13:    $\text{next}[u] \leftarrow k$ 

```

satisfy the listed transition constraints—which capture the same push/pop dynamics of Algorithm 6—then the constructed pointers (prev, next) are exactly the output of this algorithm, thus being the same as their claimed definition.

Theorem 3. If the following constraints hold:

Boolean and counting constraints. The selector f_j is a Boolean and that exactly k pushes occur:

$$\forall j \in [2k], f_j(1 - f_j) = 0, \quad (28)$$

$$\sum_{j=0}^{2k-1} f_j = k. \quad (29)$$

Stack initialization and null conditions. Pops cannot occur from an empty stack: $\forall j \in [2k],$

$$(1 - f_j) \cdot \mathbf{1}_{\{\text{top}_j = \perp_{\text{start}}\}} = 0. \quad (30)$$

Counter consistency. The counter c_j tracks the number of pushes: $\forall j \in [2k-1],$

$$c_{j+1}[0] = c_j[0] + f_j, \quad c_0[0] = 0. \quad (31)$$

Element validity. Every consumed element must come from the input sequence or a special null:

$$c_j \subset \{(i, t_i, e_i) : i \in [k]\} \cup \{(k, T, 0, M)\}. \quad (32)$$

Stack transitions. The stack state is threaded correctly from one step to the next: $\forall j \in \{1, 2, \dots, 2k-1\},$

$$\text{top}_j = \text{top}'_{j-1}, \quad \text{top}_0 = \perp_{\text{start}}, \quad \text{top}'_{2k-1} = \perp_{\text{start}}. \quad (33)$$

Push vs. pop rule. Depending on f_j , either a new element is pushed or a previous one is popped: $\forall j \in [2k]$

$$f_j = 1 \rightarrow \text{top}'_j = c_j \wedge f_j = 0 \rightarrow \text{top}'_j = \text{top}_{\text{last}, j}. \quad (34)$$

Monotonicity condition. The stack must be strictly decreasing in the value field: $\forall j \in [2k],$

$$f_j(\text{top}_j[3] - c_j[3]) + (1 - f_j)(c_j[3] - \text{top}_j[3] - 1) \geq 0. \quad (35)$$

Last-top tracking. This maintains the pointer to the most recent surviving top element:

$$(\text{top}_{\text{last}}, \text{top}) \subset (f \circ \text{top}, f \circ \text{top}') \cup \{(\perp_{\text{start}}, \perp_{\text{start}})\}. \quad (36)$$

Then we can construct prev and next as follows:

$$\text{prev}[c_j[0]] := \text{top}_j[0] \quad (f_j = 1),$$

$$\text{next}[\text{top}_j[0]] := c_j[0] \quad (f_j = 0).$$

Then, (prev, next) coincides with the output of Algorithm 6.

PROOF OF THEOREM 3. For every step j , we consider c_j as the current element to be pushed or not. Concretely, if c_{j-1} was not pushed at time $j-1$, we consider $c_j = c_{j-1}$. If c_{j-1} was pushed at time $j-1$, then the next row in (t, e) becomes the new c being considered.

We consider $f_j \in \{0, 1\}$ as the *push flag* (1=push, 0=pop), $\text{top}_j, \text{top}'_j$ as the stack top before/after the step, and $\text{top}_{\text{last},j}$ as the second-to-top element before the step. The comparison gate enforces: if $f_j = 0$ then $\text{top}_j[3] \leq c_j[3]$ (allowing pop), while if $f_j = 1$ then $\text{top}_j[3] > c_j[3]$ (allowing push). Assume there is a witness satisfies all constraints, and define

$$\text{prev}[c_j[0]] := \text{top}_j[0] \quad (f_j = 1), \quad (37)$$

$$\text{next}[\text{top}_j[0]] := c_j[0] \quad (f_j = 0). \quad (38)$$

We prove that prev, next are exactly the outputs of MONOTONOUSSTACK.

- *Input indexing.* The constraints (28) and (29) imply the k pushes of the stack.

The recurrence constraints (31) ensure push steps enumerate indices $0, 1, \dots, k-1$ without repetition or gaps. Note that there is a boundary case that: when all pushes have happened, the constraint want to take a look at the input with indice k which doesn't exist.

Thus, we add a element equal to k in (32), which guarantees the correctness in boundary cases. The constraint (32) force c_j to equal the corresponding input element, so each push introduces the correct value from the public input and the boundary case has been solved.

- *Stack initialization and termination.* The constraints (33) with the condition (30) excluding pops on a null $\perp_{\text{start}}$, guarantee that every pop is well-defined and that the stack returns to empty after exactly $2k$ steps.
- *Step semantics.* The constraint (35) ensure that whenever $\text{top}_j[3] \leq c_j[3]$, a pop must occur, and whenever $\text{top}_j[3] > c_j[3]$, a push must occur.

The constraint (34) together with the condition (36) for proving all second-to-top element $\text{top}_{\text{last},j}$'s definition, formalizes the standard stack update: pop moves the second-to-top up, push inserts the new element while demoting the previous top. Hence the constraints force the same control flow as the while-loop in MONOTONOUSSTACK.

- *Correctness of neighbor relations.* By induction on the input index i , after processing i the stack is strictly decreasing in the v -coordinate. When a pop occurs, the popped element u satisfies $v_u \leq v_i$ and i is precisely its nearest greater neighbor to the right, so $\text{next}[u] = i$ is exactly the output of the algorithm. When a push occurs, the current top is strictly larger than v_i , hence it is the nearest greater neighbor on the left of i , so $\text{prev}[i] = \text{top}_j[0]$ is exactly the output of the algorithm. Thus the prev, next arrays coincide with those of the algorithm. $\square$

We build our PIOP for the nearest greater neighbors based on Theorem 3. This theorem guarantees the soundness of our PIOP. The completeness of our PIOP is straightforward.

We consider The Nearest Greater Neighbors Relation $\mathcal{R}_{\text{NGN}}(k; t, e, c, f, \text{top}, \text{top}_{\text{last}}, \text{top}') = 1$ iff: let prev, next $\in \{0, \dots, k-1\}^k$ be defined as $\text{prev}[c_j[0]] := \text{top}_j[0]$, where $f_j = 1$, $\text{next}[\text{top}_j[0]] := c_j[0]$, where $f_j = 0$. Then when $\{(t_i, e_i)\}_{i=0}^k$ is the input, prev and next are the output of Alg. 6. Here, $t \in \mathbb{F}^k, e \in \mathbb{F}^{k \times 2}, c, \text{top}, \text{top}_{\text{last}}, \text{top}' \in \mathbb{F}^{2k \times 4}, f \in \mathbb{F}^{2k}$.

We present the PIOP for the relation $\mathcal{R}_{\text{NGN}}$:

- 1: Let $t(x), e(x), c(x), f(x), \text{top}(x), \text{top}_{\text{last}}(x), \text{top}'(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_t, \text{com}_e, \text{com}_c, \text{com}_{\text{top}}, \text{com}_{\text{top}_{\text{last}}}$ and $\text{com}_{\text{top}'}$.
- 2: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $f(x) \cdot (1 - f(x))$.
- 3: $\mathcal{P}$ and $\mathcal{V}$ run a Sumcheck to check $\sum_{j=0}^{2k-1} f(x) = k$.
- 4: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck and a LogUp to check $(1 - f) \circ \text{top}[0] \subset [0, 2k]$.
- 5: Let $\text{mask} \in \mathbb{F}^{2k}$ s.t. $\text{mask} = 1$ if $i = 2k-1$ and 0 otherwise.
- 6: $\mathcal{P}$ constructs $c^{\text{shift}}, \text{top}^{\text{shift}}, \text{top}_{\text{last}}^{\text{shift}}$ s.t.
 - $c_i^{\text{shift}} = c_{i+1}$ for $i \neq 2k-1$, and $c_{2k-1}^{\text{shift}} = c_0$;
 - $\text{top}_i^{\text{shift}} = \text{top}_{i+1}$ for $i \neq 2k-1$, and $\text{top}_{2k-1}^{\text{shift}} = \text{top}_0$;
 - $\text{top}_{\text{last},i}^{\text{shift}} = \text{top}_{\text{last},i+1}$ for $i \neq 2k-1$, and $\text{top}_{\text{last},2k-1}^{\text{shift}} = \text{top}_{\text{last},0}$.
- 7: Let $\text{mask}(x), c^{\text{shift}}(x), \text{top}^{\text{shift}}(x), \text{top}_{\text{last}}^{\text{shift}}(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_{c^{\text{shift}}}, \text{com}_{\text{top}^{\text{shift}}}$ and $\text{com}_{\text{top}_{\text{last}}^{\text{shift}}}$.
- 8: $\mathcal{P}$ and $\mathcal{V}$ run several Permcheck to check the correctness of $c^{\text{shift}}(x), \text{top}^{\text{shift}}(x)$ and $\text{top}_{\text{last}}^{\text{shift}}(x)$.
- 9: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - \text{mask}(x)) \cdot (c^{\text{shift}}(x \| \text{bin}_2(0)) - c(x \| \text{bin}_2(0)) - f(x))$.
- 10: $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check $c \subset (i, t, e) \cup \{(k, T, 0, M)\}$.
- 11: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - \text{mask}(x)) \cdot (\text{top}^{\text{shift}}(x) - \text{top}'(x))$.
- 12: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $f(x) \cdot (\text{top}'(x) - c(x)) + (1 - f(x)) \cdot (\text{top}'(x) - \text{top}_{\text{last}}(x))$.
- 13: $\mathcal{P}$ and $\mathcal{V}$ run Zerocheck and Rangecheck to check $f \circ (\text{top}[3] - c[3]) + (1 - f) \circ (c[3] - \text{top}[3] - 1) \subset [0, M]$.
- 14: $\mathcal{P}$ and $\mathcal{V}$ run Zerocheck and LogUp to check $(\text{top}_{\text{last}}, \text{top}) \subset (f \circ \text{top}, f \circ \text{top}') \cup \{(\perp_{\text{start}}, \perp_{\text{start}})\}$.
- 15: $\mathcal{V}$ checks that $c(\text{bin}_{\log 2k}(0)) = 0$, $\text{top}(\text{bin}_{\log 2k}(0)) = \perp_{\text{start}}$, $\text{top}'(\text{bin}_{\log 2k}(2k-1)) = \perp_{\text{start}}$, and $\text{top}_{\text{last}}(\text{bin}_{\log 2k}(0)) = \perp_{\text{start}}$.

Throughout this work, the largest values to be checked by Rangecheck arise from three sources: differences in node identifiers, distances, and time steps. Since $\text{id} \in [0, N]$, the maximum distance is $2^{16} \cdot d$, and the number of time steps grows only logarithmically in N , the latter can be ignored. Thus, when integer embedding values lie in $[0, 2^8]$, it suffices to set $M = \max\{N, 2^{16} \cdot d\}$. For example, when $N = 10^6, d = 2^7, M = 2^{23}$. If M is large, we use digital decomposition to do Rangecheck: decompose $\log M$ -bit v into several L -bit chunks and run LogUp on the corresponding L -bit subtables. Here, for a vector $x \in \mathbb{F}^n$, we write $1 - x = (1 - x_0, 1 - x_1, \dots, 1 - x_n)$. The notation extends analogously to matrices. For a matrix $A \in \mathbb{F}^{a \times b}$ with associated polynomial $A(x)$, we define $A(x_1 \| x_2) = A_{\text{bin}_{\log a}^{-1}(x_1)}[\text{bin}_{\log b}^{-1}(x_2)]$, where $x_1 \in \mathbb{F}^{\log a}$ encodes the row index of A and $x_2 \in \mathbb{F}^{\log b}$ encodes the column index.

Complexity. The total online prover time is $O(k + M)$. The verifier time and the proof size are both $O(\log k)$.

Completeness and Soundness. The completeness error is $\frac{O(k+M)}{|\mathbb{F}|}$. The knowledge soundness error is $\frac{O(k+M)}{|\mathbb{F}|}$.

F.3 PIOP for Priority Queue Update

The First part is to prove the validity of being a multiset. Constraints (2)–(5) can be rewrite as: $\forall j \in [k]$,

$$\text{ins}_{p_j}[0] < \text{del}_j[0] \quad (39)$$

$$\text{ins}_{p_j}[1 : 2] = \text{del}_j[1 : 2] \quad (40)$$

$$\mathbf{p} \subset [m] \wedge \mathbf{m} \cdot (1 - \mathbf{m}) = 0 \quad (41)$$

$$\frac{\prod_{e \in \{\text{ins}\}} (\alpha - e[1])}{\prod_{e \in \{\text{del}\}} (\alpha - e[1])} \stackrel{?}{=} \prod_{x \in \text{PQ}_{\text{final}}} (\alpha - x[0]). \quad (42)$$

where the vector $\mathbf{m} (m \neq \mathbf{m})$ denotes the $\mathbf{m}$ in the (1) when using LogUp to prove $\mathbf{p} \subset [m]$.

The second part is to prove the validity of being a priority queue. Firstly, we use the witness $\mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}'$ and check the constraint $\mathcal{R}_{\text{NGN}}(k, \text{del}; \mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}') = 1$ to guarantee the completeness and soundness of (prev, next) as the nearest-greater-neighbor witness.

We introduce a selector $s_i \in \{0, 1\}$ and boundary pointers (ℓ_i, r_i) satisfying $t'_{\ell_i-1} \leq t_i < t'_{\ell_i}$ and $t'_{r_i-1} < t_{di} \leq t'_{r_i}$ (which can be easily proved). The selector is defined as $s_i = 1[\ell_i < r_i]$, i.e., $s_i = 1$ if and only if the lifetime interval (t_i, t_{di}) contains at least one deletion. This definition can also be easily proved.

For those ins_i with $s_i = 1$ (with $\text{maxlb}_i \neq -\infty$), Constraints (7)–(10) can be rewritten as: $\forall i \in [m]$,

$$(L, \text{del}_L) \subset (i, \text{del}) \quad (43)$$

$$\text{ins}_i[2] \geq \text{del}_{L_i}[2] \quad (44)$$

$$\ell_i \leq L_i < r_i \quad (45)$$

$$t_i < \text{del}_{L_i}[0] < t_{di} \quad (46)$$

$$(\mathbf{t}_d, \text{ins}[1 : 2]) \subset \text{del} \cup (T + 1, \text{PQ}_{\text{final}}) \quad (47)$$

$$(1, L, \text{prev}) \subset (\mathbf{f}, \mathbf{c}[0], \text{top}[0]) \quad (48)$$

$$(0, L, \text{next}) \subset (\mathbf{f}, \text{top}[0], \mathbf{c}[0]) \quad (49)$$

$$\text{prev}_{L_i} \leq \text{ins}_i[0] \quad (50)$$

$$\text{next}_{L_i} \geq t_{di} \quad (51)$$

together with $s_i \cdot (\text{maxlb}_i - \text{del}_{L_i}[2]) = 0$.

For those ins_i with $\text{maxlb}_i = -\infty$, the selector takes value $s_i = 0$, which implies $\ell_i = r_i$ and we simply constrain:

$$\text{maxlb}_i = M, \quad L_i = 0. \quad (52)$$

where M is set to $\max(N, 2^{16} \cdot d)$. This captures the case where no deletion occurs within the lifetime of ins_i , ensuring completeness and soundness of the definition of maxlb_i .

We define the relation that captures the semantics of a valid priority queue execution trace. Consider the *Priority-Queue Update relation* $\mathcal{R}_{\text{PQ}}(\ell_q, m, k; \text{PQ}_{\text{final}}, \text{ins}, \text{del}; \mathbf{t}_d, \mathbf{p}, L, \text{prev}, \text{next}, \ell, r, s, \mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}') = 1$ iff the insertion trace $\text{ins} \in \mathbb{F}^{m \times 3}$ and deletion trace $\text{del} \in \mathbb{F}^{k \times 3}$, together with witness values $(\mathbf{t}_d, \mathbf{p}, L, \text{prev}, \text{next}, \ell, r, s, \mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}')$, form a valid execution of a minimum priority queue ending in the final state $\text{PQ}_{\text{final}} \in$

$\mathbb{F}^{(m-k) \times 2}$ (cf. Definition 2). $\ell_q \in \mathbb{F}$, $m, k \in \mathbb{N}$ with $k \leq m$, $\text{PQ}_{\text{final}} \in \mathbb{F}^{(m-k) \times 2}$, $\text{ins} \in \mathbb{F}^{m \times 3}$, $\text{del} \in \mathbb{F}^{k \times 3}$, $\mathbf{t}_d, L, \text{prev}, \text{next}, \ell, r, s \in \mathbb{F}^m$, $\mathbf{p} \in \mathbb{F}^k$, $\mathbf{c}, \text{top}, \text{top}_{\text{last}}, \text{top}' \in \mathbb{F}^{2k \times 4}$, $\mathbf{f} \in \mathbb{F}^{2k}$.

Intuitively, $\mathbf{t}_d$ records deletion times for each insertion, $\mathbf{p}$ realizes the pairing with deletions (as in Theorem 1), $(L, \text{prev}, \text{next})$ together prove the maximum-lower-bound condition (6), and (ℓ, r, s) certify whether the lifetime interval of an insertion contains any deletion or is empty (in which case $\text{maxlb} = -\infty$). Finally, $(\mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}')$ are auxiliary witness for checking the prev and next semantics we need to use.

We present the PIOP for the relation $\mathcal{R}_{\text{PQ}}$:

- 1: Let $\text{PQ}_{\text{final}}(x), \text{ins}(x), \text{del}(x), \mathbf{t}_d(x), \mathbf{p}(x), L(x), \text{prev}(x), \text{next}(x), \ell(x), r(x), s(x), z(x), w(x), \text{maxlb}(x), \mathbf{c}(x), \mathbf{f}(x), \text{top}(x), \text{top}_{\text{last}}(x), \text{top}'(x)$ be the MLEs of their respective vectors. $\mathcal{P}$ computes and sends commitments $\text{com}_{\text{PQ}_{\text{final}}}, \text{com}_{\text{ins}}, \text{com}_{\text{del}}, \text{com}_{\mathbf{t}_d}, \text{com}_{\mathbf{p}}, \text{com}_L, \text{com}_{\text{prev}}, \text{com}_{\text{next}}, \text{com}_{\ell}, \text{com}_r, \text{com}_s, \text{com}_z, \text{com}_w, \text{com}_{\text{maxlb}}, \text{com}_{\mathbf{c}}, \text{com}_{\mathbf{f}}, \text{com}_{\text{top}}, \text{com}_{\text{top}_{\text{last}}}, \text{com}_{\text{top}'}$, and derived tables $\text{del}^{\text{curr}}, \text{del}^{\text{prev}}$.
- 2: Let $(\text{ins}_p)_j = \text{ins}_{p_j}$ and $\text{ins}_p(x)$ its MLE. $\mathcal{P}$ computes and sends $\text{com}_{\text{ins}_p}$.
- 3: $\mathcal{P}$ and $\mathcal{V}$ run a LogUp to check $(\mathbf{p}, \text{ins}_p) \subset (i, \text{ins})$. $\mathcal{V}$ runs a Rangecheck to ensure $\text{del}[0] - \text{ins}_p[0] - 1 \subset [0, M]$. $\mathcal{V}$ runs a Zerocheck to verify $\text{ins}_p(x)[1] + \text{yins}_p(x)[2] - \text{del}(x)[1] - \text{ydel}(x)[2] = 0$. $\mathcal{P}$ and $\mathcal{V}$ run a LogUp to check $\mathbf{p} \subset [m]$ and mask booleanity. $\mathcal{P}$ and $\mathcal{V}$ run three Prodcheck on S, S_1, S_2 . $\mathcal{V}$ checks $S = S_1 \cdot S_2$.
- 4: $\mathcal{P}$ and $\mathcal{V}$ invoke the PIOP for the nearest greater neighbors to prove $\mathcal{R}_{\text{NGN}}(k, \text{del}; \mathbf{c}, \mathbf{f}, \text{top}, \text{top}_{\text{last}}, \text{top}') = 1$.
- 5: For each insertion index domain: $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check $(\ell, \text{del}^{\text{prev}}[\ell], \text{del}^{\text{curr}}[\ell]) \subset (i, \text{del}^{\text{prev}}, \text{del}^{\text{curr}})$. $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check $(r, \text{del}^{\text{prev}}[r], \text{del}^{\text{curr}}[r]) \subset (i, \text{del}^{\text{prev}}, \text{del}^{\text{curr}})$. $\mathcal{P}$ and $\mathcal{V}$ run Rangecheck: $\text{ins}[0] - \text{del}^{\text{prev}}[\ell] \subset [0, M]$, $\text{del}^{\text{curr}}[\ell] - \text{ins}[0] - 1 \subset [0, M]$, $\mathbf{t}_d - \text{del}^{\text{prev}}[r] \subset [0, M]$, $\text{del}^{\text{curr}}[r] - \mathbf{t}_d - 1 \subset [0, M]$.
- 6: Let $\delta(x) = r(x) - \ell(x)$. $\mathcal{P}$ commits $z(x), w(x)$ as auxiliary variables. $\mathcal{P}$ and $\mathcal{V}$ run Zerocheck to enforce $z(x) \cdot \delta(x) = 0$, $(1 - z(x)) \cdot w(x) \cdot \delta(x) = 0$, $z(x)(z(x) - 1) = 0$, $s(x) + z(x) - 1 = 0$, $s(x)(s(x) - 1) = 0$.
- 7: Let $\text{del}_L(x)$ be the MLE of del_{L_i} with sentinel row $\text{del}_0 = (t'_0, v'_0) = (-\infty, M)$. $\mathcal{P}$ and $\mathcal{V}$ run a masked LogUp to check $(s, s \cdot L, s \cdot \text{del}_L) \subset (1, i, \text{del}) \cup \{(0, 0, \text{del}_0)\}$. $\mathcal{P}$ and $\mathcal{V}$ run Rangecheck to check: $s \circ (L - \ell) \subset [0, M]$, $s \circ ((r - 1) - L) \subset [0, M]$.
- 8: $\mathcal{P}$ and $\mathcal{V}$ run Rangecheck to check that $s \circ (\text{del}_L[0] - \text{ins}[0]) \subset [0, M]$, $s \circ (\mathbf{t}_d - \text{del}_L[0]) \subset [0, M]$.
- 9: $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check that $(1, L, \text{prev}) \subset (\mathbf{f}, \mathbf{c}[0], \text{top}[0])$, $(0, L, \text{next}) \subset (\mathbf{f}, \text{top}[0], \mathbf{c}[0])$.
- 10: $\mathcal{P}$ and $\mathcal{V}$ run Rangecheck to check that $s \circ (\text{ins}[0] - \text{prev}) \subset [0, M]$, $s \circ (\text{next} - \mathbf{t}_d) \subset [0, M]$.
- 11: $\mathcal{P}, \mathcal{V}$ run Zerocheck on $s(x) \cdot (\text{maxlb}(x) - \text{del}_L[2](x)) = 0$, $(1 - s(x)) \cdot (\text{maxlb}(x) - M) = 0$. Finally, $\mathcal{P}, \mathcal{V}$ runs a Rangecheck to check $\text{ins}[2] - \text{maxlb} \subset [0, M]$.

Complexity. The total online prover time is $O(m + k + M)$. The verifier time and the proof size are both $O(\log m + \log k)$.

Completeness and Soundness. The completeness error is $\frac{O(m+k+M)}{|\mathbb{F}|}$. The knowledge soundness error is $\frac{O(m+k+M)}{|\mathbb{F}|}$.

F.4 PIOP for Checking Membership Selector

Theorem 4 (Checker for the membership selector). Let $\{(t_i, v_i, n_i)\}_{i=0}^{\ell_q-1}$ be a tuple sequence where $n_i \in \mathbb{Z}$, $v_i \in \{0, 1\}$, and $t_i \in [\ell_q]$. Let $(t^{\text{seq}}, v^{\text{seq}}, n^{\text{seq}})$ denote the lexicographically sorted sequence by ascending n , breaking ties using ascending t . Then v_t satisfies Eq. (11) iff (12)–(15) hold.

PROOF. ($\Rightarrow$) Assume the sequence $\{(n_t, v_t)\}$ satisfies the Eq. (11). Above all, v is a vector of bits (i.e., condition (12) holds). Sort the sequence lexicographically by n_t (ascending), breaking ties using t (also ascending), yielding $(t^{\text{seq}}, v^{\text{seq}}, n^{\text{seq}})$. Then:

- Since the first element in the sorted sequence doesn't have membership in the set $\emptyset$, we must have

$$v^{\text{seq}}_0 = 0.$$

- For any $i > 0$, if $v^{\text{seq}}_i = 0$, the equation (14) clearly holds. If $v^{\text{seq}}_i = 1$, then by definition n^{seq}_i must have appeared in the previous row: $n^{\text{seq}}_i = n^{\text{seq}}_{i-1}$, i.e.,

$$v^{\text{seq}}_i \cdot (n^{\text{seq}}_i - n^{\text{seq}}_{i-1}) = 0.$$

- For any $i > 0$, if $v^{\text{seq}}_i = 0$, then this must be a new key not seen before: $n^{\text{seq}}_i \neq n^{\text{seq}}_{i-1}$, we have that

$$v^{\text{seq}}_i + (1 - v^{\text{seq}}_i) \cdot (n^{\text{seq}}_i - n^{\text{seq}}_{i-1}) \neq 0.$$

where if $v^{\text{seq}}_i = 1$, we have $v^{\text{seq}}_i + (1 - v^{\text{seq}}_i) \cdot (n^{\text{seq}}_i - n^{\text{seq}}_{i-1}) = 1$, which also passes above constraint.

Therefore, conditions (12)–(15) all hold.

($\Leftarrow$) Assume the sorted sequence $(t^{\text{seq}}, v^{\text{seq}}, n^{\text{seq}})$ satisfies (12)–(15).

Then for each key value, we examine the structure in the sorted sequence:

- By (12), we know v is a selector vector of bits.
- By (13), we know the first element is not a member.
- For $i > 1$:
 - If $v^{\text{seq}}_i = 0$, then (14) implies

$$n^{\text{seq}}_i = n^{\text{seq}}_{i-1},$$

which means that it has membership in the set.

- If $v^{\text{seq}}_i = 0$, then (15) implies

$$n^{\text{seq}}_i \neq n^{\text{seq}}_{i-1},$$

which means that it is the first time this key appears.

Hence, for each key, there is exactly one entry (the first one) that is labeled with $v^{\text{seq}}_i = 0$. All other occurrences are labeled with 1. Thus, the selector v exactly satisfies Eq. (11). $\square$

The formal relation definition and the corresponding PIOP instantiation are given as follows. The Membership Selector Relation $\mathcal{R}_{\text{MS}}(\ell_q; n, v, t^{\text{seq}}, n^{\text{seq}}, v^{\text{seq}}) = 1$ iff for all $t \in [\ell_q]$, there is $v_t = 1 \Leftrightarrow \forall s < t, n_s \neq n_t$, where $\ell_q \in \mathbb{F}$, $n, n^{\text{seq}}, t^{\text{seq}} \in \mathbb{F}^{\ell_q}$, $v, v^{\text{seq}} \in \{0, 1\}^{\ell_q}$.

We present the PIOP for the relation $\mathcal{R}_{\text{MS}}$:

- 1: Let $n(x), v(x)$ be the MLEs of n and v . Let $t(x) = x$ for $x \in \mathbb{F}$, denoting the time. $\mathcal{P}$ computes and sends the commitments $\text{com}_n, \text{com}_v, \text{com}_t$.
- 2: Let $t^{\text{seq}}(x), v^{\text{seq}}(x), n^{\text{seq}}(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends the commitments $\text{com}_{t^{\text{seq}}}, \text{com}_{v^{\text{seq}}}, \text{com}_{n^{\text{seq}}}$.
- 3: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $v(1 - v)$.
- 4: $\mathcal{P}$ and $\mathcal{V}$ run a Permcheck on (t, v, n) and $(t^{\text{seq}}, v^{\text{seq}}, n^{\text{seq}})$.
- 5: Let $\text{mask} \in \mathbb{F}^{\ell_q}$ s.t. $\text{mask}_i = 1$ if $i = 0$ and 0 otherwise.
- 6: $\mathcal{P}$ constructs $t^{\text{shift}}, n^{\text{shift}}$ s.t.
 - $t^{\text{shift}}_i = t^{\text{seq}}_{i-1}$ for $i \neq 0$, and $t^{\text{shift}}_0 = t^{\text{seq}}_{\ell_q-1}$;
 - $n^{\text{shift}}_i = n^{\text{seq}}_{i-1}$ for $i \neq 0$, and $n^{\text{shift}}_0 = n^{\text{seq}}_{\ell_q-1}$
- 7: Let $\text{mask}(x), t^{\text{shift}}(x), n^{\text{shift}}(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends the commitments $\text{com}_{\text{mask}}, \text{com}_{t^{\text{shift}}}, \text{com}_{n^{\text{shift}}}$.
- 8: $\mathcal{P}$ and $\mathcal{V}$ run a Permcheck to check $t^{\text{shift}}, n^{\text{shift}}$.
- 9: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck and a Rangecheck to check $(1 - \text{mask}) \circ (n^{\text{seq}} - n^{\text{shift}}) \in [0, M]$.
- 10: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck and a Rangecheck to check $(1 - \text{mask}) \circ v^{\text{seq}} \circ (t^{\text{seq}} - t^{\text{shift}} - 1) \in [0, M]$.
- 11: $\mathcal{P}, \mathcal{V}$ run Zerocheck on $v^{\text{seq}}(x) \cdot (n^{\text{seq}}(x) - n^{\text{shift}}(x))$.
- 12: $\mathcal{P}$ and $\mathcal{V}$ run Zerocheck and a Rangecheck to check $(1 - \text{mask}) \circ (v^{\text{seq}} + (1 - v^{\text{seq}}) \circ (n^{\text{seq}} - n^{\text{shift}} - 1)) \in [0, M]$.
- 13: $\mathcal{V}$ checks that $v^{\text{seq}}(\text{bin}(0)) = 0$.

Steps 6-14 are proving the constraints involving a shifted index $i - 1$, which is nontrivial to express in the multilinear extension. For example, in order to prove $x_i - x_{i-1} \geq 0$ for $i \in [\ell_q] \setminus \{0\}$, $\mathcal{P}$ commits to an auxiliary vector x^{shift} such that $x^{\text{shift}}_i = x_{i-1}$ for $i \neq 0$, and $x^{\text{shift}}_0 = x_{\ell_q-1}$. $\mathcal{P}$ and $\mathcal{V}$ perform a Permcheck to check the claimed shifted relationship. $\mathcal{P}$ uses a LogUp to prove that $(1 - \text{mask}) \circ (x^{\text{shift}} - x) \geq 0$, where the mask vector $\text{mask}_i = 1$ if $i = 0$ and 0 otherwise. In this way, $\mathcal{P}$ proves that $x_i - x_{i-1} \geq 0$ for $i \in [\ell_q] \setminus \{0\}$.

Complexity. The total online prover time is $O(\ell_q + M)$. The verifier time and the proof size are both $O(\log \ell_q)$.

Completeness and Soundness. The completeness error is $\frac{O(\log \ell_q + M)}{|\mathbb{F}|}$. The knowledge soundness error is $\frac{O(\ell_q + M)}{|\mathbb{F}|}$.

G Detailed Construction of PIOPs for Searching Strategies

G.1 PIOP for Greedy Search

We consider the *Greedy Search Relation* $\mathcal{R}_{\text{greedy}}(d, K, h, \ell_q, \text{com}_{t_D}, \text{com}_{t_H}, \text{ep}; m_{\text{unique}}, t_S, c, n, b, f) = 1$ iff the following holds: (1) The best update rule (Equation (19)–(21)); (2) The c -update rule (Equation 17); (3) The n -update rule (Equation 18) and (4) It terminates, where $d, K, h, \ell_q, m_{\text{unique}} \in \mathbb{F}$, $\text{ep} \in \mathbb{F}$, $t_S = (\text{Id}_S, \text{Emb}_S, \text{Dist}_S) \in \mathbb{F}^{m_{\text{unique}} \times (d+2)}$, $\text{Id}_S \in \mathbb{F}^{m_{\text{unique}}}$, $\text{Emb}_S \in \mathbb{F}^{m_{\text{unique}} \times d}$, $\text{Dist}_S \in \mathbb{F}^{m_{\text{unique}}}$, $c \in \mathbb{F}^{\ell_q}$, $n, b, f \in \mathbb{F}^{\ell_q \times 2}$, $P \in \mathbb{F}^{\ell_q/K}$.

We present the PIOP for the relation $\mathcal{R}_{\text{greedy}}$:

- 1: Let $c(x), n(x), b(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_c, \text{com}_n$ and com_b .

- 2: Let $\text{Id}_S(x)$, $\text{Emb}_S(x)$, $\text{Dist}_S(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends com_{Id_S} , $\text{com}_{\text{Emb}_S}$ and $\text{com}_{\text{Dist}_S}$.
- 3: $\mathcal{P}$, $\mathcal{V}$ run a HybridLookup to check $(\text{Id}_S, \text{Emb}_S) \subset \mathbf{t}_D$.
- 4: $\mathcal{V}$ generates and sends a random vector $\mathbf{r}_m \in \mathbb{F}^{\log m_{\text{unique}}}$.
- 5: $\mathcal{P}$ and $\mathcal{V}$ run a Sumcheck to check $\text{Dist}_S(\mathbf{r}_m) = \sum_{x \in \{0,1\}^{\log d}} \text{Emb}_S(\mathbf{r}_m \| x) \cdot \tilde{q}(x)$.
- 6: $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check $\mathbf{n} \subset (\text{Id}_S, \text{Dist}_S) \cup \{\perp\}$ and $\mathbf{b} \subset (\text{Id}_S, \text{Dist}_S) \cup \{\perp\}$.
- 7: $\mathcal{P}$ and $\mathcal{V}$ run a HybridLookup to check that $(h, \mathbf{c}_{\text{bin}(0)} \| i, \mathbf{n}_{\text{bin}(0)} \| i, \mathbf{n}_{\text{bin}(1)} \| i, \dots, \mathbf{n}_{\text{bin}(K-1)} \| i) \subset \mathbf{t}_H$.
- 8: $\mathcal{V}$ checks $c(0) = \text{ep}$ and $b(\text{bin}(0)) = n(\text{bin}(0))$.
- 9: Let $\text{mask}_{\text{loop}} \in \mathbb{F}^{\ell_q}$ s.t. $\text{mask}_{\text{loop}_i} = 1$ if $i = k \cdot K$ and 0 otherwise. Let $\text{mask} \in \mathbb{F}^{\ell_q}$ s.t. $\text{mask}_i = 1$ if $i = 0$ and 0 otherwise.
- 10: $\mathcal{P}$ constructs $\mathbf{c}^{\text{shift}}$, $\mathbf{b}^{\text{shift}}$ s.t.
 - $\mathbf{c}_i^{\text{shift}} = \mathbf{c}_{i-1}$ for $i \neq 0$, and $\mathbf{c}_0^{\text{shift}} = \mathbf{c}_{\ell_q-1}$.
 - $\mathbf{b}_i^{\text{shift}} = \mathbf{b}_{i-1}$ for $i \neq 0$, and $\mathbf{b}_0^{\text{shift}} = \mathbf{b}_{\ell_q-1}$.
- 11: Let $\text{mask}_{\text{loop}}(x)$, $\text{mask}(x)$, $\mathbf{c}^{\text{shift}}(x)$, $\mathbf{b}^{\text{shift}}(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_{\text{mask}_{\text{loop}}}$, com_{mask} , $\text{com}_{\mathbf{c}^{\text{shift}}}$ and $\text{com}_{\mathbf{b}^{\text{shift}}}$.
- 12: $\mathcal{P}$ and $\mathcal{V}$ run two Permcheck to check the correctness of $\mathbf{c}^{\text{shift}}(x)$ and $\mathbf{b}^{\text{shift}}(x)$.
- 13: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $f(x) \cdot (1 - f(x))$ and $(1 - \text{mask}(x)) \cdot ((1 - f(x)) \cdot (b(x \| 1) - \mathbf{b}^{\text{shift}}(x \| 1)) + f(x) \cdot (b(x \| 1) - n(x \| 1)))$.
- 14: $\mathcal{P}$ and $\mathcal{V}$ run a Rangecheck to check $(1 - \text{mask}) \circ (2 \circ f - 1) \circ (\mathbf{b}^{\text{shift}}[1] - \mathbf{n}[1]) \subset [0, M]$.
- 15: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - \text{mask}(x)) \cdot (\text{mask}_{\text{loop}}(x) \cdot (c(x) - b(x \| 0)) + (1 - \text{mask}_{\text{loop}}(x)) \cdot (c(x) - \mathbf{c}^{\text{shift}}(x)))$.
- 16: Let $f(x)$ and $P(x)$ be the MLEs of f and P . $\mathcal{P}$ computes and sends com_f , comp .
- 17: $\mathcal{P}$ and $\mathcal{V}$ run a modified Prodcheck to check that $\forall x \in \{0,1\}^{\log(\ell_q/K)}$, $P(x) = \prod_{y \in \{0,1\}^{\log K}} (1 - f(x \| y))$. This modification treats $\prod_{x \in \{0,1\}^{\log \ell_q}} (1 - f(x))$ as the full multiplication tree, but only checks the first $\log K$ levels of the multiplication tree.
- 18: $\mathcal{P}$ and $\mathcal{V}$ run a Sumcheck to check $\sum_{i=0}^{\ell_q/K-1} P_i = 1$.
- 19: $\mathcal{V}$ checks that $P_{\frac{\ell_q}{K}-1} = 1$.

Complexity. The total online prover time is $O(\ell_q d + M)$. The verifier time and the proof size are both $O(\log \ell_q + \log d)$.

Completeness and Soundness. The completeness error is $\frac{O(\ell_q + d + M)}{|\mathbb{F}|}$. The knowledge soundness error is $\frac{O(\ell_q + d + M)}{|\mathbb{F}|}$.

G.2 PIOP for Best-First Search

We consider the *Best-First Search Relation* $\mathcal{R}_{\text{BF}}(\ell_q, d, K, \text{com}_{\mathbf{t}_D}, \text{com}_{\mathbf{t}_H}, \text{ep}, \min \mathbf{C}_{\text{final}}, \max \mathbf{R}_{\text{final}}; \mathbf{C}_{\text{final}}, \mathbf{R}_{\text{final}}, m_{\text{unique}}, \mathbf{t}_S, \mathbf{c}, \mathbf{n}, \mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}}, \mathbf{v}) = 1$ iff the following holds: (1) The c -update rule (Equations 24); (2) The n -update rule (Equation 25); (3) The $\mathbf{C}_{\text{ins}}$ and $\mathbf{R}_{\text{ins}}$ update rules (Equations 26, 27); (4) The PQ update consistency: $(\mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{C}_{\text{final}})$ and $(\mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}}, \mathbf{R}_{\text{final}})$ must both satisfy the PQ update relation defined in Section 4.2; (5) The v -update rule: the visit bit v satisfies the set update relation defined in Section 4.3, (6) It terminates, where $\ell_q, d, K, \text{ep} \in \mathbb{F}$, $\min \mathbf{C}_{\text{final}}, \max \mathbf{R}_{\text{final}} \in \mathbb{F}^2$, $\mathbf{t}_S = (\text{Id}_S, \text{Emb}_S, \text{Dist}_S) \in \mathbb{F}^{m_{\text{unique}} \times (d+2)}$, $\text{Id}_S \in \mathbb{F}^{m_{\text{unique}}}$, Emb_S

$\in \mathbb{F}^{m_{\text{unique}} \times d}$, $\text{Dist}_S \in \mathbb{F}^{m_{\text{unique}}}$, $\mathbf{c}, \mathbf{v} \in \mathbb{F}^{\ell_q}$, $\mathbf{n} \in \mathbb{F}^{\ell_q \times 2}$, $\mathbf{C}_{\text{final}} \in \mathbb{F}^{(\ell_q - \ell_q/K + 1) \times 2}$, $\mathbf{R}_{\text{final}} \in \mathbb{F}^{\text{efS} \times 2}$, $\mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}} \in \mathbb{F}^{\ell_q \times 3}$.

We present the PIOP for the relation $\mathcal{R}_{\text{BF}}$:

- 1: Let $\mathbf{C}_{\text{final}}(x)$, $\mathbf{R}_{\text{final}}(x)$, $c(x)$, $n(x)$, $\mathbf{C}_{\text{ins}}(x)$, $\mathbf{C}_{\text{del}}(x)$, $\mathbf{R}_{\text{ins}}(x)$, $\mathbf{R}_{\text{del}}(x)$ and $v(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_{\mathbf{C}_{\text{final}}}$, $\text{com}_{\mathbf{R}_{\text{final}}}$, com_c , com_n , $\text{com}_{\mathbf{C}_{\text{ins}}}$, $\text{com}_{\mathbf{C}_{\text{del}}}$, $\text{com}_{\mathbf{R}_{\text{ins}}}$, $\text{com}_{\mathbf{R}_{\text{del}}}$ and com_v .
- 2: Let $\text{Id}_S(x)$, $\text{Emb}_S(x)$, $\text{Dist}_S(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends com_{Id_S} , $\text{com}_{\text{Emb}_S}$ and $\text{com}_{\text{Dist}_S}$.
- 3: $\mathcal{P}$, $\mathcal{V}$ run a HybridLookup to check $(\text{Id}_S, \text{Emb}_S) \subset \mathbf{t}_D$.
- 4: $\mathcal{V}$ generates and sends a random scalar vector $\mathbf{r}_d \in \mathbb{F}^{\log d}$.
- 5: $\mathcal{P}$ and $\mathcal{V}$ run a Sumcheck to check $\text{Dist}_S(\mathbf{r}_m) = \sum_{x \in \{0,1\}^{\log d}} \text{Emb}_S(\mathbf{r}_m \| x) \cdot \tilde{q}(x)$.
- 6: $\mathcal{P}$ and $\mathcal{V}$ run LogUp to check $\mathbf{n}, \mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}} \subset (\text{Id}_S, \text{Dist}_S) \cup \{\perp\}$.
- 7: $\mathcal{P}$ and $\mathcal{V}$ run a HybridLookup to check that $(h, \mathbf{c}_{\text{bin}(0)} \| i, \mathbf{n}_{\text{bin}(0)} \| i, \mathbf{n}_{\text{bin}(1)} \| i, \dots, \mathbf{n}_{\text{bin}(K-1)} \| i) \subset \mathbf{t}_H$.
- 8: $\mathcal{V}$ checks $c(0) = \text{ep}$.
- 9: Let $\text{mask}_{\text{loop}} \in \mathbb{F}^{\ell_q}$ s.t. $\text{mask}_{\text{loop}_i} = 1$ if $i = k \cdot K$ and 0 otherwise. Let $\text{mask} \in \mathbb{F}^{\ell_q}$ s.t. $\text{mask}_i = 1$ if $i = 0$ and 0 otherwise. $\mathcal{P}$ constructs $\mathbf{c}^{\text{shift}}$ s.t. $\mathbf{c}_i^{\text{shift}} = \mathbf{c}_{i-1}$ for $i \neq 0$, and $\mathbf{c}_0^{\text{shift}} = \mathbf{c}_{\ell_q-1}$.
- 10: Let $\text{mask}_{\text{loop}}(x)$, $\text{mask}(x)$, $\mathbf{c}^{\text{shift}}(x)$, $\mathbf{b}^{\text{shift}}(x)$ be the MLEs of their corresponding vectors. $\mathcal{P}$ computes and sends $\text{com}_{\text{mask}_{\text{loop}}}$, com_{mask} and $\text{com}_{\mathbf{c}^{\text{shift}}}$.
- 11: $\mathcal{P}$ and $\mathcal{V}$ run a Permcheck on $c(x)$ and $\mathbf{c}^{\text{shift}}(x)$.
- 12: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $\text{mask}_{\text{loop}}(x) \cdot (c(x) - \mathbf{C}_{\text{del}}(x \| \text{bin}_2(1))) + (1 - \text{mask}(x)) \cdot (1 - \text{mask}_{\text{loop}}(x)) \cdot (c(x) - \mathbf{c}^{\text{shift}}(x))$.
- 13: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - \text{mask}_{\text{loop}}(x)) \cdot (\mathbf{C}_{\text{del}}(x \| \text{bin}_2(1)) - \perp)$.
- 14: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - v(x)) \cdot (\mathbf{C}_{\text{ins}}(x \| \text{bin}_2(1)) - \perp) + v(x) \cdot (\mathbf{C}_{\text{ins}}(x \| \text{bin}_2(1)) - n(x \| 0))$.
- 15: $\mathcal{P}$ and $\mathcal{V}$ run a Zerocheck on $(1 - v(x)) \cdot (\mathbf{R}_{\text{ins}}(x \| \text{bin}_2(1)) - \perp) + v(x) \cdot (\mathbf{R}_{\text{ins}}(x \| \text{bin}_2(1)) - n(x \| 0))$.
- 16: $\mathcal{P}$ extracts from $\mathbf{C}_{\text{del}}$ the rows with indices $k \cdot K$ for $k = 0, 1, \dots, \frac{\ell_q}{K}$, and denotes them as $\mathbf{C}_{\text{del}}^*$. $\mathcal{V}$ can easily open $\mathbf{C}_{\text{del}}^*$ using $\text{com}_{\mathbf{C}_{\text{del}}}$. $\mathcal{P}$ constructs $\mathbf{C}_{\text{ins}}^*$ by inert a ep into $\mathbf{C}$ at time step -1, which can easily open using $\text{com}_{\mathbf{C}_{\text{ins}}}$.
- 17: $\mathcal{P}$ constructs the priority queue R 's augmented insert sequence $\mathbf{R}_{\text{ins}}^*$ by additionally $\text{efS} - 1 \perp$ into R at time step $-\text{efS}, \dots, -2$ and a ep into R at time step -1. $\mathcal{V}$ can easily open $\mathbf{R}_{\text{ins}}^*$ from $\text{com}_{\mathbf{R}_{\text{ins}}}$.
- 18: $\mathcal{P}$ and $\mathcal{V}$ run the PIOP for priority queue update from §F.3, using $\mathbf{C}_{\text{ins}}^*, \mathbf{C}_{\text{del}}^*, \mathbf{C}_{\text{final}}$ as columns 1–2 with their time steps as column 0, to check C .
- 19: $\mathcal{P}$ and $\mathcal{V}$ run the PIOP for priority queue update from §F.3, using $\mathbf{R}_{\text{ins}}^*, \mathbf{R}_{\text{del}}, \mathbf{R}_{\text{final}}$ as columns 1–2 with their time steps as column 0, to check R .
- 20: $\mathcal{P}$ and $\mathcal{V}$ run the PIOP for checking membership selector from §F.4 to check the correctness of v .
- 21: $\mathcal{V}$ generates and sends a random scalar vector $\mathbf{r}$. $\mathcal{P}$ and $\mathcal{V}$ opens $\mathbf{C}_{\text{final}}(\mathbf{r})$. $\mathcal{P}$ runs LogUp with $\mathcal{V}$ to check that $\min \mathbf{C}_{\text{final}} \leq \min(\mathbf{C}_{\text{final}})$, and $\max \mathbf{R}_{\text{final}} \geq \max(\mathbf{R}_{\text{final}})$.
- 22: $\mathcal{V}$ checks $\mathbf{C}_{\text{final}}(\mathbf{r}) = \perp \vee \min \mathbf{C}_{\text{final}} > \max \mathbf{R}_{\text{final}}$.

Complexity. The total online prover time is $O(\ell_q d + M)$. The verifier time and the proof size are both $O(\log \ell_q + \log d)$.

Completeness and Soundness. The completeness error is $\frac{O(\ell_q d + M)}{|\mathbb{F}|}$. The knowledge soundness error is $\frac{O(\ell_q d + M)}{|\mathbb{F}|}$.

H PIOP for HNSW Query

We consider the *HNSW Query Relation* $\mathcal{R}_{\text{HNSW}}(\ell_q, d, K, t_{\mathcal{D}}, t_{\mathcal{H}}, \text{ep}_{\text{default}}, \text{topk}, \text{minC}_{\text{final}}, \text{maxR}_{\text{final}}; \text{ep}, \text{C}_{\text{final}}, \text{R}_{\text{final}}, m_{\text{unique}}, t_{\mathcal{S}} = (\text{Id}_{\mathcal{S}}, \text{Emb}_{\mathcal{S}}, \text{Dist}_{\mathcal{S}}) \in \mathbb{F}^{m_{\text{unique}} \times (d+2)}, \mathbf{c} \in \mathbb{F}^{\ell_q}, \text{Id}_{\mathcal{S}} \in \mathbb{F}^{m_{\text{unique}}}, \text{Emb}_{\mathcal{S}} \in \mathbb{F}^{m_{\text{unique}} \times d}, \text{Dist}_{\mathcal{S}} \in \mathbb{F}^{m_{\text{unique}}}, \mathbf{c}, \mathbf{n}, \mathbf{b}, \mathbf{f}, \mathbf{c}_{\text{BF}}, \mathbf{n}_{\text{BF}}, \mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}}, \mathbf{v}) = 1$ iff the topk is the result of Alg. 1 given $G = (t_{\mathcal{D}}, t_{\mathcal{H}})$ and $\mathbf{q}$ as the input, where $\ell_q, \text{ep} \in \mathbb{F}^H, H, d, K, \text{ep}_{\text{default}}, \text{minC}_{\text{final}}, \text{maxR}_{\text{final}} \in \mathbb{F}^2, t_{\mathcal{S}} \in \mathbb{F}^{m \times (d+2)}, \mathbf{c}, \mathbf{f} \in \mathbb{F}^{\sum_{h=1}^H \ell_q^{(h)}}, \mathbf{P} \in \mathbb{F}^{\sum_{h=1}^H (\ell_q^{(h)}/K)}, \mathbf{n}, \mathbf{b} \in \mathbb{F}^{\sum_{h=1}^H \ell_q^{(h)} \times 2}, \mathbf{c}_{\text{BF}} \in \mathbb{F}^{\ell_q^{(0)}}, \mathbf{n}_{\text{BF}} \in \mathbb{F}^{\ell_q^{(0)} \times 2}, \mathbf{v} \in \mathbb{F}^{\ell_q^{(0)}}, \mathbf{C}_{\text{final}} \in \mathbb{F}^{\ell_q^{(0)} (1 - \frac{1}{2K}) \times 2}, \mathbf{R}_{\text{final}} \in \mathbb{F}^{\text{efs} \times 2}, \mathbf{C}_{\text{ins}}, \mathbf{C}_{\text{del}}, \mathbf{R}_{\text{ins}}, \mathbf{R}_{\text{del}} \in \mathbb{F}^{\ell_q^{(0)} \times 3}$.

PIOP for HNSW query. We present the PIOP for $\mathcal{R}_{\text{HNSW}}$:

- 1: $\mathcal{V}$ checks $\text{ep}_H = \text{ep}_{\text{default}}$.
- 2: $\mathcal{P}$ constructs the witness for each level h as $\mathbf{c}^{(h)} = \mathbf{c}_{[\sum_{i=h+1}^H \ell_q^{(i)}, \sum_{i=h}^H \ell_q^{(i)} - 1]}$
 $\mathbf{n}^{(h)} = \mathbf{n}_{[\sum_{i=h+1}^H \ell_q^{(i)}, \sum_{i=h}^H \ell_q^{(i)} - 1]}, \mathbf{b}^{(h)} = \mathbf{b}_{[\sum_{i=h+1}^H \ell_q^{(i)}, \sum_{i=h}^H \ell_q^{(i)} - 1]}, \mathbf{f}^{(h)} = \mathbf{f}_{[\sum_{i=h+1}^H \ell_q^{(i)}, \sum_{i=h}^H \ell_q^{(i)} - 1]}, \mathbf{P}^{(h)} = \mathbf{P}_{[\sum_{i=h+1}^H (\ell_q^{(i)}/K), \sum_{i=h}^H (\ell_q^{(i)}/K) - 1]}$.
- 3: In each level $h = 1$ to H :
 - Let $\mathbf{c}^{(h)}(x), \mathbf{n}^{(h)}(x), \mathbf{b}^{(h)}(x), \mathbf{f}^{(h)}(x), \mathbf{P}^{(h)}(x)$. $\mathcal{P}$ computes $\text{com}_{\mathbf{c}^{(h)}}, \text{com}_{\mathbf{n}^{(h)}}, \text{com}_{\mathbf{b}^{(h)}}, \text{com}_{\mathbf{f}^{(h)}}, \text{com}_{\mathbf{P}^{(h)}}$.
 - $\mathcal{P}$ and $\mathcal{V}$ run a GreedySearchCheck to check the greedy search on level h .
- 4: $\mathcal{P}, \mathcal{V}$ run Zerocheck to check $\text{ep}_h = \mathbf{b}^{(h+1)}(\text{bin}_{\log \ell_q^{(h+1)}}(\ell_q^{(h+1)} - 1)), h \in [H]$.
- 5: In base level $h=0$, $\mathcal{P}$ and $\mathcal{V}$ run a BestFirstCheck to check the best-first search.
- 6: $\mathcal{P}, \mathcal{V}$ run batched lookup and LogUp to check that topk is exactly the top- k nodes with the k smallest distances in $\mathbf{R}_{\text{final}}$.

Theorem 5. zkRAG has completeness error $\frac{O(\sum_{h=0}^H \ell_q^{(h)} + d + M)}{|\mathbb{F}|}$ and soundness error $\frac{O(\sum_{h=0}^H \ell_q^{(h)} + d + M)}{|\mathbb{F}|}$. When instantiated with the KZG [38] and PST [50] PCS, zkRAG achieves prover time $O(\sum_{h=0}^H \ell_q^{(h)} + m_{\text{unique}} \cdot d)$ (where $m_{\text{unique}} < \sum_{h=0}^H \ell_q^{(h)}$), and succinct verifier time and proof size.

PROOF. All PIOPs we construct (and all standard gadgets we invoke) have prover time linear in ℓ_q , or M , except HybridLookup for $t_{\mathcal{S}} \subset t_{\mathcal{D}}$ and a sumcheck for proving the distances correctness of table $t_{\mathcal{S}}$. The HybridLookup in §4.1 has prover time linear in the query vector size; after our random linear combination, the effective query vector length becomes $m_{\text{unique}} = |t_{\mathcal{S}}|/d$. The Sumcheck for proving the distances correctness of table $t_{\mathcal{S}}$ has prover time $O(m_{\text{unique}} \cdot d)$. Thus, the total prover complexity is $O(\sum_{h=0}^H \ell_q^{(h)} + m_{\text{unique}} \cdot d)$. With succinct PCS (e.g., KZG [38] and PST [50]), the verifier time and proof size are succinct.

For errors, each invoked gadget contributes completeness and soundness error at most linear in $1/|\mathbb{F}|$ with respect to its size (either $\sum_{h=0}^H \ell_q^{(h)}$, or d , or M), and the random linear combination over the

d coordinates incurs an additional $\frac{O(d)}{|\mathbb{F}|}$ soundness loss. By a union bound over all checks, both the completeness error and soundness error are bounded by $\frac{O(\sum_{h=0}^H \ell_q^{(h)} + d + M)}{|\mathbb{F}|}$. $\square$

I Adding Zero-Knowledge to zkRAG

I.1 Zero-Knowledge PIOP

We follow [13, 15] and define the (honest verifier) zero-knowledge property of PIOPs.

Definition 6 (Perfect zero-knowledge for PIOPs). A PIOP $\langle \mathcal{P}, \mathcal{V} \rangle$ has *perfect zero-knowledge* with query bound t and query checker C if there is a PPT simulator $\mathcal{S}$ such that for every field $\mathbb{F}$, index i , instance x , witness w , and every (t, C) -admissible verifier $\mathcal{V}^*$, the following transcripts are identically distributed:

$$\text{View}(\mathcal{P}(\mathbb{F}, i; x; w), \mathcal{V}^*) \approx \mathcal{S}^{\mathcal{V}^*}(\mathbb{F}, i; x).$$

Here the view consists of $\mathcal{V}^*$'s randomness, the non-oracle messages sent by $\mathcal{P}$, and the list of answers to $\mathcal{V}^*$'s oracle queries. A verifier is (t, C) -admissible if it makes no more than t queries, and each query is accepted by the checker C . We say that $\langle \mathcal{P}, \mathcal{V} \rangle$ is honest-verifier-zero-knowledge (HVZK) if there is a simulator for $\mathcal{V}$.

Lemma 1 (Theorem 4 in [13]). If the PCS Γ has witness-extended emulation, and if the t -round PIOP for $\mathcal{R}$ has negligible knowledge error, then Π , the output of the PIOP compilation, is a secure (non-oracle) argument of knowledge for $\mathcal{R}$. The compilation also preserves HVZK if Γ is hiding and Eval is HVZK.

I.2 Polynomial Masking and Zero-Knowledge SumCheck

Polynomial masking. We follow the definition of polynomial masking in [15].

Definition 7 (Polynomial masking). A randomized algorithm msk is a (t, C, μ) -masking if the following conditions hold.

- (1) For every $d \in \mathbb{N}$ and every polynomial $f \in \mathbb{F}_{\mu}^{(\leq d)}$, the masked polynomial $f^* \xleftarrow{\$} \text{msk}(f, t, C)$ does not change evaluations over the Boolean hypercube B_{μ} .
- (2) For every $d \in \mathbb{N}$, every polynomial $f \in \mathbb{F}_{\mu}^{(\leq d)}$, and every list of queries $\mathbf{q} := (q_1, \dots, q_t)$ that is accepted by the checker C , let $f^* \xleftarrow{\$} \text{msk}(f, t, C)$. It holds that $(f^*(q_1), \dots, f^*(q_t))$ is uniformly distributed over $\mathbb{F}^t$.

[15] gives a construction way of a (t, C_{ℓ}, μ) -masking algorithm $\text{msk}(f, t, \ell)$ for every $\mu, t \in \mathbb{N}$ and $\ell \in [\mu]$, where the checker C_{ℓ} accepts a list of queries $(q_1, \dots, q_t)$ if and only if $b_{i,\ell} \notin \{0, 1, b_{1,\ell}, \dots, b_{i-1,\ell}\}$ for every query $q_i := (b_{i,1}, \dots, b_{i,\mu}) \in \mathbb{F}^{\mu}$ ($1 \leq i \leq t$). The masking algorithm msk is: Given a polynomial $f \in \mathbb{F}_{\mu}^{(\leq d)}$, query bound t , and checker C_{ℓ} , the algorithm proceeds as follows:

- Sample a univariate polynomial $R(X) := c_0 + c_1 X + \dots + c_{t-1} X^{t-1}$, where $c_0, \dots, c_{t-1} \xleftarrow{\$} \mathbb{F}$.
 - Output $f^* := f + Z(X_{\ell}) \cdot R(X_{\ell})$, where $Z(X_{\ell}) := X_{\ell} \cdot (1 - X_{\ell})$.
- It is clear that f^* has degree $\max(d, t + 1)$.

Zero-knowledge sumcheck. We use the adapted construction introduced in [15].

I.3 Zero-knowledge LogUp and HybridLookUp

Recall that both [23, 34] construct lookup arguments by checking the following equation:

$$\sum_{i=0}^{N-1} \frac{m_i}{r+t_i} = \sum_{i=0}^{\ell_q-1} \frac{1}{r+q_i} \quad (53)$$

where m_i represents the number of times each table element t_i appears in the query $q \in \mathbb{F}^{\ell_q}$, and r is a random challenge issued by $\mathcal{V}$. Note that LogUp is indeed a Sumcheck, but when the query table size and the original table size are different. We often split it into two Sumcheck $S_1 = \sum_{i=0}^{N-1} \frac{m_i}{r+t_i}$ and $S_2 = \sum_{i=0}^{\ell_q-1} \frac{1}{r+q_i}$ and then check whether $S_1 = S_2$.

However, directly showing the value of S_1, S_2 to $\mathcal{V}$ may leak information of the underlying polynomials themselves because S_1 and S_2 are indeed the linear combination of query table f or the table t . We must mask the two sums S_1, S_2 to achieve a zero-knowledge LogUp.

Zero-knowledge LogUp. The LogUp check exposes two scalar sums S_1, S_2 . To hide them, we add a one-time-pad-style mask built from a short random vector. Specifically, sample a vector $r_{zk} \in \mathbb{F}^{\ell_q}$ such that it has three uniformly random entries at positions 0, 1, 2 and zeros elsewhere. Define $r_{zkN} \in \mathbb{F}^N$ by setting $r_{zkNi} = r_{zki}$ for $i \in \{0, 1, 2\}$ and $r_{zkNi} = 0$ for $i = 3, 4, \dots, N-1$. Let $R := \sum_{i=0}^{\ell_q-1} r_{zki} = \sum_{i=0}^{N-1} r_{zkNi}$ be the induced mask value. Our ZKLogUp PIOP construction is:

- 1: $\mathcal{P}$ applies standard polynomial masking to *all* protocol oracles needed for the LogUp check, commits to the masked oracles, and sends the commitments to $\mathcal{V}$.
- 2: $\mathcal{P}$ samples r_{zk} as above, computes $\text{com}_{r_{zk}}$, and sends it to $\mathcal{V}$.
- 3: $\mathcal{P}$ computes the masked sums $S_1^* = \sum_{i=0}^{N-1} \left(\frac{m_i}{r+t_i} + r_{zkNi} \right)$, $S_2^* = \sum_{i=0}^{\ell_q-1} \left(\frac{1}{r+q_i} + r_{zki} \right)$, and sends S_1^*, S_2^* to $\mathcal{V}$.
- 4: $\mathcal{V}$ checks the scalar equality $S_1^* = S_2^*$.
- 5: $\mathcal{P}$ and $\mathcal{V}$ run two zero-knowledge Sumcheck protocols to certify that S_1^* and S_2^* are computed as the above sums over the (masked) committed oracles.
- 6: $\mathcal{V}$ opens all committed oracles as required by the sum-checks, except r_{zkN} . For r_{zkN} inside S_1^* , we use a virtual-oracle encoding, e.g.,

$$\tilde{r}_{zkN}(x||y) = \tilde{r}_{zk}(x) \cdot \tilde{e}q(0)(y),$$

so that opening $\tilde{r}_{zkN}$ reduces to opening $\tilde{r}_{zk}$.

ZKLogUp is HVZK. Because the two sum-checks are zero-knowledge, it suffices to argue that (i) revealing the scalars S_1^*, S_2^* and (ii) the final oracle openings leak no information about the unmasked witnesses beyond what is implied by correctness. All protocol oracles (except r_{zk}) are protected by polynomial masking. Provided the query bound t is respected and every query is (t, C) -admissible, the openings of these masked oracles are simulatable and hence reveal nothing about the underlying witnesses. In our protocol, the verifier triggers at most two admissible openings related to $\text{com}_{r_{zk}}$ (one via r_{zk} and one via the virtual oracle for r_{zkN}). Each opening yields at most one linear constraint on the three unknown random entries

$u := (r_0, r_1, r_2) \in \mathbb{F}^3$. Each opening at a point z induces at most one linear constraint of the form $\langle a(z), u \rangle = b(z)$ for some coefficient vector $a(z) \in \mathbb{F}^3$ determined by z . Thus, conditioned on the verifier's entire view, there remains at least one degree of freedom in u . Moreover, we require the admissibility checker C to reject any pair of opening points (z_1, z_2) such that $(1, 1, 1) \in \text{span}\{a(z_1), a(z_2)\}$. Under this restriction enforced by C , the revealed constraints do not determine R . Consequently, R is (statistically) uniform from the verifier's perspective, and the revealed masked sums satisfy $S_1^* = S_1 + R$, $S_2^* = S_2 + R$, so S_1^*, S_2^* leak no information about S_1, S_2 . In summary, by composing the simulators for the ZK sum-checks, the ZK evaluation/opening of the PCS, and the above argument for the masking term R , the overall ZKLogUp PIOP is HVZK.

Theorem 6. ZKLogUp is HVZK. Moreover, it preserves negligible completeness and negligible soundness.

PROOF. To prove HVZK, we describe the simulator as follows.

Simulator $\mathcal{S}^{\mathcal{V}}(x)$. The simulator interacts with the honest verifier $\mathcal{V}$ and produces a transcript in the PIOP model (i.e., verifier randomness, non-oracle messages, and answers to oracle queries). Let $\mathcal{S}_{SC}^{(1)}$ and $\mathcal{S}_{SC}^{(2)}$ denote the simulators for the two ZK sum-checks.

- (1) Sample $S^* \xleftarrow{\$} \mathbb{F}$ uniformly at random and set $S_1^* \leftarrow S^*$ and $S_2^* \leftarrow S^*$. Send (S_1^*, S_2^*) to $\mathcal{V}$.
- (2) Output the two ZK sum-check transcripts by invoking the sum-check simulators: $\tau_1 \leftarrow \mathcal{S}_{SC}^{(1)\mathcal{V}}(x, S_1^*)$ and $\tau_2 \leftarrow \mathcal{S}_{SC}^{(2)\mathcal{V}}(x, S_2^*)$.
- (3) Answer oracle queries of $\mathcal{V}$ as follows (maintaining consistency on repeated queries):
 - For any query to a polynomial-masked oracle $\mathcal{O}$ at a point z , return a uniformly random value in $\mathbb{F}$.
 - For any query to the virtual oracle r_{zkN} at a point $(x||y)$, return $\tilde{r}_{zk}(x) \cdot \tilde{e}q(0)(y)$, where $\tilde{r}_{zk}(x)$ is answered as below.
 - For any query to r_{zk} at a point z , interpret it as a linear form $\langle a(z), u \rangle$ over $u = (r_0, r_1, r_2) \in \mathbb{F}^3$. Sample $u \xleftarrow{\$} \mathbb{F}^3$ once at the beginning and answer consistently with $v \leftarrow \langle a(z), u \rangle$, aborting only if the query violates (t, C) -admissibility.

Hybrids. Let H_0 be the real interaction $\text{View}(\langle \mathcal{P}, \mathcal{V} \rangle(x))$. We define the following hybrids.

- **Hybrid H_1 .** Identical to H_0 except that the two sum-check sub-protocols are replaced with the outputs of $\mathcal{S}_{SC}^{(1)}$ and $\mathcal{S}_{SC}^{(2)}$. By the zero-knowledge property of the sum-checks, $H_1 \approx H_0$.
- **Hybrid H_2 .** Identical to H_1 except that all answers to queries to polynomial-masked oracles are replaced by uniformly random field elements (consistent across repeated queries). Since every verifier is (t, C) -admissible and the masking degree is chosen accordingly, oracle answers in the real execution are uniformly random subject to consistency; hence $H_2 \approx H_1$.
- **Hybrid H_3 .** Identical to H_2 except that the masked sums (S_1^*, S_2^*) are replaced by a uniformly random pair (S^*, S^*) with $S^* \xleftarrow{\$} \mathbb{F}$. In the real execution, $S_1^* = S_1 + R$ and $S_2^* = S_2 + R$ for $R = \sum_{i=0}^{\ell_q-1} r_i$. Under (t, C) -admissibility, the verifier observes at most two openings related to r_{zk} , yielding at most two linear constraints on $u = (r_0, r_1, r_2)$. Moreover, C rejects any pair of opening points (z_1, z_2) such that these constraints determine $R = \langle (1, 1, 1), u \rangle$.

Therefore R remains (statistically) uniform conditioned on the verifier's view, and (S_1^*, S_2^*) is distributed as (S^*, S^*) with $S^* \xleftarrow{\$} \mathbb{F}$, implying $H_3 \approx H_2$.

- **Hybrid H_4 .** Identical to H_3 except that the oracle r_{zk} (and the induced virtual oracle r_{zkN}) is answered using the simulator's pre-sampled $u \xleftarrow{\$} \mathbb{F}^3$ (as in $\mathcal{S}$), rather than the real prover's choice. Since u is uniform in the real protocol as well, the verifier's view does not change at all, so $H_4 \equiv H_3$.

Finally, observe that H_4 is exactly the output distribution of $\mathcal{S}^V(x)$. Chaining the above, we conclude $\mathcal{S}^V(x) \approx \text{View}(\langle \mathcal{P}, \mathcal{V} \rangle(x))$, and hence ZKLogUp is HVZK.

Completeness. Completeness is unchanged since all ZK modifications preserve the same identities on the Boolean hypercube.

Soundness. Soundness remains negligible because the ZK modifications only incur a constant-factor degree increase and a constant-factor reduction of the challenge space, and ZK Sumcheck preserves soundness. $\square$

Zero-knowledge HybridLookup. According to §4.1, HybridLookup is based on the LogUp identity, but verifies the table side (vectors t, m) via a *univariate* quotient-based sum-check, while verifying the query side (vector q) via a standard *multivariate* (MLE) sum-check. We follow the same masking strategy as in ZKLogUp, but use four random scalars (instead of three), since r_{zk} is evaluated three times in total in HybridLookup. Concretely, we sample a sparse vector $r_{zk} \in \mathbb{F}^{\ell_q}$ with four uniformly random entries (and zeros elsewhere), and define $r_{zkN} \in \mathbb{F}^N$ by embedding those four entries into the first four positions and setting all remaining coordinates to 0. We commit to r_{zk} in *both* encodings: a univariate commitment $\text{com}_{ur_{zk}}$ for the univariate quotient-based check, and a multivariate commitment $\text{com}_{mr_{zk}}$ for the MLE sum-check on the query side.

On the query side, we run a zero-knowledge MLE sum-check for the masked claim $S_2^* = \sum_{i=0}^{\ell_q-1} \left(\frac{1}{r+q_i} + r_{zki} \right)$. On the table side, the same MLE-based check is not directly applicable because the protocol uses a univariate view. Instead, we follow the standard CQ-style fully ZK pipeline of [14]. In our setting, the CQ-style lookup is invoked in a LogUp-style manner by treating $B(X)$ as a constant polynomial S_1^*/N . Finally, we must ensure that $\text{com}_{ur_{zk}}$ and $\text{com}_{mr_{zk}}$ indeed commit to the *same* underlying vector r_{zk} . This is enforced by a standard consistency check between the two encodings. Our ZKHybridLookup construction is:

- 1: Perform the zero-knowledge preprocessing and commitment as in [14]. In addition, $\mathcal{P}$ applies polynomial masking to all other protocol oracles, commits to the masked oracles, and sends the commitments to $\mathcal{V}$.
- 2: $\mathcal{P}$ samples the $r_{zk} \in \mathbb{F}^{\ell_q}$ as claimed. $\mathcal{P}$ compute the univariate commitment $\text{com}_{ur_{zk}}$ and multivariate commitment $\text{com}_{mr_{zk}}$, and sends both to $\mathcal{V}$.
- 3: $\mathcal{P}$ computes $S_1^* = \sum_{i=0}^{N-1} \left(\frac{m_i}{r+q_i} + r_{zki} \right)$ and $S_2^* = \sum_{i=0}^{\ell_q-1} \left(\frac{1}{r+q_i} + r_{zki} \right)$, and sends S_1^*, S_2^* to $\mathcal{V}$.
- 4: $\mathcal{V}$ checks $S_1^* = S_2^*$.
- 5: $\mathcal{P}$ and $\mathcal{V}$ follow the zkcq+ protocol of [14] to verify that S_1^* is correctly summed. Concretely, we treat $B(X)$ as the constant

polynomial $B(X) \equiv S_1^*/N$, and we modify Eq. (9) in [14] to $(A(X) + r(X) - \vartheta^{-1}B(X)z(X) + \eta^2 S(X)) - (R_C(X) + \eta^2 R_S)X = Q_C(X) \nu_K(X)$. We then batch Eq. (7) and this modified Eq. (9) exactly as in [14], obtaining the corresponding new Eq. (11).

- 6: $\mathcal{P}$ and $\mathcal{V}$ run one MLE zero-knowledge Sumcheck in §I.2 to check that S_2^* is correctly summed.
- 7: Besides opening all the oracles, $\mathcal{V}$ also samples $x \in \mathbb{F}$ and checks $ur_{zk}(x) = mr_{zk}(x || x^2 || x^4 || \dots || x^{2^{\log \ell_q - 1}})$.

ZKHybridLookup is HVZK. All invoked sub-protocols are zero-knowledge:

the applied zkcq+ from [14], the zero-knowledge Sumcheck on the query side, and polynomial masking. Thus, it remains to argue that revealing S_1^*, S_2^* does not leak the unmasked sums. By construction, $S_1^* = S_1 + R$ and $S_2^* = S_2 + R$, where $R = \sum_i r_{zki}$. Across the whole HybridLookup sub-protocol, the verifier obtains at most three admissible evaluations related to r_{zk} (one from the univariate check, one from the multivariate check, and one from the encoding-consistency check). Since r_{zk} contains four independent random scalars, these three evaluations impose at most three linear constraints, leaving at least one degree of freedom. Moreover, the admissibility checker C rejects any set of evaluation points whose induced constraints would determine R . Hence R remains (statistically) uniform conditioned on the verifier's view, and therefore (S_1^*, S_2^*) leaks no information about (S_1, S_2) . In summary, the ZKHybridLookup protocol is HVZK.

Theorem 7. ZKHybridLookup is HVZK. Moreover, it preserves negligible completeness and negligible soundness.

PROOF. We treat the following simulators as black boxes: (i) $\mathcal{S}_{zkcq+}$ for the (modified) zkcq+ univariate component, (ii) $\mathcal{S}_{SC}$ for the ZK MLE sum-check on the query side.

Simulator $\mathcal{S}_{ZKHybridLookup}^V(x)$. The simulator outputs the verifier's view (randomness, non-oracle messages, and oracle answers).

- (1) **Sample the masked scalar claims.** Sample $S^* \xleftarrow{\$} \mathbb{F}$ uniformly and set $S_1^* \leftarrow S^*$ and $S_2^* \leftarrow S^*$. Send (S_1^*, S_2^*) to $\mathcal{V}$.
- (2) **Simulate the univariate table-side protocol.** Invoke the zkcq+ simulator on the same public inputs, but with (i) the modified Eq. (9) (i.e., $A(X)$ replaced by $A(X) + r(X)$) and (ii) $B(X) \equiv S_1^*/N$ as a constant polynomial:

$$\tau_{uni} \leftarrow \mathcal{S}_{zkcq+}^V(x, S_1^*).$$

Output τ_{uni} following the message order of the real protocol.

- (3) **Simulate the multivariate query-side sum-check.** Invoke the ZK sum-check simulator for the masked claim S_2^* :

$$\tau_{mle} \leftarrow \mathcal{S}_{SC}^V(x, S_2^*).$$

Output τ_{mle} following the message order of the real protocol.

- (4) **Answer oracle queries.** Maintain a lookup table to ensure consistency for repeated queries.

- For every query to a polynomial-masked oracle $\mathcal{O}$ at point z , return a fresh uniform $v \xleftarrow{\$} \mathbb{F}$ (unless $(\mathcal{O}, z)$ was queried before).
- For queries involving the mask oracle r_{zk} , sample once $u = (r_0, r_1, r_2, r_3) \xleftarrow{\$} \mathbb{F}^4$ and answer each query at point z as a linear form $v = \langle a(z), u \rangle$, recording the induced constraint. If the

accumulated constraints would determine $R = \langle (1, 1, 1, 1), \mathbf{u} \rangle$, abort (this matches the (t, C) -admissibility restriction).

- For the virtual oracle $\mathbf{r}_{\mathbf{zkN}}$, answer consistently with its definition (reducing to an answer for $\mathbf{r}_{\mathbf{zk}}$ on the designated slice, and 0 off-slice).

(5) **Simulate the encoding-consistency check.** When $\mathcal{V}$ samples $x \xleftarrow{\$} \mathbb{F}$ for the consistency test, answer the two evaluations $\mathbf{ur}_{\mathbf{zk}}(x)$ and $\mathbf{mr}_{\mathbf{zk}}(\phi(x))$ consistently from the same $\mathbf{u}$ so that the equality holds.

Hybrids. Let H_0 be the real view $\text{View}(\langle \mathcal{P}, \mathcal{V} \rangle(x))$.

- **Hybrid H_1 .** Replace the univariate zkCQ+ sub-protocol and the MLE ZK sum-check with $\mathcal{S}_{\text{zkCQ+}}$ and $\mathcal{S}_{\text{SC}}$, respectively. Then $H_1 \approx H_0$ by the ZK property of these sub-protocols.
- **Hybrid H_2 .** In H_1 , replace all answers to queries on polynomial-masked oracles by uniform random field elements (consistent on repeats). Then $H_2 \approx H_1$ by the standard masking lemma for (t, C) -admissible queries.
- **Hybrid H_3 .** In H_2 , replace (S_1^*, S_2^*) with (S^*, S^*) for uniform $S^* \xleftarrow{\$} \mathbb{F}$. In the real protocol, $S_1^* = S_1 + R$ and $S_2^* = S_2 + R$ with $R = \sum_{i=0}^3 r_i$. The verifier obtains at most three admissible evaluations related to $\mathbf{r}_{\mathbf{zk}}$, yielding at most three linear constraints on $\mathbf{u} \in \mathbb{F}^4$; and C rejects any set of evaluation points that would determine R . Hence R remains uniform conditioned on the view, implying $H_3 \approx H_2$.
- **Hybrid H_4 .** In H_3 , answer all $\mathbf{r}_{\mathbf{zk}}$ -related queries using the simulator's sampled $\mathbf{u} \xleftarrow{\$} \mathbb{F}^4$ (and the induced virtual oracle definition for $\mathbf{r}_{\mathbf{zkN}}$). Since $\mathbf{u}$ is uniform in the real protocol as well, $H_4 \equiv H_3$. Finally, H_4 is exactly the output distribution of $\mathcal{S}_{\text{ZKHybrdLookup}}^{\mathcal{V}}(x)$.

Therefore, $\mathcal{S}_{\text{ZKHybrdLookup}}^{\mathcal{V}}(x) \approx \text{View}(\langle \mathcal{P}, \mathcal{V} \rangle(x))$, and ZKHybrdLookup is HVZK.

Completeness. Completeness is unchanged since all ZK modifications preserve the same identities on the Boolean hypercube.

Soundness. Soundness remains negligible because the ZK modifications only incur a constant-factor degree increase and a constant-factor reduction of the challenge space, and ZK Sumcheck and zkCQ+ preserve soundness. $\square$

I.4 Zero-Knowledge Compilation of zkRAG

Zero-knowledge gadgets. Note that we use the same gadgets in [15], and according to the Appendix A of [15], we use the ZK compiler described in [15] to make the gadgets Zerocheck, Permcheck and Prodcheck zero-knowledge.

Putting it together. Our HNSW query proof system only uses five PIOP gadgets: Zerocheck, Permcheck, Prodcheck, LogUp, and HybridLookup. Importantly, the public inputs to these gadgets contain no concrete secret values (they only specify dimensions, indices, and public challenges). Our ZK compilation therefore proceeds as follows: (i) transform the non-ZK preprocessing into its ZK variant following [14]; (ii) apply polynomial masking msk to all protocol oracles; and (iii) replace each gadget by its ZK version. Since all oracles are masked and the verifier's queries are restricted to be (t, C) -admissible, the resulting PIOP for the HNSW query in $\mathcal{SH}$ is

HVZK. To obtain an end-to-end HVZK argument, we additionally require a hiding PCS with a zero-knowledge evaluation/opening protocol. By Theorem 10 of [10], any additive and m -spanning PCS can be compiled into a hiding PCS with a ZK Eval protocol. Our zkRAG instantiates the PCS with KZG and PST, both of which are additive and m -spanning; hence we can equip them with hiding commitments and ZK evaluation/opening in the same spirit as [15]. Finally, by Lemma 1, zkRAG instantiated with KZG and PST can be compiled to be HVZK.

Theorem 8. With the zero-knowledge compilation, zkRAG is a zero-knowledge RAG retrieval scheme by Definition 5.

PROOF. Fix the algorithm Retrieve (Algorithm Alg. 1) and the HNSW query relation $\mathcal{R}_{\text{HNSW}}$ from $\mathcal{SH}$, which holds iff the claimed output (topk, ℓ_q) is consistent with running Algorithm Alg. 1 on $(t_{\mathcal{D}}, t_{\mathcal{H}})$ and query q with public hyperparameters pp_R . We prove completeness, soundness, and zero knowledge of zkRAG.

Completeness. Let $pp \leftarrow \text{zkRAG.Setup}(1^\lambda)$ and $(\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}) \leftarrow \text{zkRAG.Commit}(t_{\mathcal{D}}, t_{\mathcal{H}}; pp, r)$. For any query q , the honest prover runs Retrieve($t_{\mathcal{D}}, t_{\mathcal{H}}, pp_R, q$) to obtain (topk, ℓ_q) , and constructs a witness w consisting of the execution trace variables required by $\mathcal{R}_{\text{HNSW}}$ (e.g., $\text{ep}, C_{\text{final}}, R_{\text{final}}, t_S, \dots$) together with the openings information tying these variables to the committed tables (i.e., the corresponding lookup witnesses into $t_{\mathcal{D}}, t_{\mathcal{H}}$). By construction, $\mathcal{R}_{\text{HNSW}}(\ell_q, d, K, t_{\mathcal{D}}, t_{\mathcal{H}}, \text{ep}_{\text{default}}, \text{topk}; w) = 1$. The zero-knowledge compilation produces a proof π for the statement

$$x := (\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}, pp_R, q, \text{topk}, \ell_q)$$

asserting the existence of such a witness w satisfying $\mathcal{R}_{\text{HNSW}}$ and the commitment-consistency constraints. Hence, by completeness of the underlying zkSNARK zkRAG, the verifier accepts with probability 1:

$$\Pr [\text{zkRAG.Verify}(\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}, pp_R, q, \text{topk}, \ell_q, \pi; pp) = 1] = 1.$$

Soundness. Suppose for contradiction that there exists a PPT adversary $\mathcal{A}$ that outputs $(t_{\mathcal{D}}^*, t_{\mathcal{H}}^*, pp_R^*, q^*, \text{topk}^*, \ell_q^*, \pi^*, r^*)$ such that the verifier accepts, yet $(\text{topk}^*, \ell_q^*) \neq \text{Retrieve}(t_{\mathcal{D}}^*, t_{\mathcal{H}}^*, pp_R^*, q^*)$. Let $(\text{com}_{t_{\mathcal{D}}}^*, \text{com}_{t_{\mathcal{H}}}^*) := \text{zkRAG.Commit}(t_{\mathcal{D}}^*, t_{\mathcal{H}}^*; pp, r^*)$. Verifier acceptance implies that π^* is a valid zkSNARK proof for the statement $x^* := (\text{com}_{t_{\mathcal{D}}}^*, \text{com}_{t_{\mathcal{H}}}^*, pp_R^*, q^*, \text{topk}^*, \ell_q^*)$, i.e., there exists a witness w^* such that the compiled relation holds. By knowledge-soundness of zkRAG, from any adversary that produces such an accepting π^* one can extract (with non-negligible probability) a witness w^* satisfying the relation. In particular, w^* must satisfy $\mathcal{R}_{\text{HNSW}}(\cdot) = 1$, which by definition of $\mathcal{R}_{\text{HNSW}}$ implies that $(\text{topk}^*, \ell_q^*)$ equals the output of running Algorithm Alg. 1 on $(t_{\mathcal{D}}^*, t_{\mathcal{H}}^*)$ and q^* with parameters pp_R^* , i.e., $(\text{topk}^*, \ell_q^*) = \text{Retrieve}(t_{\mathcal{D}}^*, t_{\mathcal{H}}^*, pp_R^*, q^*)$, contradicting the assumption. Therefore, the adversary's success probability is negligible.

Zero Knowledge. We construct a simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ for the Real/Ideal experiments in Definition 5. On input $(1^\lambda, pp)$, $\mathcal{S}_1$ outputs commitments $(\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}})$ distributed identically to the real scheme (either by honestly running zkRAG.Commit on some dummy tables if the commitment is hiding, or, equivalently, by sampling from the commitment distribution as guaranteed by the commitment's hiding interface). Given $(pp, \text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}, pp_R, q)$, $\mathcal{S}_2$

first obtains the public output $(\text{topk}, \ell_q) = \text{Retrieve}(t_{\mathcal{D}}, t_{\mathcal{H}}, pp_R, q)$ via its oracle access (as in the Ideal experiment). It then invokes the zkRAG’s simulator for the compiled relation to generate a simulated proof π for the statement $x = (\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}, pp_R, q, \text{topk}, \ell_q)$ without using any witness. By the zero-knowledge property of the underlying zkSNARK zkRAG, the joint distribution of $(\text{com}_{t_{\mathcal{D}}}, \text{com}_{t_{\mathcal{H}}}, q, \text{topk}, \ell_q, \pi)$ in the Ideal experiment is computationally indistinguishable from that in the Real experiment.

Combining the three properties, zkRAG satisfies Definition 5 and hence is a zero-knowledge verifiable RAG retrieval scheme. $\square$

J Additional Experiment Setup and Data

J.1 The Offline Preprocessing Time

Table 5: Preprocessing time t of different N and d , where N is the dataset size, and d is the embedding dimension.

N	d	t
2^{15}	2^7	1 hr
2^{18}	2^7	11 hrs
2^{20}	2^6	23 hrs
2^{20}	2^7	48 hrs
2^{20}	2^8	100 hrs

J.2 Performance of zkRAG on Different d

Table 6: Performance of zkRAG when varying the embedding dimension d . $|\mathcal{P}|$, $|\mathcal{V}|$, and $|\pi|$ denote the prover time, verifier time, and the proof size.

d	$ \mathcal{P} $	$ \mathcal{V} $	$ \pi $
2^6	43.7 s	0.99 s	0.5 MB
2^7	51.5 s	1.00 s	0.5 MB
2^8	64.6 s	1.01 s	0.5 MB